Università degli Studi di Trieste

Dipartimento di Matematica, Informatica e Geoscienze - MIGE

Corso di Laurea in Intelligenza Artificiale e Data Analytics

Tesi di laurea triennale

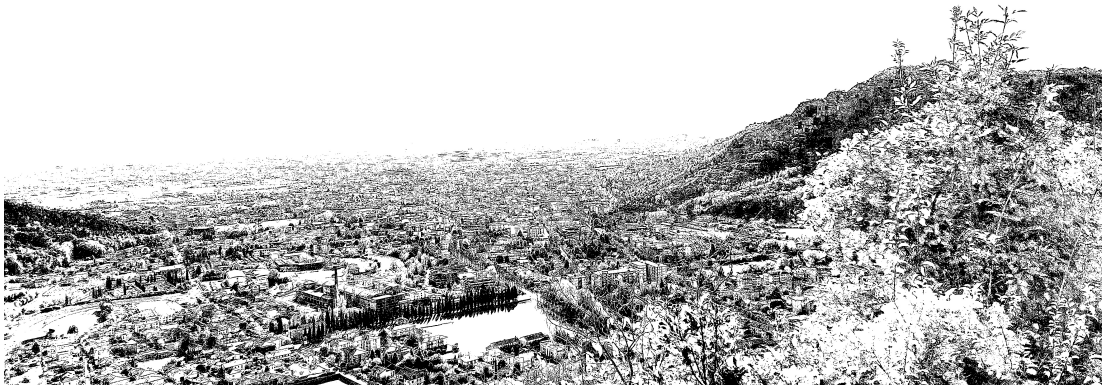
Hybridizing Deep Learning and Classical Mathematical Modeling: theory and new applications

Laureando:
Dino Meng
Matricola: SM3201466

Relatore:
Prof. Alberto d'Onofrio

ANNO ACCADEMICO 2025–2026

Alla mia famiglia, per cui tutto è stato possibile,
Ai miei compagni di svago di Vittorio (e dintorni),
Ai colleghi universitari con cui ho fatto amicizia



«All models are wrong, but some are useful.»

George Box

Contents

List of Figures

List of Tables

List of Code Blocks

List of Main Acronyms

UDE Univeral Differential Equation

NODE Neural Differential Equation

ODE Ordinary Differential Equation

UODE Universal Ordinary Differential Equation

NODE Neural Ordinary Differential Equation

ML Machine Learning

ANN Artificial Neural Network

ReLU Rectified Linear Unit

SiLU Sigmoid Linear Unit

RK4 Runge–Kutta Algorithm of 4th Order

MSE Mean Square Error

Introduction

Recent advances in Deep Learning popularized the application of Neural Networks in various contexts, such as Natural Language Processing for chat bots and translation services, or Computer Vision for facial recognition, object detection and self-driving cars. These types of applications were made possible by the availability of vast quantities of data, as neural networks are data-driven models; in fact, one could say that data is currently the "new oil" [36].

However, there are other fields which can benefit from the applications of Neural Networks, particularly scientific fields such as biology, physics and engineering: in this case, rather than relying on a very large amount of data, we require an embedding of physical knowledge in the Neural Networks in order to have an interpretable and robust model. And so, in the last decade there have been new designations of hybrid models; one particularly promising and recent learning model is the *Universal Differential Equations* (UDEs), introduced by C. Rackauckas et al. in 2020 [34].

The main goal of the thesis is to apply the UDEs to various models in biology, neurosciences and electronics. Not only we will apply the UDEs with the end of obtaining hybrid models, but we will also experiment with them in different scenarios, such as a small or big dataset, or using noisy measurements, as well as applying optimization and regularization techniques that are traditionally used on Neural Networks.

THESIS STRUCTURE This BSc thesis is organized as follows.
Chapter 1 consists of a literature review of various articles and textbooks which are necessary to provide and understand the necessary methodologies and theory behind the experiments that will

follow. The literature review includes the topics of Machine Learning, Physics-Informed Machine Learning, UDEs, Symbolic Regression, and various mathematical models which are relevant for this thesis.

Then in chapter 2 we will convey the essence of the UDEs, starting from a basic idea of the methodology and applying it to study a variant of the logistic model [2]. Successively, we will reproduce the Lotka-Volterra experiment of [34] with some slight modifications.

As soon as we have a clear grasp on the essence of the UDEs, we will dedicate the next chapters in applying this methodology to more advanced cases. In fact, with chapter 3 we will study the performance of the UDEs in learning the generalized logistic model and extrapolating the dynamics with symbolic regression. In particular, in this chapter we will observe the performances in various scenarios, such as different initial values, dataset sizes, amount of training trajectories and presence of noise.

Then with our first advanced case study in chapter 4, we will apply UDEs to neuron modeling as explained in [43] and in [16]. Particularly, we will focus on the capabilities of UDEs of extrapolating dynamics from a limited amount of data and reconstructing the parameters of the dynamics, such as the weight adjacency matrix.

The final advanced case study in chapters 5 and 6 will be about chaotic systems, considering in particular Chua model [6] with the cubic non-linear term; as before, we will leverage UDEs in order to learn the hidden nonlinear terms of the differential equation, and see how "well" it can recover the dynamics of a chaotic model through symbolic regression and generalize the trajectory considering the complex behaviors.

We will dedicate chapter 7 for the details on the implementation of various experiments, discussing the used programming languages, libraries and frameworks.

Chapter 8 will conclude this work with some final remarks on the topic, especially discussing the limits, strengths and future potentialities of UDEs.

Chapter 1

Literature Review

In this chapter we will have a general overview on the topics necessary to understand the work performed in the thesis.

1.1 Machine Learning and Dynamical Systems

MACHINE AND DEEP LEARNING *Machine Learning* (ML) is the field of study concerned with studying algorithms designed to discover hidden patterns in data and extrapolate them, possibly creating a model that is able to generalize the learned pattern on unseen data [27].

Supervised learning is the most common form form of Machine Learning [45]. It consists of learning a function from an input $x \in \mathcal{X}$ (or also known as "features") to an output $y \in \mathcal{Y}$. In practice, given a set of inputs with their relative outputs $\{x_i, y_i\}_{i=1}^N \subset \mathcal{X} \times \mathcal{Y}$ and a parametrized function $f_\theta : \mathcal{X} \longrightarrow \mathcal{Y}$, we want to find the parameter that minimizes the loss function $\mathcal{L}(\theta) = \mathcal{L}(\{f_\theta(x_i), y_i\})$. In simpler terms, the loss function determines the "error" between the predictions and the real outputs. To summarize, supervised learning can be represented as the following minimization problem:

$$\min_\theta \mathcal{L}(\theta) \tag{1.1}$$

We say that a supervised learning problem is a regression problem if the output set is a subset of $\mathbb{R}$ or $\mathbb{R}$ itself, otherwise if it is a discrete set (such as $\{0, 1\}$) then we have a classification problem.

A motivating and simple example of a ML supervised algorithm is

the linear regression, which given the features $\mathbf{x} = (x_1, \ldots, x_n) \in \mathbb{R}^n$ it learns the line function (eq. 1.2) by minimizing the Mean Squared Error (MSE) loss defined as in equation 1.3.

$$\hat{y}(\underline{\beta}) = \beta_0 + \beta_1 x_1 + \ldots + \beta_n x_n = \langle \underline{\beta}, (1, \mathbf{x}) \rangle \tag{1.2}$$

$$\mathcal{L}_{\texttt{MSE}} := \frac{1}{N} \sum_{n=1}^{N} (y_n - \hat{y}_n)^2 \tag{1.3}$$

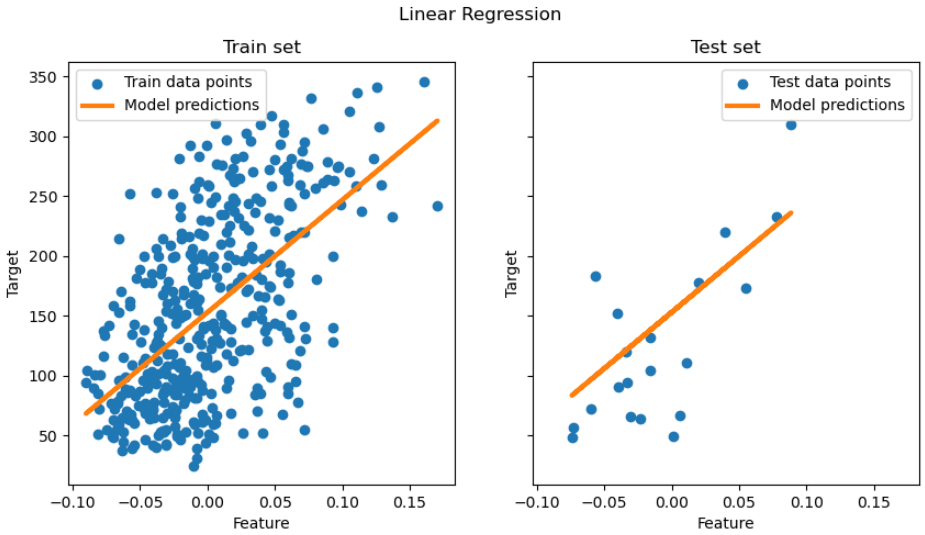


Figure 1.1.1: Example of a Linear Regression model. Figure directly taken from the Scikit-Learn documentation website: https://scikit-learn.org/stable/auto_examples/linear_model/plot_ols_ridge.html

One of the most important class of algorithms taken into consideration are the so-called *Artificial Neural Networks* (ANN), which has origins with F. Rosenblatt's perceptron model in 1958 [1], in an attempt to create a mathematical representation of learning in biological systems. We call *Deep Learning* the subset of Machine Learning that deals with such models.

Cybenko's Universal Approximation theorem [3] motivates said subfield, as it implies that any sufficiently regular function can be approximated by a Neural Network with sigmoidal activation functions.

As a matter of fact, Deep Learning became very popular in the last decade as there have been recent and interesting applications in many practical fields. One famous case is the so-called *Large*

Language Models, whose foundations were laid by the transformer architecture [47], which are able to classify and predict human language (text generation). Image classification is also a notable application of Deep Learning, which has its origins with the residual neural network architecture [18]; the current (2026) state of the art models in image classification are the YOLO [20] and RF-DETR models [53], which are also capable of real-time object detection, tracking and segmentation.

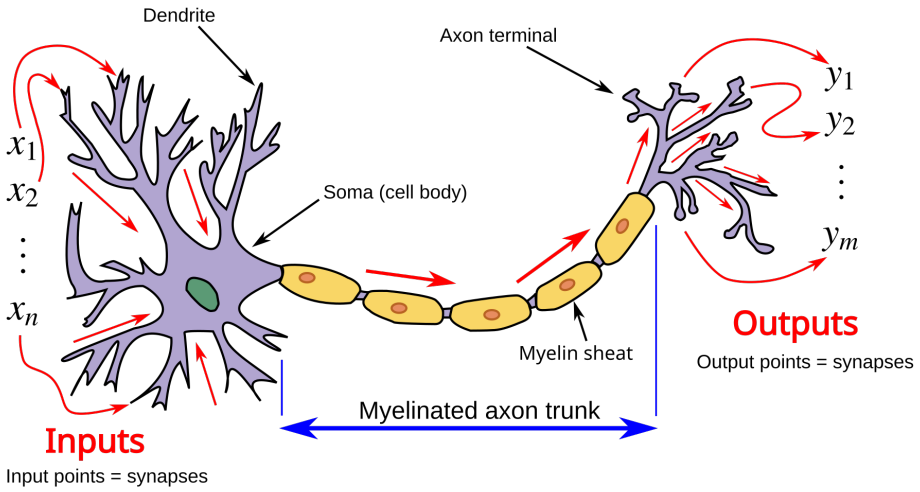


Figure 1.1.2: A biological neuron: an idea. Figure is produced by Egm4313.s12 (Prof. Loc Vu-Quoc) - Own work, CC BY-SA 4.0, https://commons.wikimedia.org/w/index.php?curid=72816083

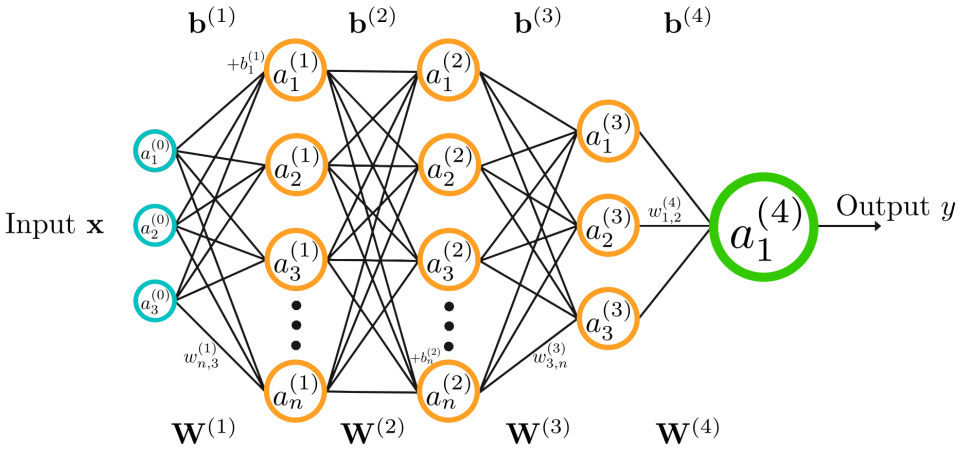


Figure 1.1.3: Example of a feedforward neural network with four layers. By QuantuMechaniX8 - Own work, CC0, https://commons.wikimedia.org/w/index.php?curid=166026387

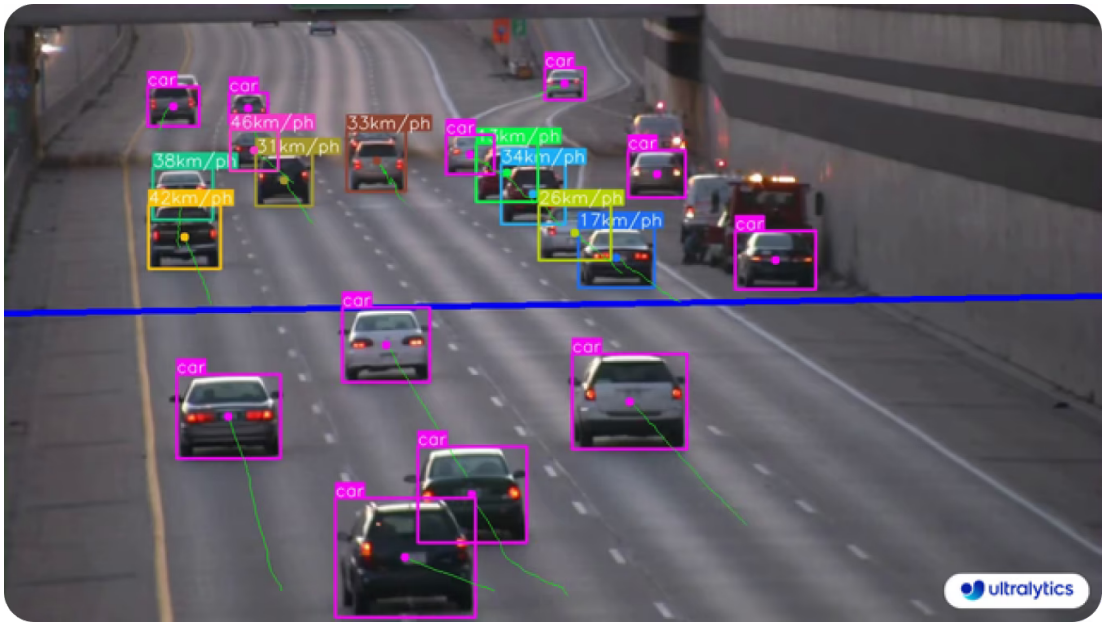


Figure 1.1.4: Example of a recent application of Deep Learning: speed estimation from videos. This figure is directly taken from the Ultralytics YOLO documentation website: https://docs.ultralytics.com/guides/defining-project-goals

DYNAMICAL SYSTEMS In applied mathematics, dynamical systems are systems that evolve in time and are applied to study the behavior of some systems in sciences or engineering. In fact, dynamics was born as a branch of physics with Newton's conception of differential equations (DEs) [25]. For example, Newton's second law of motion expresses the force as $F = ma$, which is a second-order ODE:

$$F(x(t), t) = m\frac{\mathrm{d}^2 x}{\mathrm{d}t^2}(t)$$

Mathematically, there are two ways to describe a dynamical systems: DEs and recurrence relations. We will only study ODEs in this thesis, although the other case can exhibit interesting behaviours.

HYBRID MODELS However, the two subject matters mentioned above are not completely independent from each other; rather, they can be joined to form more robust predictive models, or conversely, obtain meaningful insights about dynamical systems for mathematical modeling.

Physics-informed machine learning is the field that concerns itself with integrating mathematical models into ML learning algorithms [39]. In the next section, we will dive into the specifics of this field, alongside with some examples of physics-informed models.

1.2 Physics-Informed Machine Learning

Problems in ML and classical mathematical modeling represent two different scenarios in terms of availability in data and physics knowledge: with Machine Learning, in particular with Deep Learning, we have the big data regime where a lot of observations are available, albeit without any knowledge about the governing laws. Conversely, in the classical paradigm only the most essential data about the problem is known, that is the initial and the boundary conditions, however the equations describing the physics of the problem is completely known [39].

In a certain sense, the two paradigms represent the two extremes in terms of data and physics availability (fig 1.2.5). However, there is also an intermediate class of problems where the governing equations is partially known as well as there are some measurements of the system. [39]

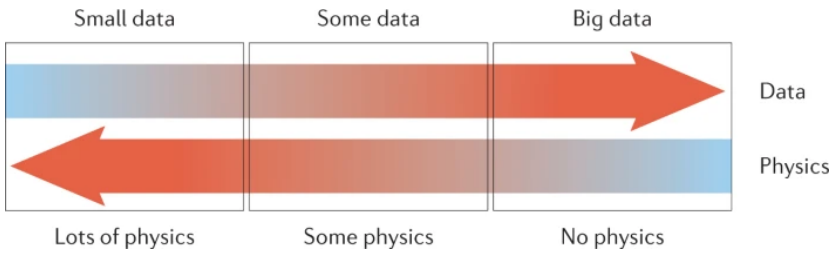


Figure 1.2.5: The spectrum of scenarios of data and physics availability. This figure is directly taken from [39]. The figure is reproduced with permission from Springer Nature

Thus, one of the main challenges in this scenario is to embed mathematical modeling in ML algorithms in order to obtain a more robust, interpretable and better performing model. As in [39], this process is also known as Physics-informed Machine Learning.

G.E. Karniadakis et al. [39] mentions three biases that can guide a

learning algorithm in favoring solutions that are physically consistent (fig. 1.2.6). The first one are the observational biases, which is realized by directly introducing physically-consistent data; a way to do this is through the so-called process of data augmentation. Then there are inductive biases, which consist of modifying a ML architecture in a way that the physical constraints are followed. Finally, learning biases can be introduced by intervening the learning process of an algorithm: for instance, they can be introduced with an appropriate choice of the loss function.

Some instances of physics-informed learning algorithms are Physics-informed Neural Networks [29], Neural Ordinary Differential Equations [24] and Universal Differential Equations [34]. The case of Universal Differential Equations (UDEs) will be discussed in the next sections.

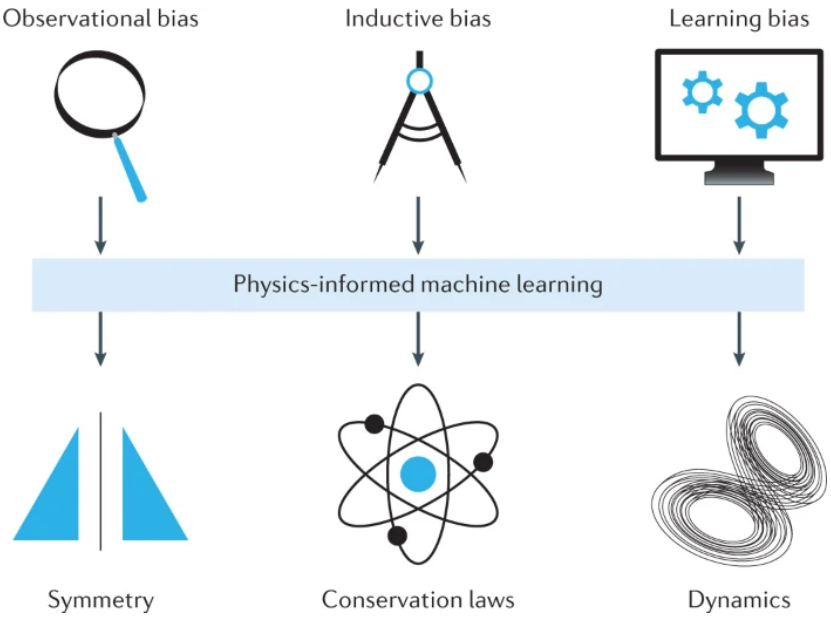


Figure 1.2.6: Three pathways to achieve a phyiscs-informed learning algorithm. This figure is directly taken from [39]. The figure is reproduced with permission from Springer Nature

1.2.1 Universal Differential Equations

THEORY OF UDEs The *Universal Differential Equations* have been proposed in [34] as a hybrid model that is able to incorporate scientific knowledge (under the form of mechanistic models) into learning

algorithms.

To quote the same paper where this hybrid model is proposed, the definition of UDEs is as follows: *"Universal Differential Equations are differential equation models where part of the differential equation contains an embedded UA (Universal Approximator)".*

In other words, we are incorporating learning models into the mathematical models themselves. In this work, we will focus on the motivating case of Ordinary Universal Differential Equations (UODEs), which is defined as follows:

$$x'(t) = f(x, t, U_\theta(x, t)) \tag{1.4}$$

In equation 1.4, f denotes a mathematical model, where some unknown dynamics are represented with the universal approximator U_θ. In this work, we will always take the approximal universal as a neural network.

Given a training set of points $\mathcal{D} \subseteq \mathbb{R}^+ \times \mathbb{R}^n$ represented as a time series, we can train an UODE in two main ways: the simplest way being to use differentiable numerical solvers, such as the Euler's method, and directly back propagate the loss to the universal approximator; otherwise, in [34] the author proposes a differentiable programming framework in order to handle more type of UDEs, such as Universal Stochastic Differential Equations, Universal Partial Differential Equations or Universal Delay Differential Equations.

Rackauckas et al. [34] argues that UDEs are robust and interpretable models that can be applied to problems related to classical modeling, such as identifying equations from a small dataset and accurately extrapolate the dynamics, or alternatively provide a surrogate for a faster, more accurate and consistent simulation of the dynamics.

In a certain sense, the Universal Differential Equations are a generalization of the Neural Ordinary Differential Equations [24], which is defined as the following:

$$\dot{x}(t) = U(x, t; \theta) \tag{1.5}$$

The generalization lies in the part where we incorporate an additional "physical" term $F(x,t)$ to equation 1.5 in the UDEs, which allows the hybrid model to leverage the known physics of the dataset even further; with this, we would have better interpretability, robustness and extrapolation abilities of the model.

CASE STUDY WITH PREDATOR-PREY DYNAMICS As such, in [34] C. Rackauckas et al. propose the first case study of UODEs, which entails as follows. Consider the following system of ODEs, also known as Lotka-Volterra equations:

$$\begin{aligned} \dot{x} &= \alpha x - \beta xy \\ \dot{y} &= \gamma xy - \delta y \end{aligned} \tag{1.6}$$

Where $\alpha, \beta, \gamma, \delta$ are positive factors.
Assuming that we only know the terms α, δ we can formulate the following UODE which replaces the interaction term xy with two neural networks:

$$\begin{aligned} \dot{x} &= \alpha x - U_1(x,y) \\ \dot{y} &= U_2(x,y) - \delta y \end{aligned} \tag{1.7}$$

Then as the UODE is trained, the author performed sparse symbolic regression with the SINDy algorithm [19] in order to recover the unknown interaction equations by using the learnt $U_{1,2}$ terms to obtain more training data and to estimate the derivative.

According to the supplementary material of [34], the following configuration has been used: 31 training data points have been sampled with constant step size $\Delta t = 0.1$, from the simulation of equation 1.6 with $\alpha = 1.3, \beta = 0.9, \gamma = 0.5, \delta = 1.8$ and $(x_0, y_0) = (0.44249296, 4.6280594)$. The UAs U_1, U_2 are two ANNs consisting of one input layer, three hidden layers with 32 neurons each with the swish activation function (i.e. SiLU) and an output layer.
The UDE was initially trained for 1000 iterations with ADAM using the learning rate of 0.03, and then for 10000 iterations with gradient descent using the learning rate of 0.0004. The loss chosen for back propagation was the MSE L2 loss defined as the average of the square distances between the output and the data points in the L2

norm:

$$\mathcal{L} = \frac{1}{N} \sum_{i=1,\dots,N} \|\hat{\mathbf{x}}(t_i) - \mathbf{x}_i\|_2^2 \tag{1.8}$$

Where $\hat{\mathbf{x}}$ is the solution of the initial value problem 1.7.

Moreover, the author trained a Neural Ordinary Differential Equation on the same dataset, with the same goal.

As a result, Rackauckas et al. reported that UODEs was successful in recovering the original equations and extrapolating the dynamics, while the NDE-based approach failed to recover the correct coefficients and recovered the wrong terms of the dynamics.

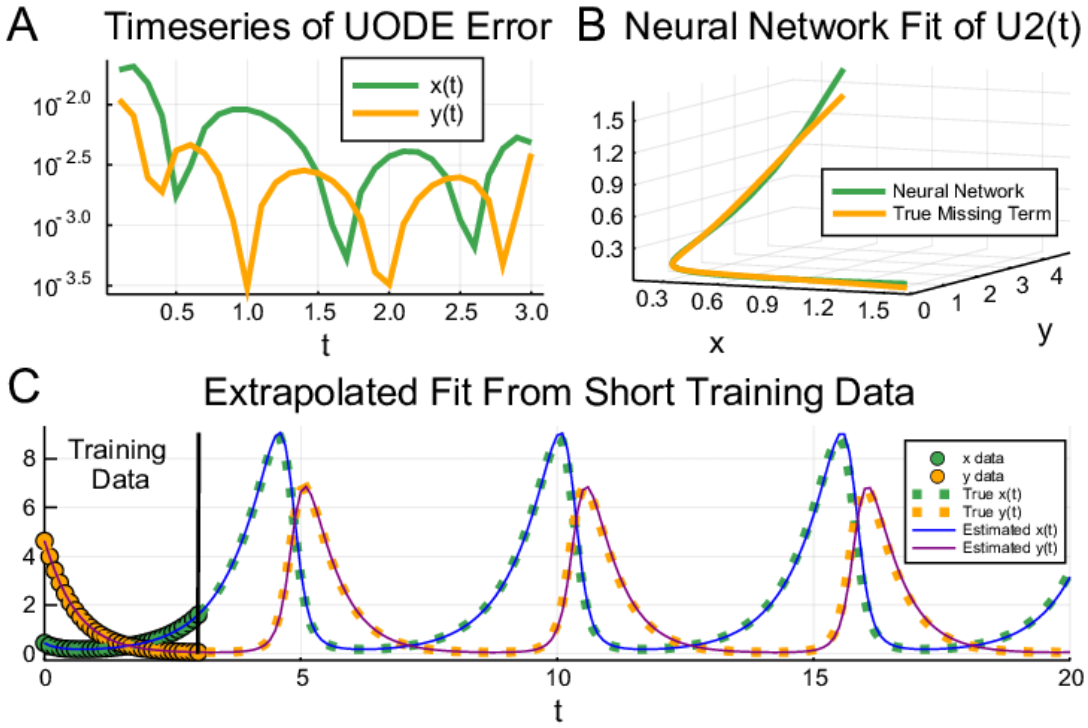


Figure 1.2.7: Result of the Lotka-Volterra's experiment in [34]. The figure is directly taken from [34].

Other similar experiments have been proposed in [34], which leverage the use of Stochastic Differential Equations and other methodologies; as they are out of scope in this work, they are omitted.

1.3 Symbolic Regression with SINDy

One important problem that lies in the intersection between data-driven methods and mathematical modeling is system identification, which consists of identifying the underlying model and extrapolating its dynamics given some measurements of the system.

In [19], Steven L. Brunton et al. present this problem under the assumption that only a few relevant terms compose the dynamics as the sparse identification of nonlinear dynamics (SINDy). More specifically, the paper considers the dynamical systems in the form of differential equations:

$$\dot{\underline{x}}(t) = F(\underline{x}(t)) \tag{1.9}$$

Where $\underline{x} \in \mathbb{R}^n$ represents the state of the system and F the underlying dynamics.

The goal of SINDy is to identify F as accurately as possible, given a time series of the states $\underline{x}(t)$ and its derivatives $\dot{\underline{x}}(t)$.

To do this, we denote with $\mathbf{X}$ and $\dot{\mathbf{X}}$ as the matrix of observations, where each row represents an observation of the system under a certain timestamp. For example, $\mathbf{X}$ has the following structure:

$$\mathbf{X} = \begin{pmatrix} \underline{x}^T(t_1) \\ \vdots \\ \underline{x}^T(t_n) \end{pmatrix} \tag{1.10}$$

The structure is analogous for $\dot{\mathbf{X}}$.

Then we define the candidate of functions that will identify F as the matrix $\Theta(\mathbf{X})$, where each column represents the candidate term calculated on the dataset $\mathbf{X}$. For example, Θ can contain polynomial or trigonometric functions as a basis. With only polynomial functions, the matrix $\Theta(\mathbf{X})$ would look as the following:

$$\Theta(\mathbf{X}) = \begin{pmatrix} \vdots & \vdots & \vdots & & \vdots \\ \mathbf{X}^{P_0} & \mathbf{X}^{P_1} & \mathbf{X}^{P_2} & \dots & \mathbf{X}^{P_n} \\ \vdots & \vdots & \vdots & & \vdots \end{pmatrix} \tag{1.11}$$

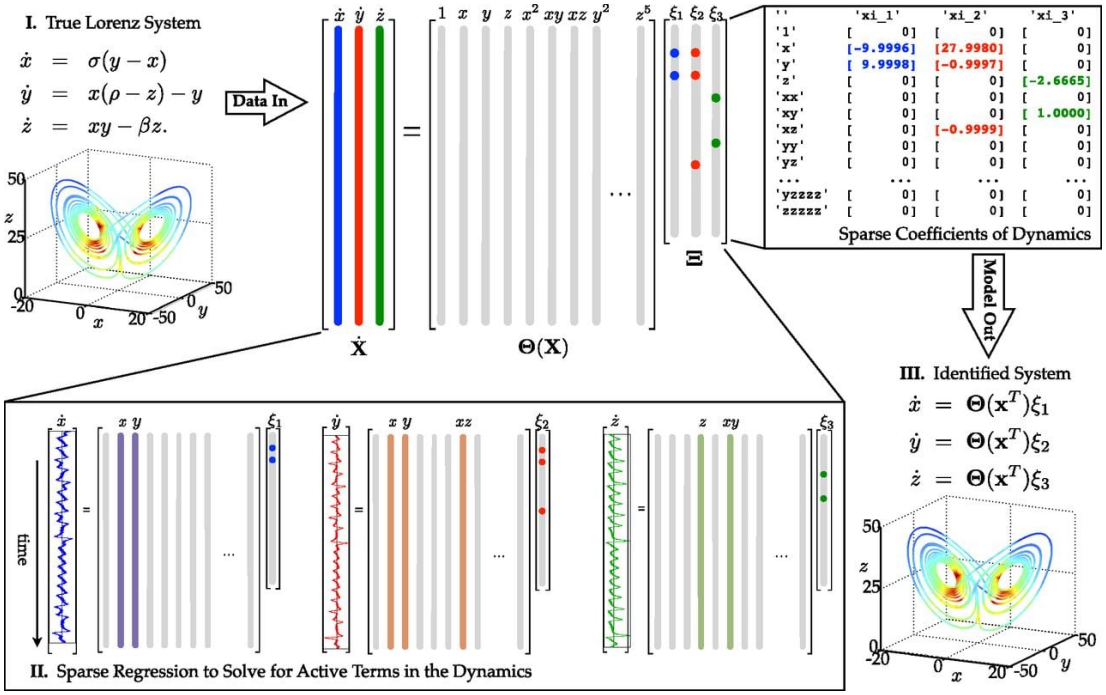


Figure 1.3.8: Idea of the SINDy algorithm, on the Lorentz Equations. Figure directly taken from [19]

Where $\mathbf{X}^{P_k}$ is the matrix that contains all possible polynomials over the dataset $\mathbf{X}$. For example, $\mathbf{X}^{P_2}$ contains all the possible products of the data rows.

To determine the coefficients related to each term, we define Ξ as the vector of coefficients such that the model can be reconstructed as follows:

$$\dot{\mathbf{X}} \approx \Theta(\mathbf{X})\Xi \tag{1.12}$$

The matrix Ξ and its relative coefficient columns ξ_k are found by solving the following optimization, as formulated in [44]:

$$\min_{\Xi} \|\dot{\mathbf{X}} - \Theta(\mathbf{X})\Xi\|_2^2 + R(\Xi) \tag{1.13}$$

Where $R(\Xi)$ is a regularization term to enforce sparsity in the fit.

One potential limitation of the SINDy method is linked to the linearity of equation 1.12. In fact, the core idea of the method lies in the sparse linear regression in order to find Ξ.

To summarize, the essential ingredients for a SINDy regression are the following:

- A sample of observations $\mathbf{X}$
- The derivatives of each observation $\dot{\mathbf{X}}$; however, in most cases this term is unknown and the derivative is usually estimated with numerical derivation or other techniques
- A set of candidates defined in Θ
- An algorithm capable of solving sparse regression, such as the well-known LASSO regression [7] or the SR3 algorithm [30]

1.4 Prediction Error Method Variant of UDEs

Recently, a variant of the UDE model has been proposed for handling chaotic models [51]. The novel approach entails in combining the prediction-error method with the UDE model, by adding an error term $K\varepsilon(t;\theta)$ in the equation, obtaining the following:

$$\begin{aligned} \frac{\mathrm{d}x}{\mathrm{d}t}(t) &= F(t,x) + K\varepsilon(t) + U(x;\theta) \\ \varepsilon(t) &= \hat{x}(t) - x(t) \end{aligned} \tag{1.14}$$

Where F, U respectively represent the known physics and universal approximator of the UDE. In regard to the new terms, we have $\hat{x}(t)$ and K. $\hat{x}(t)$ represents an interpolation of the data at time t and $K \geq 0$ is a hyperparameter representing the error gain factor. We will call the model defined in equation 1.14 as PEM-UDE.

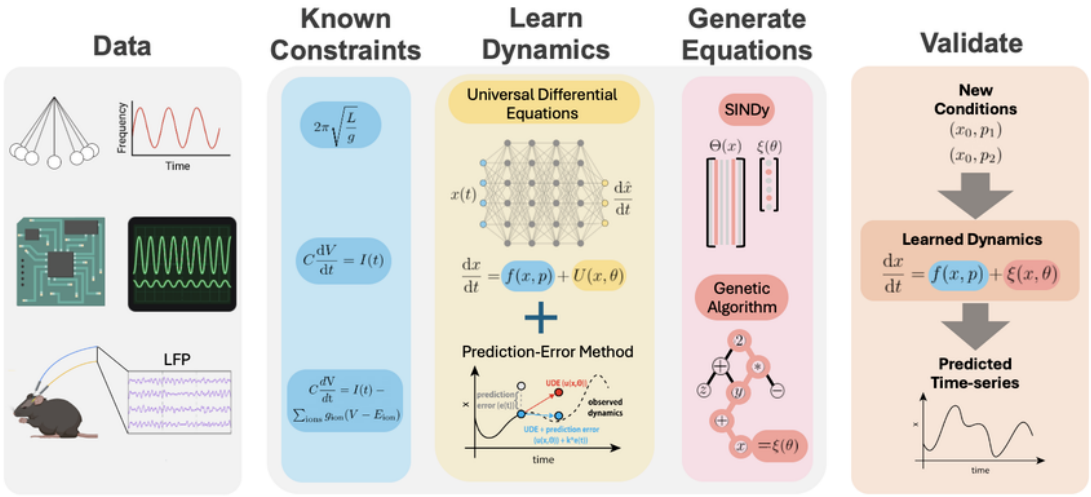


Figure 1.4.9: Schematic idea of the PEM-UDE variant. Figure directly taken from [51]

The model was proposed to handle scenarios where chaotic properties are present since that the prediction error term performs a smoothing on the loss landscape of the neural network, making it more approachable and less reliant on initial conditions [51].

In fact, in paper [51] it is shown that the PEM-UDE variant outperforms the original version in fitting the trajectories of chaotic systems. In their case, they considered the Rössler attractor, the Petrzela-Polak circuit and spiking neural networks as case studies for the applications of the PEM-UDE variant [51]. A notable example of this result is the training of a PEM-UDE on the Rössler attractor, as seen in figure 1.4.10: in particular, from (a) and (c) of the figure we can see that the PEM-UDE is able to fit the complex dynamics correctly, whereas the original UDE is not even able to reach a satisfactory loss as it stagnates over the value of 10^2.

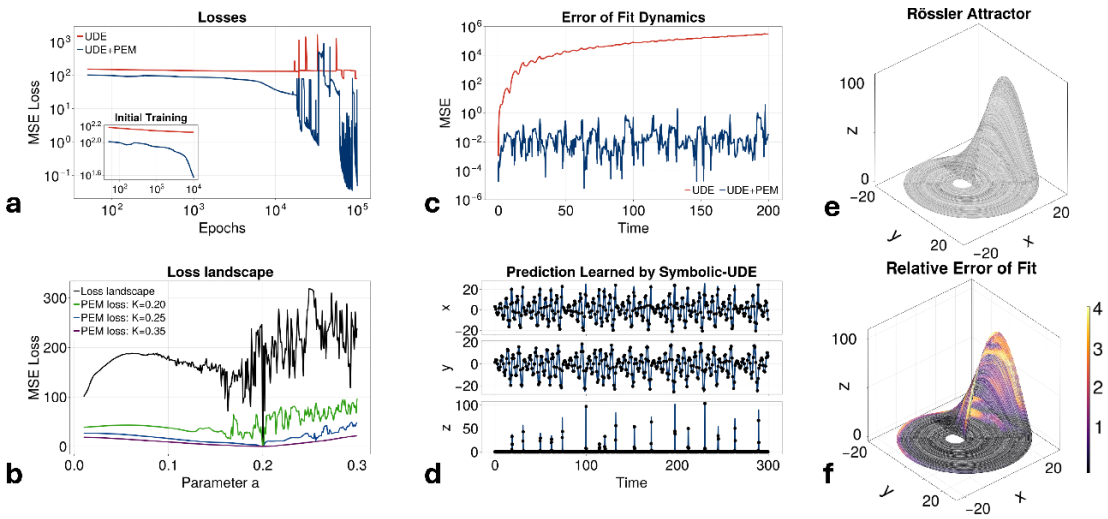


Figure 1.4.10: Example of PEM-UDE application and comparison with original UDE: Rössler attractor. Directly taken from [51]

1.5 Some Mathematical Models

As a famous mathematician once said, a mathematician is a maker of patterns[1]; that is, one of the main applications in mathematics is identifying patterns in everyday life and every field. One of the ways

[1] G. H. Hardy, *A Mathematician's Apology* (1940):

> A mathematician, like a painter or poet, is a maker of patterns. If his patterns are more permanent than theirs, it is because they are made with ideas

to concretize these patterns is with classical mathematical modeling, which translates real-world phenomena into equations. In this section, we will have a brief overview of differential equations that model the most important systems in biology, neurosciences and electronics.

1.5.1 Population Biology

The study of population growths is the most historical instance of applications of mathematical modeling, starting in 1202 with a mathematical model for a growing rabbit population written in an arithmetic book by Leonardo of Pisa [9].

Here we will have a very brief overview on the models describing population growths, which will be subject to application in this thesis.

VERHULST LOGISTIC MODEL In 1798, Malthus described one of the simplest models of population growth [9]:

$$\dot{N} = bN - dN = \rho N \tag{1.15}$$

Where b, d, ρ are respectively the birth, death and reproduction rate of the population. For $N(0) = N_0$, the model described in equation 1.15 yields the following solution:

$$N(t) = N_0 e^{\rho t} \tag{1.16}$$

This suggests an exponential growth of the population, which is fairly unrealistic. In fact, the crucial shortcoming of the model is the fact that it assumes that the resources are infinite, even on the long term.

So, in 1838 Verhulst proposed the model that limits itself when it becomes too large due to the lack of resources with the following model [9]:

$$\dot{N} = rN(1 - N/K) \tag{1.17}$$

Where $r > 0$ is the reproduction rate of the population and $K > 0$ is the carrying capacity, that is the limit of the growth for the

population. Equation 1.17 also yields an open-form solution:

$$N(t) = \frac{N_0 K e^{rt}}{K + N_0(e^{rt} - 1)} \tag{1.18}$$

In 1845, Verhulst named the equation 1.18 as the logistic equation [9].

SPRUCE BUDWORM OUTBREAK MODEL In 1978, D. Ludwig et al. [2] proposed a model for the growth of a population that interacts with an external environment that preys upon the population, according to its density. It is an extension of the logistic model (eq. 1.17) that is subtracted with a functional form $\varphi(x)$ that saturates for $x \to +\infty$:

$$\dot{x} = rx(1 - x/K) - \beta \frac{x^2}{\alpha^2 + x^2} \tag{1.19}$$

It can be extensively shown that the model presents a hysteresis bifurcation according to the parameter β (fig. 1.5.11); that is, there exists two critical values β_1, β_2 such that for $\beta < \beta_1$ and $\beta > \beta_2$, the model presents one stable fixed point; however for $\beta \in (\beta_1, \beta_2)$ the model presents two locally asymptotically stable fixed points and one unstable equilibrium.

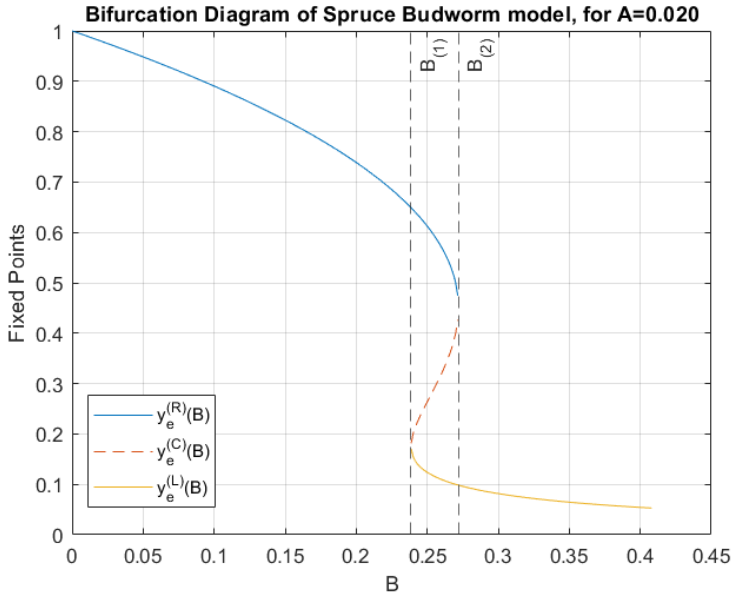


Figure 1.5.11: Bifurcation diagram of the Budworm model (eq. 1.19) for $\alpha = 0.02$

LOTKA-VOLTERRA MODEL Another important class of biological models are the predator-prey dynamics, where we have one population of "preys" x and one population of "predators" y that obey the following equation:

$$\begin{aligned} \dot{x} &= \rho_x(x, y)x \\ \dot{y} &= \rho_y(x, y)y \end{aligned} \tag{1.20}$$

Where $\rho_{x,y}$ are the respective growth rates and η is the conversion rate.

In 1926, Volterra proposed the simplest realization of equation 1.20 making a linear choice for the growth rates [9]:

$$\begin{aligned} \dot{x} &= x(\alpha - \beta y) \\ \dot{y} &= y(\gamma x - \delta) \end{aligned} \tag{1.21}$$

The most notable characteristic about the model described in equation 1.21 is that the system presents periodic solutions, orbiting around the fixed point of the equation (fig. 1.5.12).

Further generalizations of the Lotka-Volterra equations (eq. 1.21) can account for other sort of dynamics, such as in [50] with psychological fear effect and resource limitations.

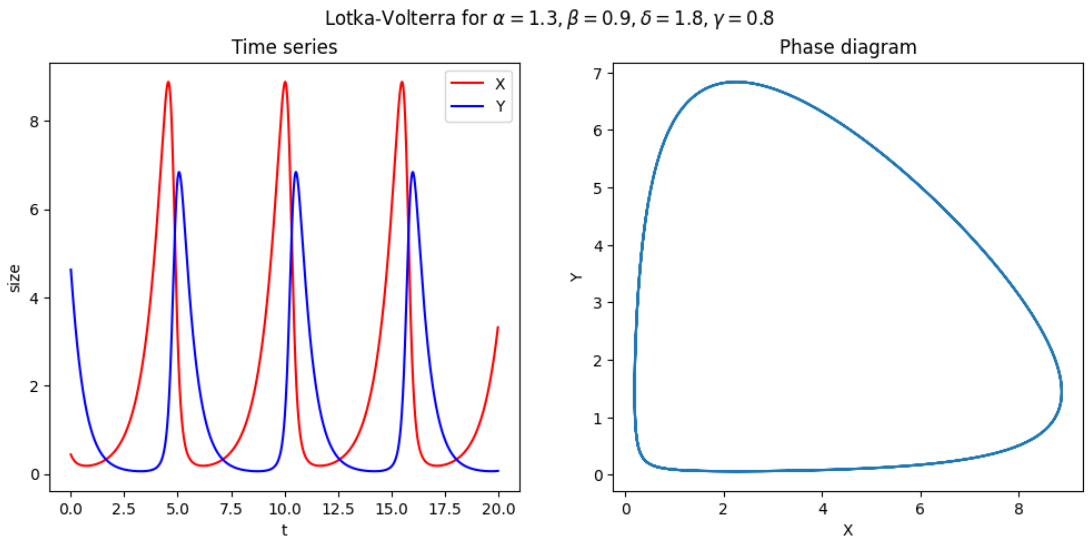


Figure 1.5.12: Periodic Solution of Lotka-Volterra model (eq. 1.21)

1.5.2 Biological Neurons

One major field of interest in mathematical modeling is neurosciences, in particular modeling and studying the dynamics of neurons and network of neurons. Recently there have been major advancements in this field, such as the full-scale simulation of the adult Drosophila fly brain [49], motivating this case study even further.

In [43], two kinds of neuron models have been proposed as case studies: rate models and network of spiking neurons. In this work, we will mainly see the first class of neuronal models.

NEURAL MASS MODELS Neural mass models (also known as rate models) consist of modeling neurons as a population with a continuous variable that obeys an ODE [43], with the variable representing the mean activity of the neurons. The simplest form of a rate model can be described with the following ODE:

$$\tau \dot{r} = -r + \Phi(I_{\text{ext}} + Jr) \tag{1.22}$$

Where in equation 1.22 τ refers to a time constant of the dynamics, Φ is a non-linear transfer function which transforms impulse to firing rate, I_{ext} is the external impulse of the population and J is the strength of the intra-population connections.

We say that the population is excitatory for $J > 0$, inhibitory otherwise. The Φ function can take different forms, with the most popular case being the threshold-linear function (also known as ReLU):

$$\Phi(I) := \max\{0, I - T\} \tag{1.23}$$

Where $T > 0$ of equation 1.23 is a threshold value. In other cases, it can be also defined as a sigmoidal function, as follows:

$$\Phi(I) := \frac{r_{\max}}{1 + e^{-\beta(I-T)}} \tag{1.24}$$

Notice how the equation 1.24 converges to the following function for $\beta \to +\infty$:

$$\lim_{\beta \to +\infty} \frac{r_{\max}}{1 + e^{-\beta(I-T)}} = \begin{cases} r_{\max}, & I > T \\ 0, & I < T \end{cases} \tag{1.25}$$

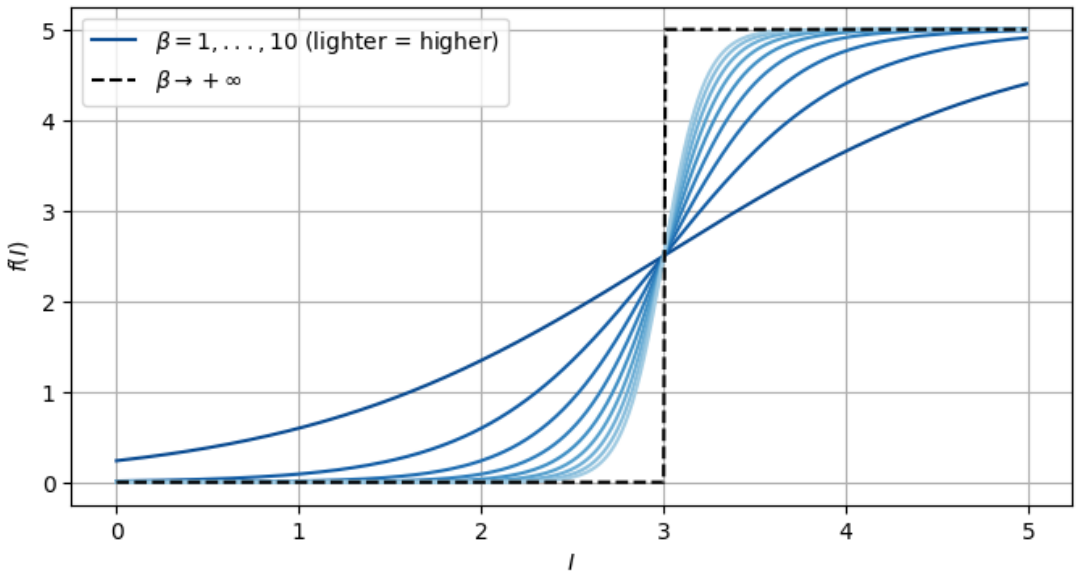


Figure 1.5.13: Idea of the convergence of the parametrized sigmoidal function (eq. 1.24) towards the threshold function (eq. 1.25). Lighter hues of blue indicate a higher value for β

Given more populations of neurons $r_1, \ldots, r_n$, we can extend the firing rate of singular populations as described in equation 1.22 into a model describing the network of coupled neurons.

In [16] the following model has been proposed:

$$\forall i = 1, \ldots, n; \tau \dot{r}_i = -r_i + \Phi\left(\sum_{j=1,\ldots,n} w_{ij} r_j + I_{\text{ext},i}\right) \tag{1.26}$$

Where w_{ij} represents the bond between the population i and j.

It is interesting to observe how in [16] the model described with equation 1.26 was derived from a generalization of the logistic Lotka-Volterra equation as an analogy of neural populations.

As proposed in [43], an alternative way to extend the single-population firing rate model (eq. 1.22) is to introduce a continuous variable x that represents the "location" of the population and rewrite the model as the following integro-differential equation (IDE):

$$\tau \frac{\mathrm{d}r}{\mathrm{d}t}(x,t) = -r(x,t) + \Phi\left(I_{\text{ext}} + \int_\Omega J(x,y) r(y,t)\ \mathrm{d}y\right) \tag{1.27}$$

However, we will adopt the "discrete" approach (i.e. the model described in equation 1.26) in this thesis as it provides a system of

ODEs rather than an IDE.

INTRINSIC AND STIMULUS-DRIVEN DYNAMICS In order to study network dynamics, there are two approaches: intrinsic network dynamics consists of assuming constant and uniform external impulse; whereas the stimulus-driven approach observes the behavior of the network when the external impulse is represented as a time-valued function [43].

WINNER TAKE ALL MECHANISM One particular case study of the intrinsic network dynamics is the so-called winner-take-all mechanism as described in [43]. Consider the following realization of equation 1.26:

$$\begin{aligned} \tau \dot{r}_1 &= -r_1 + \Phi(I_0 - J r_2) \\ \tau \dot{r}_2 &= -r_2 + \Phi(I_0 - J r_1) \end{aligned} \tag{1.28}$$

Where $J > 0$ describes the recurrent inhibition.

It can be shown that the system of DEs 1.28 presents the homogeneous steady state, where $r_1 = r_2 \equiv r_s$ such that $r_s = \Phi(I_0 - r_s)$. However, for a sufficiently large value of J it can be also shown that such steady state is unstable and instead we reach an equilibrium where the activity of either sub-network is enhanced and the other sub-network is suppressed.

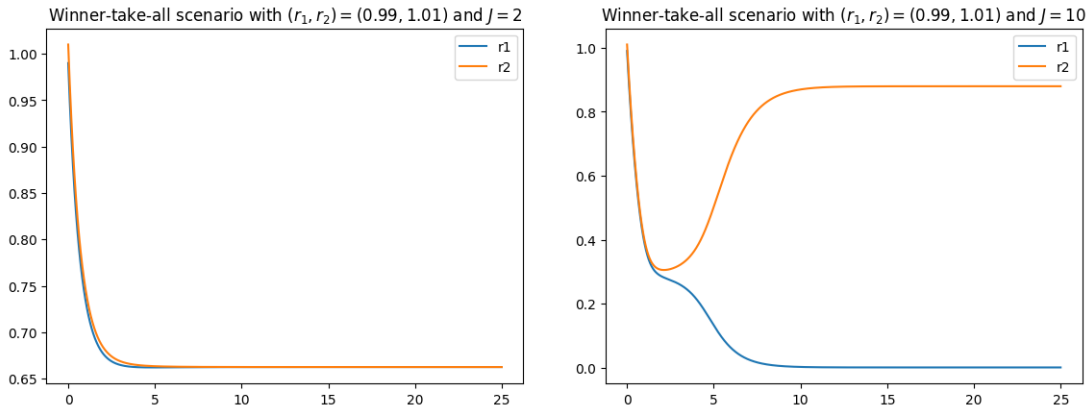


Figure 1.5.14: Two steady states of equation 1.28

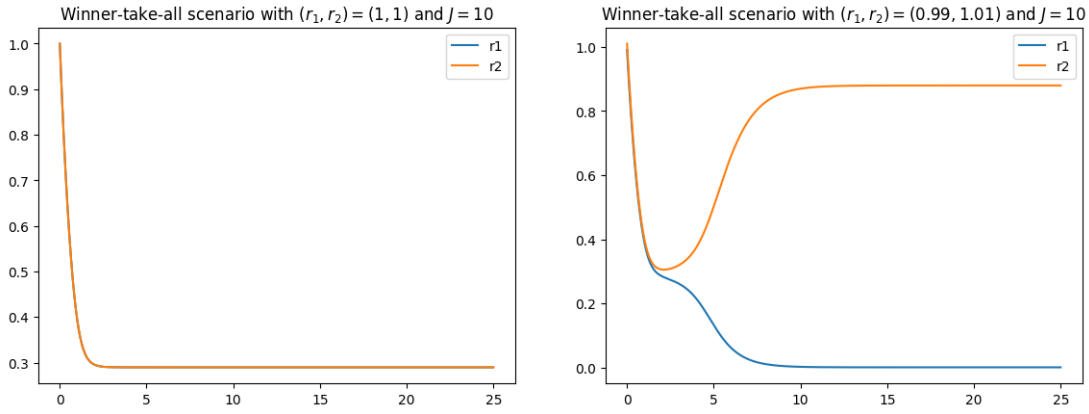


Figure 1.5.15: Instability of the homogeneous steady state under large J of equation 1.28

1.5.3 Chua's Circuit: an electronic circuit that shows chaotic properties

In the study of dynamic systems, one of the most fascinating behavior that one might encounter are the chaotic systems. Generally, chaotic systems are deterministic systems that exhibit non-periodic and non-quasi periodic trajectories and are sensitive to initial conditions [25]. In this subsection, we will review one of the systems whose chaotic behavior was first shown experimentally: the Chua electronic circuit model.

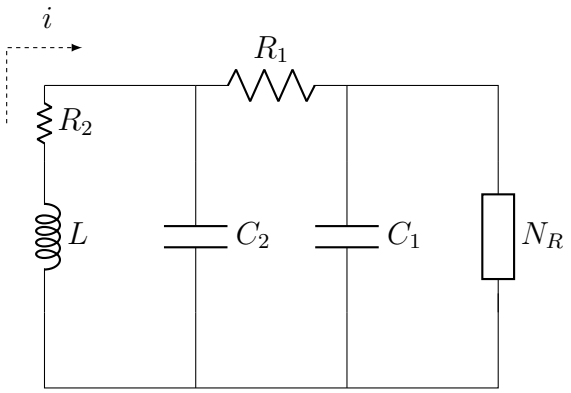


Figure 1.5.16: Chua's Circuit as in [4]

Consider the electronic circuit depicted in figure 1.5.16 which consists of one inductor, two capacitors, two resistors and the Chua's diode, which is a non-linear resistor. From such circuit, we can easily derive the equations for the voltages of the capacitors and the

current in the inductor with Kirchhoff's laws as done in [4]:

$$\begin{aligned} V_1' &= \frac{1}{C_1}\left[\frac{1}{R}(V_2 - V_1) - f(V_1)\right] \\ V_2' &= \frac{1}{C_2}\left[\frac{1}{R}(V_1 - V_2) + i\right] \\ i' &= -\frac{1}{L}V_2 \end{aligned} \tag{1.29}$$

It is possible to rescale the time constants and re-define variables in order to obtain the dimensionless version of equation 1.29, as it has been done in [6]. This process yields the following equations:

$$\begin{aligned} \dot{x} &= \alpha(y - g(x)) \\ \dot{y} &= x - y + z \\ \dot{z} &= -\beta y - \gamma z \end{aligned} \tag{1.30}$$

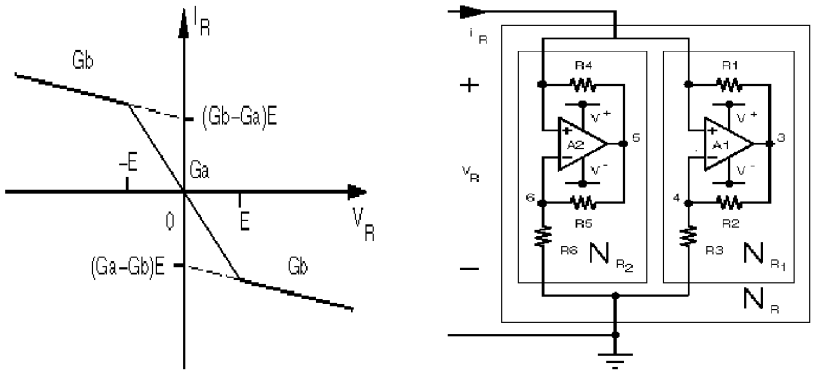
Where x, y, z respectively represent the scaled version of the variables V_1, V_2, i.

Originally, the non-linear term $f(V)$ was a piecewise-linear function defined as in equation 1.31. With this function, many interesting behaviors have been observed, such as strange attractors [4].

$$f(V; a, b) = \begin{cases} bV + a - b, & x \geq 1 \\ aV, & x \in [-1, 1]\ ; a < 0 \\ bV - a + b, & x \leq -1 \end{cases} \tag{1.31}$$

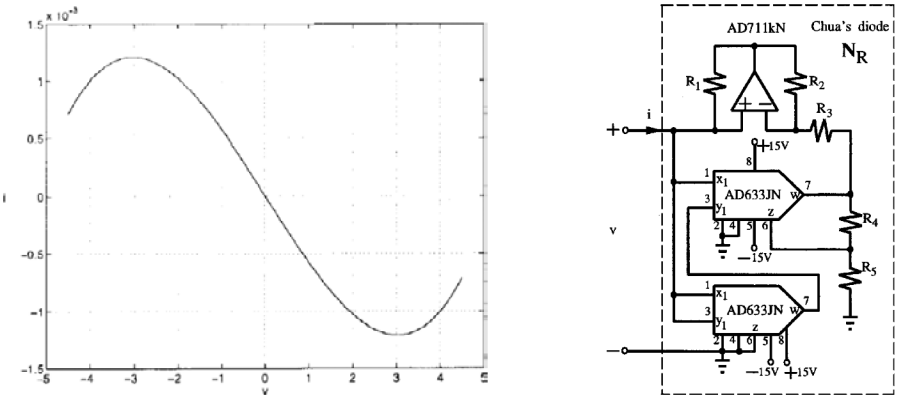
However, in some cases we may prefer to consider the non-linear term $f(V)$ as a cubic polynomial for its ease in modeling and implementation [6], [5], [10] which resembles to the piecewise-linear function as defined in equation 1.31. That is, the cubic term is to be defined as the following:

$$f(V; a, c) = ax^3 + cx \tag{1.32}$$



(a) (b)

Figure 1.5.17: (a) Plot of the piecewise nonlinear component (N_R of fig. 1.5.16) (b) Implementation of the nonlinear piecewise component of Chua's electronic circuit. Directly taken from [10]



(a) (b)

Figure 1.5.18: (a) Plot of the cubic nonlinear component (N_R of fig. 1.5.16) (b) Implementation of the nonlinear cubic component of Chua's electronic circuit. Directly taken from [10]

Although in both of the cases we have more or less the same kind of behavior, it is still to be noted that even with close fits between the cubic and the piecewise-linear function, the two models exhibit different attractors in a microscopic scale [13].

For some specific instances of the model 1.30 with the cubic term, some similar attractors can be found as discovered in the original case [6]: for example, the double-scroll attractor can be observed for the parameters $\alpha = 10, \beta = 16, \gamma = 0, a = 1, c = -0.143$. The article [6] provides an exhaustive list of examples of the model, with

many forms of chaotic, strange and periodic attractors.

Another interesting aspect of the Chua model with the cubic nonlinearity is the self-similarity of the parameter-space diagrams in $\alpha \times \beta$, for the fixed values of $a = 0.03, c = -1.2, (x_0, y_0, z_0) = (0, 0.1, 0.3)$. To quote [12], which observed this interesting phenomenon, one can see the *"islands of periodicity embedded in a sea of chaos"*. This kind of dynamic was classically presented in time-discrete systems (such as the well-known logistic model), making this phenomenon even more interesting.

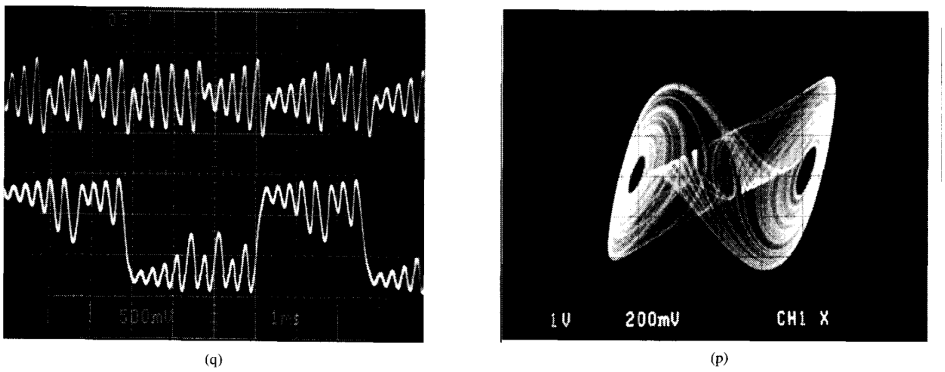


Figure 1.5.19: Experimental measurement of the Chua electronic circuit model, followed from the cubic nonlinearity version of the model. From [5]

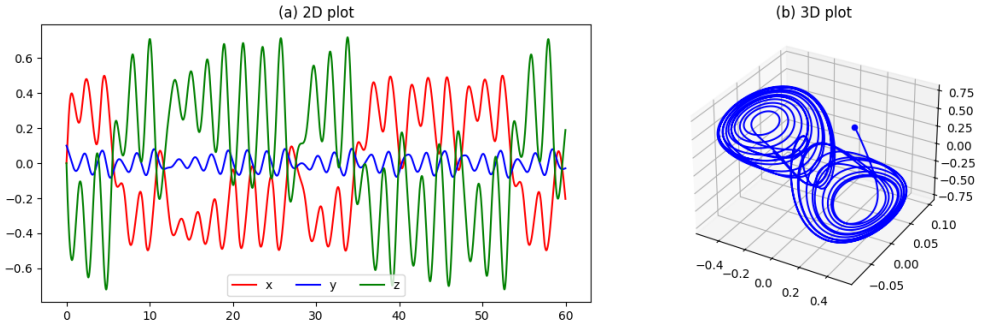


Figure 1.5.20: Computer simulation of the Chua electronic circuit model with parameters $\alpha = 10, \beta = 16, \gamma = 0, a = 1, c = -0.143$. (a) Time series plot (b) Phase plot

Chapter 2

Methodology and Reproduction of the Main Results

In this chapter, we will be reviewing the methodologies to implement the Universal Differential Equations as described in [34].

The chapter will begin with a simple implementation using Euler's method as an Initial-Value-Problem (IVP) solver in order to learn the interaction of the bud worm model (eq. 1.19); then, it will leverage symbolic regression (subsec. 1.3) to learn the polynomial term of the generalized logistic equation (eq. 3.2) enhanced through the use of UDEs.

In the end, this chapter culminates with the reproduction of the Lotka-Volterra experiment of [34] (para. 1.2.1), with some slight modifications.

2.1 The Spruce Bud-worm Model

Recall the bud worm outbreak model as in equation 1.19:

$$\dot{x} = r_0 x(1 - x/K) - \underbrace{\frac{\beta x^2}{\alpha + x^2}}_{\varphi(x)} \tag{2.1}$$

Suppose that a scientist sampled a time series of this model, however does not know the interaction term $\varphi(x)$. Then they can use the Universal Differential Equations to approximate the term $\varphi(x)$ with a neural network $U(x;\theta)$. Then, we obtain the following UDE:

$$\dot{x} = r_0 x(1 - x/K) - U(x;\theta) \tag{2.2}$$

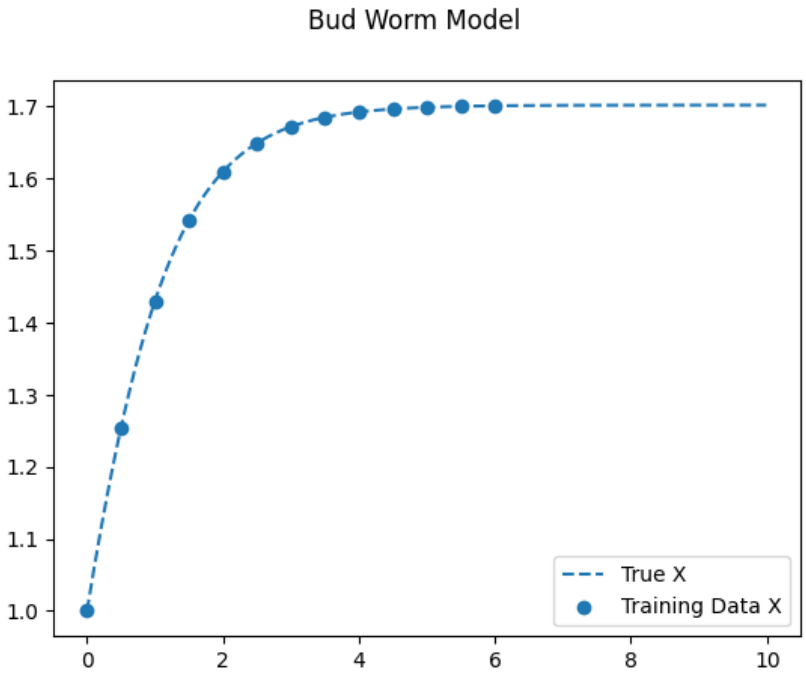


Figure 2.1.1: Budworm Model Training Data. Parameters described in the text.

In the simulation of the model (fig. 2.1.1), the following parameters have been used:

- The initial condition is set to $x_0 = 1$
- The parameters of the original model are set to $r_0 = 2, K = 3, \alpha = 3, \beta = 3$
- The training data consists of a trajectory of 13 data points with a constant step size $\Delta t = 0.5$

Then, to train the Universal Differential Equation the following methods were employed:

- The neural network is a fully-connected neural network consisting of an input layer, a hidden layer with 32 neurons with the SiLU activation function and an output layer.
- The solver used to solve the Initial Value Problem (IVP) 2.2 will be the Euler's method, defined as $x(t_{k+1}) = x(t_k) + hF(U, x, t)$. In particular, the implementation of the Euler algorithm will be compatible with the Pytorch methods for automatic differentiation.
- The model has been trained with the ADAM optimizer for 3000 epochs with a learning rate of 0.01

In the end, we obtained a model that fits the training data with a

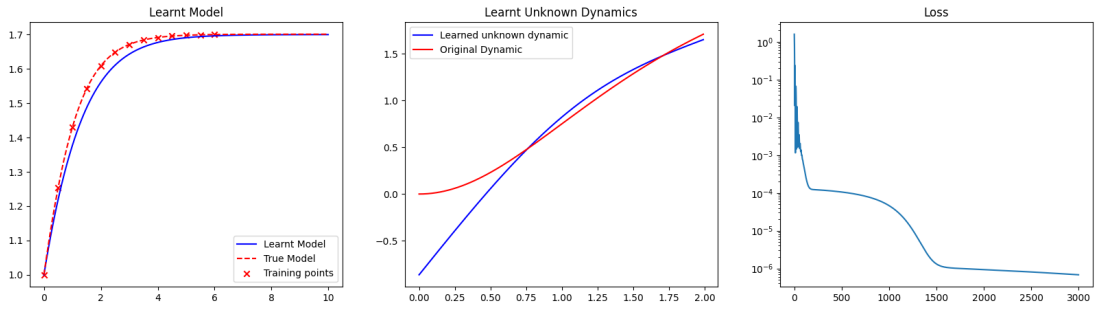


Figure 2.1.2: Budworm UDE Results

final loss of 6.7945×10^{-7}. By observing the graphs of the learned trajectories and unknown dynamics (fig. 2.1.2), we can indeed claim that UDEs hold the potentialities to learn and extrapolate dynamics from small datasets and partially unknown models.

So far, this was a sketch of the methodology used to study models with UDEs: to improve this method and use it in more advanced contexts, we will consider using more accurate differentiable IVP solvers (such as the ones provided by the libraries discussed in chapter 7), or using hyperparameter optimization techniques to find the best ANN architecture for the UDE as well as extrapolating dynamics with symbolic regression.

2.2 Lotka-Volterra Predator-Prey Equations

In [34], the Lotka-Volterra equations have been considered as a case study for UDEs and SINDy symbolic regression. Let us recall the equations (eq. 1.21) and the parameters used in the supplements of [34]:

$$\begin{aligned} \dot{x} &= \alpha x - \beta xy \\ \dot{y} &= -\delta y + \gamma xy \\ (x_0, y_0) &= (0.44249296, 4.6280594) \\ \alpha &= 1.3, \beta = 0.9, \gamma = 0.8, \delta = 1.8 \end{aligned} \tag{2.3}$$

With the model described in equation 2.3, we will perform three experiments:

- The first experiment is aimed at reproducing [34], with a very similar methodology

- The second experiment is aimed at reproducing [34] again, however with some substantial differences from the original methology
- The third experiment changes the parameters of the equation 2.3, in particular changing the constants associated to the interaction term xy.

2.2.1 Experiment 1: A Slightly Modified Reproduction of [34]

In this experiment, we will follow the a similar methodology as covered in [34]. One slight difference is in the UDE's Neural Network architecture, where in addition to the proposed layers, we have added an extra linear layer on the output layer which compresses the output layer to a single variable and from which we obtain a 2-dimensional output with another linear layer. This is done to encode the idea of learning the parameters β, γ implicitly.

Another difference is that we trained the UDE for 1000 epochs with the ADAM optimizer with a learning rate of 8×10^{-3}.

For the rest, let us recall the same methodologies proposed in [34]:

- In addition to the UDE, a Neural Ordinary Differential Equation is trained on the dataset
- The loss is the L^2 mean-squared error metric, that is the one calculated according to equation 2.4
- Three SINDy models will be trained: one is the base SINDy model, the other two are UDE and NDE-enhanced SINDy models where the derivative are calculated with the learnt models
- The SINDy models will be optimized with the Sequentially Thresholded Least Squares (STLSQ) algorithm
- The obtained coefficients from the UDE-enhanced SINDy model will be improved with a post-structure identification parameter estimation method. In our case, we will optimize the loss function defined as the sum of squared errors in the x and y components

$$\mathcal{L} := \frac{1}{N} \sum_{i=1,\dots,N} \|\hat{\mathbf{x}}(t_i) - \mathbf{x}(t_i)\|_2^2 \tag{2.4}$$

The trained UDE model yielded a final loss of 9.759×10^{-4}, while the

NDE yielded a final loss of 3.363×10^{-5}. The recovered coefficients with SINDy are reported in table 2.2.1

Original LV equations	$1.3x - 0.9xy$	$-1.8y + 0.8xy$
Name	**Recovered** $\dot{x}$	**Recovered** $\dot{y}$
Original SINDy	$1.248x + 0.124x^2$ $- 0.885xy - 0.066x^3$	$-1.8y + 0.787xy$
UDE-enhanced SINDy	$1.064x - 0.791xy$	$-1.949y + 1.350xy$
UDE + Parameter Estimation	$1.30000148x - 0.90000097xy$	$-1.8000203y + 0.80007929xy$
NDE-enhanced SINDy	$2.625x - 0.306y - 3.793x^2$ $+ 2.656x^3 - 0.429x^5$	$-1.71y + 0.279x^2 - 0.491xy$

Table 2.2.1: Recovered coefficients

Despite a lower performance in terms of the L2 MSE loss, the UDE-enhanced method managed to correctly recover the terms with a slight discrepancy in the recovered coefficients; however, with an estimation parameter method using the cofficients as an initial guess, we obtained new coefficients that are almost perfectly identical to the original equation. As we can see in figures 2.2.3 and 2.2.4, UDEs are able to extrapolate the dynamics of Lotka-Volterra equations from a short training set.

Although the original SINDy method was fairly accurate in obtain-

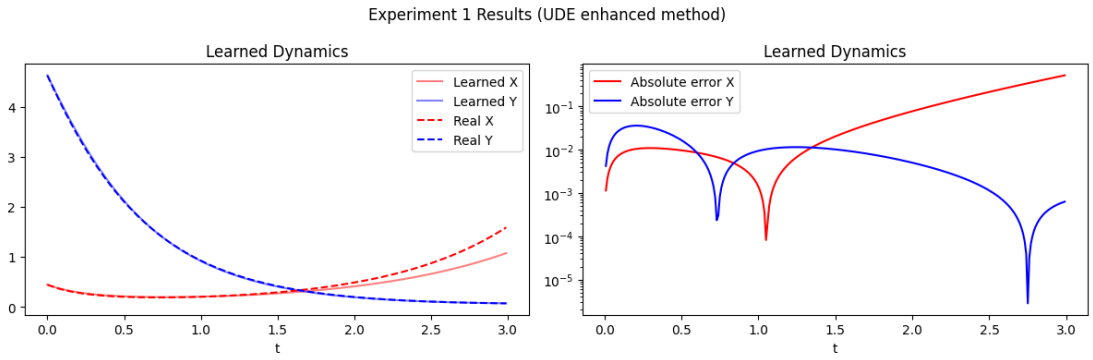


Figure 2.2.3: Simulation of the UDE-enhanced SINDy recovered model

ing the coefficients of the system, it also incorrectly introduced some other factors in the equation for $\dot{x}$, in particular the quadratic and cubic term. This is a rather critical problem, as with such result the scientist can wrongly infer the dynamics of the extrapolated system. To give an idea of this grave error, let us write the full equations

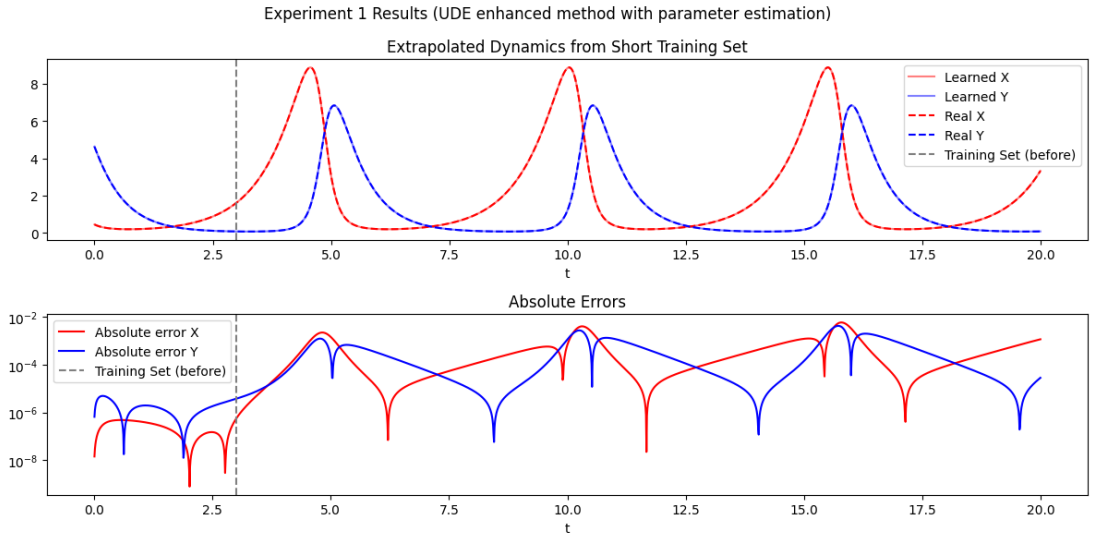


Figure 2.2.4: Simulation of the UDE-enhanced SINDy recovered model, with post-structure identification parameter estimation

for the base SINDy-recovered dynamics:

$$\dot{x} = 1.248x + 0.124x^2 - 0.885xy - 0.066x^3 \tag{2.5}$$

$$\dot{y} = -1.8y + 0.787xy \tag{2.6}$$

Assume that the two populations live in a predator-free scenario, that is we set $y_0 = 0$. Then with some simplifications of equation 2.5 we obtain the following growth model for the prey population:

$$\dot{x} = x(1.248 + 0.124x - 0.066x^2) = x(b(x) - m(x)) \tag{2.7}$$

Where $b(x)$, $m(x)$ indicate the birth and death rate of the population. By plotting the birth and death rates (fig. 2.2.5) we can observe a single intersection at around $x \approx 5.388$ which indicates a weak Allee effect of the population. Unfortunately, this is completely wrong as the original model did not intend such effect, indicating instead exponential growth for both of the populations x, y.

On the other hand, the NDE-enhanced method incorrectly returned more coefficients, especially in the first equation where also the interaction term is missing; this kind of mistake leads to an analogous observation as done previously. This confirms that the knowledge-enhanced method is more robust and better than the full neural network approach (i.e. NDE), as claimed in [34].

With this experiment, we managed to successfully reproduce the

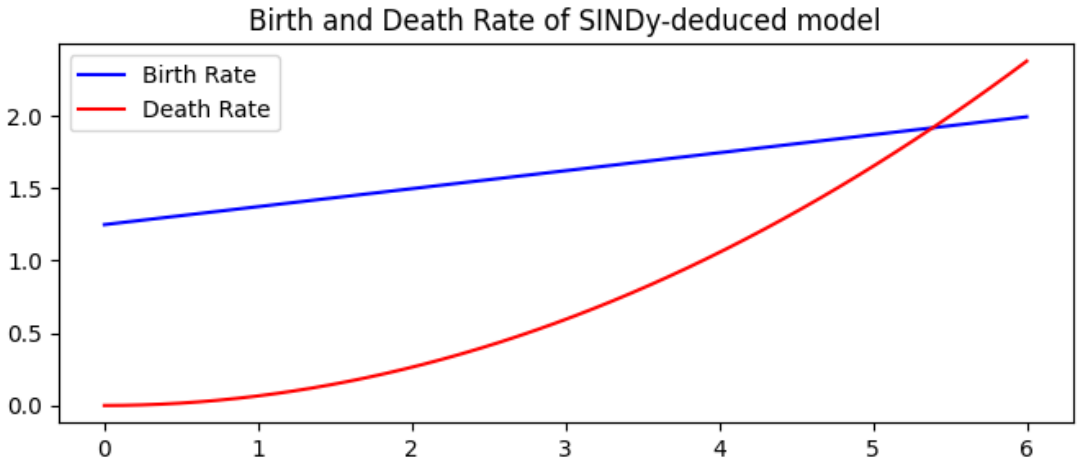


Figure 2.2.5: Birth and Death Rates of the wrongly inferred model

Lotka-Volterra case study of [34]. However, a better choice in NN architecture can be made to leverage our knowledge of the dynamics even further which will be done in the following experiment.

2.2.2 Experiment 2: Alternative UDE architecture

This experiment's set up is essentially the same as the previous one, the substantial difference is in the architecture of the UDE. In the previous experiment, the UDE was defined with the following equations:

$$\begin{aligned} \dot{x} &= \alpha x - U^{(1)}(x, y; \theta) \\ \dot{y} &= -\beta y + U^{(2)}(x, y; \theta) \end{aligned} \tag{2.8}$$

In other words, we had a neural network with two outputs. However, this might reflect poorly on the fact that both of the terms represent the same factor xy scaled by some constants. So in this experiment we propose an alternative UDE architecture, defined as follows:

$$\begin{aligned} \dot{x} &= \alpha x - U(x, y; \theta) \\ \dot{y} &= -\beta y + \varphi U(x, y; \theta) \end{aligned} \tag{2.9}$$

Where U is a single-output neural network and φ is a learnable parameter. Then we trained the UDE in the same manner as previously and used the SINDy algorithm to recover the equations.

In the end, the UDE yielded a final loss of 1.186×10^{-4} and the

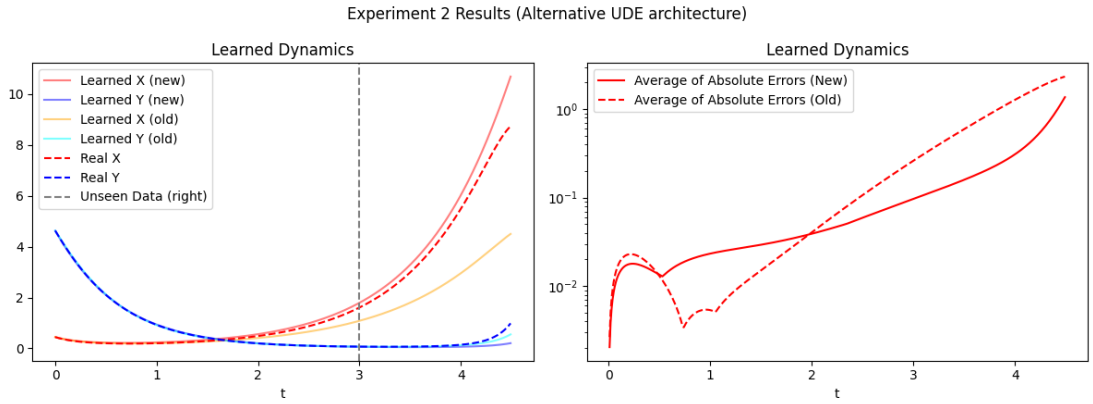


Figure 2.2.6: Comparison of the alternative UDE architecture with the previously proposed one

UDE-enhanced method recovered the following equations:

$$\begin{aligned} \dot{x} &= 1.256x - 0.815xy \\ \dot{y} &= -1.729 + 0.506xy \end{aligned} \tag{2.10}$$

By comparing it with the previous architecture (fig 2.2.6) the alternative UDE architecture is slightly better in extrapolating the dynamics from the small training set, albeit it still requires post-identification parameter estimation in order to fix the parameters.

2.2.3 Experiment 3: Lotka-Volterra with other parameters

In this experiment, we will follow the same pipeline as described in the first experiment. However, we will change the parameters of the Lotka-Volterra equations, in particular we will define the following Lotka-Volterra equations:

$$\begin{aligned} \dot{x} &= 2x - xy \\ \dot{y} &= cxy - 3y \\ (x_0, y_0) &= (0.4, 4) \end{aligned} \tag{2.11}$$

Where we will set $c = 0.5, 2$. Also, the time range from which we obtain the training set is $t \in [0, 2]$. The objective of this experiment is to test the UDE-enhanced SINDy method even further and attempt to see if there is any "preference" for any scaling factor of c.

Tables 2.2.2, 2.2.3 report the recovered coefficients of the original SINDy method and the enhanced SINDy method. As we can see,

this time the original SINDy method managed to recover the coefficients without introducing unnecessary terms; moreover, it also inferred the growth rates of x, y accurately. However, in both cases the guessed c coefficient was inaccurate, while the UDE-enhanced method managed to obtain a closer guess.

In a certain sense, we have a trade-off between the two SINDy models: in the original case, we have an accurate estimate of the growth rates but an inaccurate estimate of the interaction coefficient. On the other hand, with the UDE-enhanced model we have a more accurate estimate of the interaction term, but a slightly more inaccurate estimate of the growth rates. To know which of the two is more preferable, we will simulate the recovered models and compare their errors with respect to the original trajectory.

In fact, from figures 2.2.7 and 2.2.8 we can see that the recovered equations with the UDE-enhanced SINDy methods are able to extrapolate the dynamics of the Lotka-Volterra equations much better than the original SINDy method. In fact, in the graph of the first case (fig. 2.2.7) the average of absolute errors between the simulation reconstructed with the UDE-enhanced method and the original equations tends to stay below the order of 10^0, while the other method stays above 10^0.

In the end, we can conclude that indeed UDEs provide a way to enhance the SINDy algorithm through the partial knowledge of the original dynamics.

Original LV equations	$2x - xy$	$-3y + 0.5xy$
Name	**Recovered** $\dot{x}$	**Recovered** $\dot{y}$
Original SINDy	$2.002x - 0.997xy$	$-3.007y + 1.256xy$
UDE-enhanced SINDy	$1.993x - 0.836xy$	$-3.053y + 0.426xy$

Table 2.2.2: Recovered coefficients for $c = 0.5$

Original LV equations	$2x - xy$	$-3y + 0.5xy$
Name	**Recovered** $\dot{x}$	**Recovered** $\dot{y}$
Original SINDy	$2.002x - 0.997xy$	$-3.007y + 1.256xy$
UDE-enhanced SINDy	$1.998x - 0.942xy$	$-2.939y + 1.801xy$

Table 2.2.3: Recovered coefficients for $c = 2$

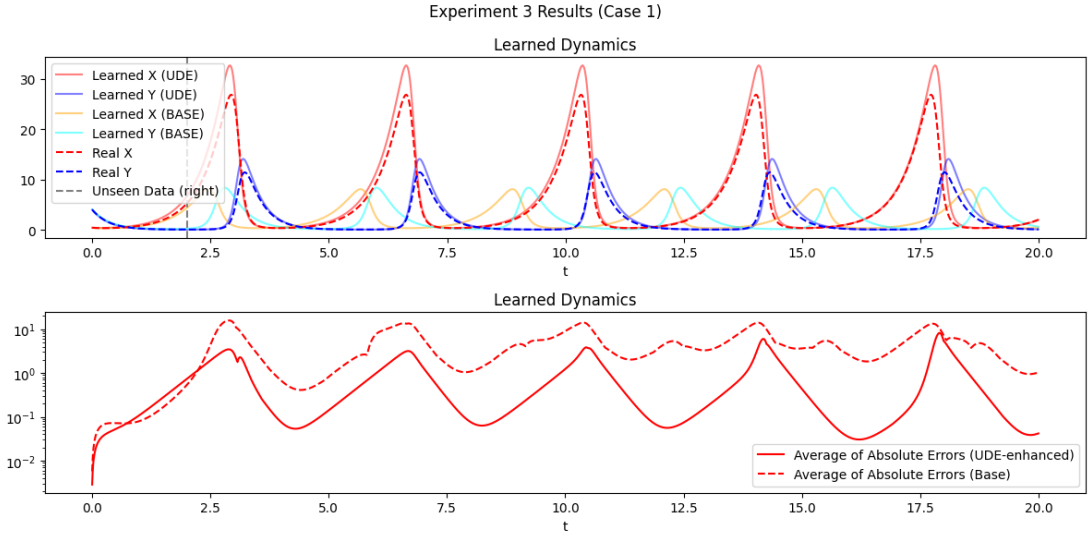


Figure 2.2.7: Simulation of the recovered equations, case 1 ($c = 0.5$)

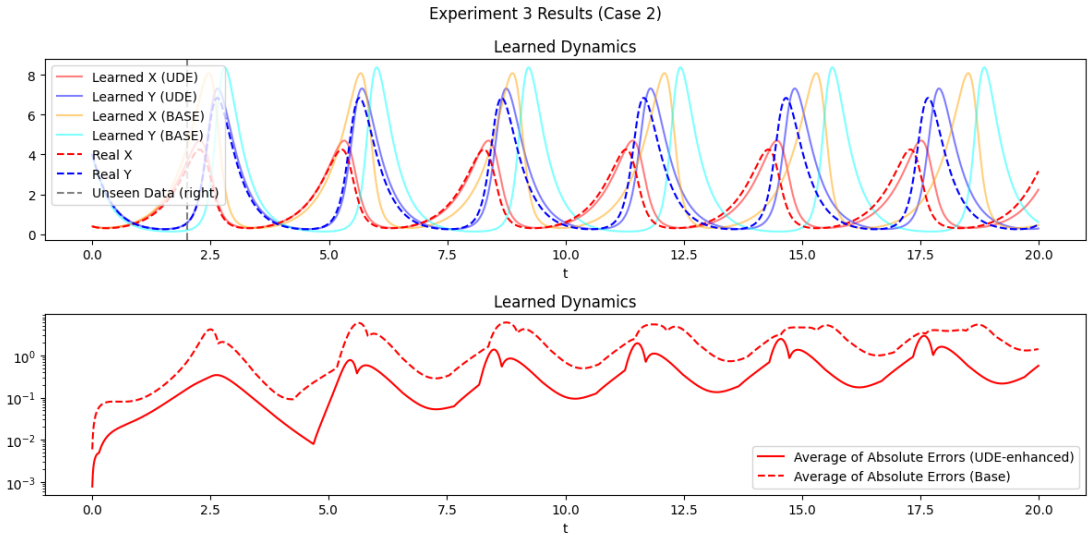


Figure 2.2.8: Simulation of the recovered equations, case 2 ($c = 2$)

Chapter 3

Case Study I: Generalized Logistic Model

One of the possible further generalizations of the logistic model (see eq. 1.17) consists of adding further terms in the second factor of higher degree. In particular, we can replace that term with a polynomial with positive coefficients:

$$\dot{N} = rN(1 - \rho_1 N - \rho_2 N^2 - \ldots - \rho_k N^k) \tag{3.1}$$

Even more generally, we can rewrite equation 3.1 as the following DE with a polynomial on the right hand side:

$$\dot{N} = q(N) \tag{3.2}$$

With $q \in \mathbb{R}[N]$ a polynomial such that $q_0 = 0, q_1 = r$ and $\forall i \geq 2, q_i < 0$. We will refer to the model described by equation 3.2 (or eq. 3.1) as the *generalized logistic equation*.

3.1 Idealized Logistic Equation

In this chapter we will consider the generalized logistic growth equation as defined previously with equation 3.1:

$$\dot{x} = r_0 x(1 \underbrace{-\rho_1 x - \rho_2 x^2 - \ldots - r_k x^k}_{:=-\rho(x)}) \tag{3.3}$$

Here, we will perform various experiments with the main goal of experimenting the potentialities of the UDE in various ways. They are the following:

1. Learn the polynomial term $\rho(x)$ of the equation 3.3 with a small dataset consisting only of three data points, and extrapolate the polynomial term with SINDy
2. Repeat the previous experiment for different values of $x_0 \in [0.01, 2]$
3. Analogous as above, but use different sizes of the dataset with $N = 3, \ldots, 39$
4. Use single or multiple trajectories for the model training phase

3.1.1 Experiment 1: Learning Polynomials

In this experiment, the set-up of the experiment is as follows:

- We take $k = 4$, with parameters $r_0 = 1, \underline{\rho} = [0, 0.25, 0.1, 0.001, 1]$
- The training data consists of only three data points, with $t = 0, 1, 3$. The initial state is set to $x(0) = 1$
- We assume the polynomial term to be unknown, so our UDE will be defined as $\dot{x} = r_0 x(U(x; \theta))$
- To solve the UDE, the Dormand-Prince method of order 5 (DOPRI-5) has been employed
- The universal approximator $U(x; \theta)$ consists of a fully-connected NN with an input layer, a hidden layer with 10 neurons, a hidden layer with 32 neurons, a hidden layer with 10 neurons and an output layer. Each hidden layer uses the hyperbolic tangent activation function (also known as tanh)

The objective of the experiment is to: (1) train an UDE that is able to reproduce the original dynamics, (2) given the trained UDE, infer the original polynomial with the SINDy method.

For the objective (2), we will train three SINDy models:

- The first model is trained directly on the training data, approximating the derivative with the finite-difference method (see [44]). This serves as a baseline model, and will be used to compare with the next models
- The second model consists of a UDE-enhanced approach, as the derivative is approximated with the learnt UDE. The training set remains the same
- The third and last model is the same as previous, however the UDE is also leveraged to obtain more training data for

the sparse regression; in particular, we will have a more dense sampling with a constant time step $\Delta t = 0.01$, from $t = 0$ to $t = 3$.

We trained the UDE with the ADAM optimizer for 1000 epochs with a learning rate of 0.01, yielding a final loss of 6.9468×10^{-8}. By also observing the learned dynamics on a longer time horizon (fig. 3.1.1), we can conclude that the objective (1) has been satisfactorily realized.

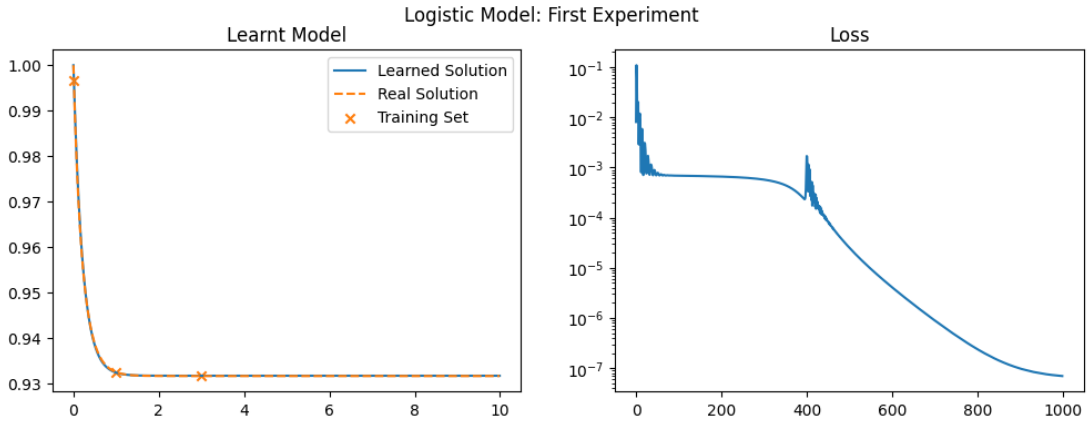


Figure 3.1.1: Generalized Logistic First Experiment Results

Then, for objective number (2), we trained the three SINDy models, yielding their respective final objective values (as defined in equation 1.13), coefficients and distance from the original polynomial quantified as the L^2 distance, defined in equation 3.4.

$$d_{L^2([0,1])}(f, g) := \int_0^1 |f(x) - g(x)|^2 \, \mathrm{d}x \tag{3.4}$$

As we can observe in the figure 3.1.2 and in the tables 3.1.2, 3.1.1, we can clearly conclude that the SINDY-enhanced approach managed to learn very accurately the target polynomial function as it reduced the baseline L^2 error from 3.593×10^{-2} to 8.052×10^{-5}. However, the approach in leveraging UDEs to generate more data points for the sparse regression turned out to be worse than the previous approach; nonetheless, it still managed to improve the baseline result, reducing the baseline error to 2.135×10^{-3}.

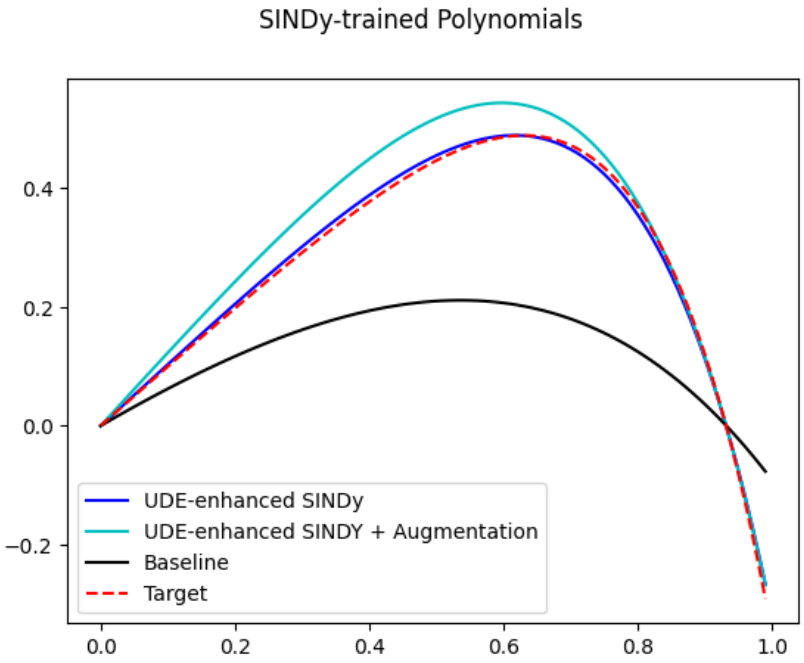


Figure 3.1.2: Generalized Logistic First Experiment Results (SINDy)

Name	**Total Error**	**L^2 distance**
Baseline	3.3612×10^{-2}	3.593×10^{-2}
UDE-enhanced	3×10^{-2}	8.052×10^{-5}
UDE-enhanced + Data Augmentation	3.0157×10^{-2}	2.135×10^{-3}

Table 3.1.1: Model comparison metrics

3.1.2 Experiment 2: Small and Large Initial Values

We can observe that the polynomial defined in the previous experiment has two roots: one at $x = 0$ and the other one at $x \approx 0.9315$. With a simple qualitative analysis of the graph of the dynamics (fig. 3.1.3) we can see that the equilibrium at $x = 0$ is unstable and the other one is globally stable. Moreover, in the previous experiment we set the initial state at $x_0 = 1$, which is fairly close to the global attractor. We considered the idea to study the same model for different initial states, with the goal to observe how the model performs for different initial conditions

So, in this experiment we will run the same procedures of the previous experiment with two different initial values: $x_0 = 0.01$ and $x_0 = 2$, which represent some sort of "extreme values" of the possible population size. Figure 3.1.4 shows the training dataset used for

Original Polynomial	$[1,\ 0,\ -0.25,\ -0.1,\ -0.001,\ -1]$
Name	**Coefficients**
Baseline	$[0.655,\ -0.295,\ -0.248,\ -0.203,\ 0,\ 0]$
UDE-enhanced	$[1.035,\ 0,\ -0.193,\ -0.295,\ -0.39,\ -0.478]$
UDE-enhanced + Data Augmentation	$[1.234,\ 0,\ -0.414,\ -0.489,\ -0.423,\ -0.230]$

Table 3.1.2: Recovered coefficients

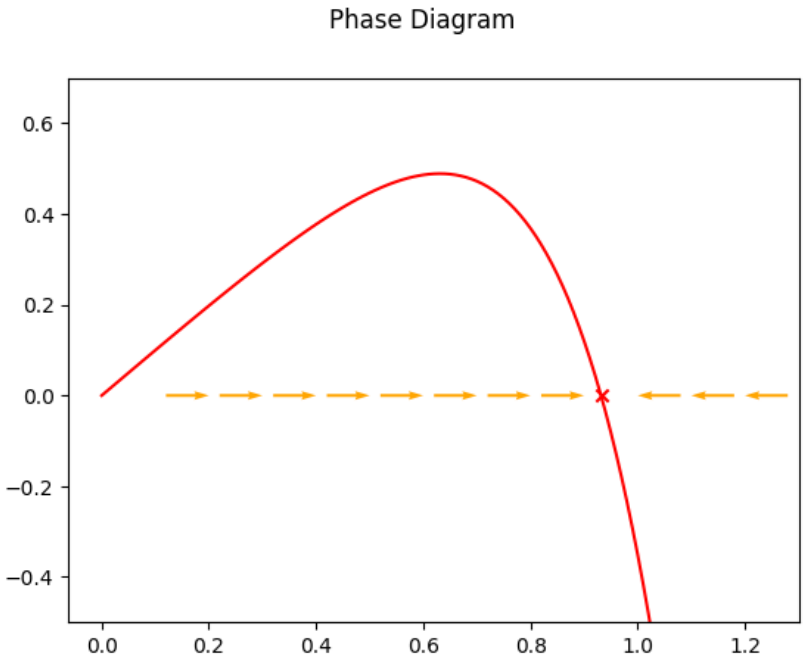


Figure 3.1.3: Phase diagram of the generalized logistic model

the experiment, both of which are extrapolated from the simulated trajectories with a constant step size.

Then we trained the UDE as defined in the previous experiment from which we trained SINDy models which will attempt to extrapolate the dynamics of the dataset.

The UDE models reached a final loss of 1.1477×10^{-5} for $x_0 = 0.01$ and 1.0920×10^{-7} for $x_0 = 2$, and the metrics relative to the SINDy models (total error defined in 1.13 and L2 distance defined in 3.4) are reported in table 3.1.3.

From the results, we can see that in both cases the UDE-based SINDy models outperform the original SINDy model significantly, especially for $x_0 = 2$ with a decrease of L^2 distance of around 98.49%

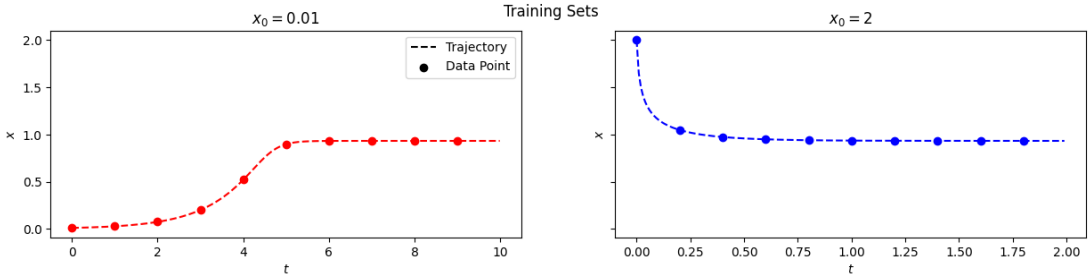


Figure 3.1.4: Training datasets of the second experiment.

Name	**Total Error**	L^2 **distance**
Case 1: $x_0 = 0.01$		
Baseline	4.1193×10^{-2}	2.318×10^{-3}
UDE-enhanced	3.015×10^{-2}	9.0172×10^{-4}
UDE-enhanced + Data Augmentation	3.3034×10^{-2}	8.3912×10^{-4}
Case 2: $x_0 = 2$		
Baseline	3.0354×10^{-1}	2.8897×10^{0}
UDE-enhanced	3.0001×10^{-2}	3.9413×10^{-2}
UDE-enhanced + Data Augmentation	3.0711×10^{-2}	7.1309×10^{-2}

Table 3.1.3: Model comparison metrics, experiment 2

(from 2.6027×10^{0} to 3.394×10^{-2}). Also, in both cases the models are able to fit the trajectories very well with a final loss approximately lower than 10^{-5}, although it does not necessarily mean that a lower loss directly implies a better performance in terms of dynamics interpolation. In fact, the UDE model which reached the higher loss value of around 10^{-5} has managed to build a SINDy model with a L^2 distance-error of around 10^{-3}, whereas the other model with the final loss of around 10^{-7} built a SINDy model with a L^2 distance-error of 10^{-2}.

3.1.3 Experiment 3: Varying Initial Conditions and Dataset Sizes

In this experiment, we will generalize the pipeline of the previous experiment in order to observe how differently will the models perform for (1) different initial conditions and for (2) different sizes of training data and for a fixed initial condition $x_0 = 1$

The set up of the experiment will entail as the following: for (1), we will take 50 equally-spaced values for $x_0 \in [0.01, 2]$. For each different initial point, the training data will consist of a trajectory

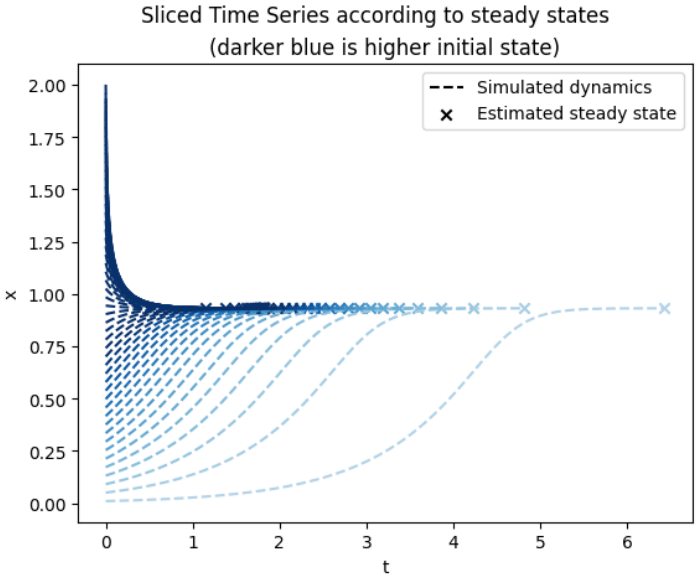


Figure 3.1.5: Generated trajectories for different initial points

of ten data points, ranging from $t = 0$ to $t = t_\infty + 4$, where t_∞ is the timestamp where can consider that the population size has reached its steady state. We define t_∞ as the first time instant such that the ℓ^1 distance between $x(t)$ and the steady state is smaller than an arbitrarily small ε:

$$t_\infty := \min_{t \geq 0} |x(t) - x_\infty| < \varepsilon \tag{3.5}$$

In our case, we set $\varepsilon = 1 \times 10^{-4}$. Figure 3.1.5 illustrates the generated time series with the above-described methodology. Then for (2) we set $x_0 = 1$ and considered $N \in [3, 29] \cap \mathbb{N}$ for $t \in [0, 3]$
The figures 3.1.6, 3.1.7 report the final losses and the L^2 errors of the trained SINDy models for different values of x_0 and n.

In the first case, we can see that the UDE-enhanced method works generally better than the baseline model for low x_0 values. However, for the values near the initial condition the UDE-enhanced method turned out to be slightly worse than the baseline method, which is opposite of what happened in the previous experiment. This could be due to the fact that we have sampled the training set differently, especially in terms of the training dataset. Then we can also observe that for higher x_0 values, the baseline method suffers a spike in its L^2 error, while the UDE-enhanced method remains stable with an L^2 error of around 10^{-2}.

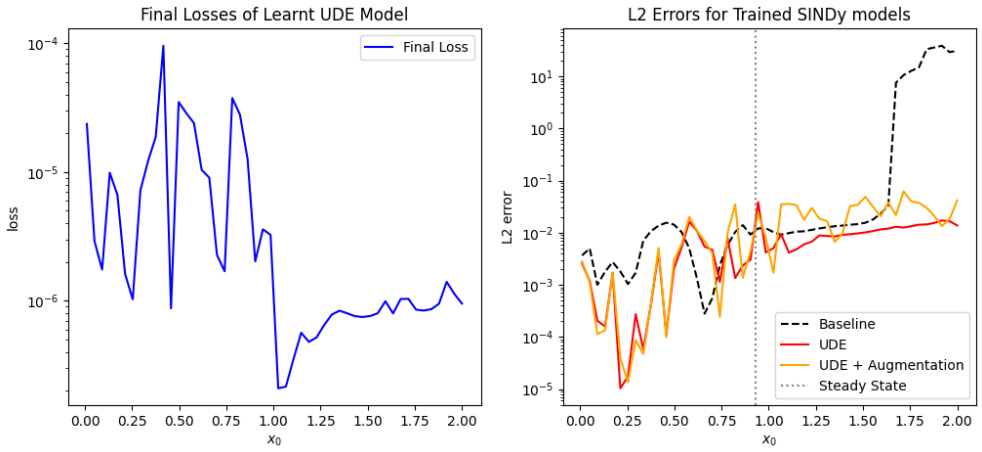


Figure 3.1.6: Results of third experiment, for $x_0 \in [0.01, 2]$

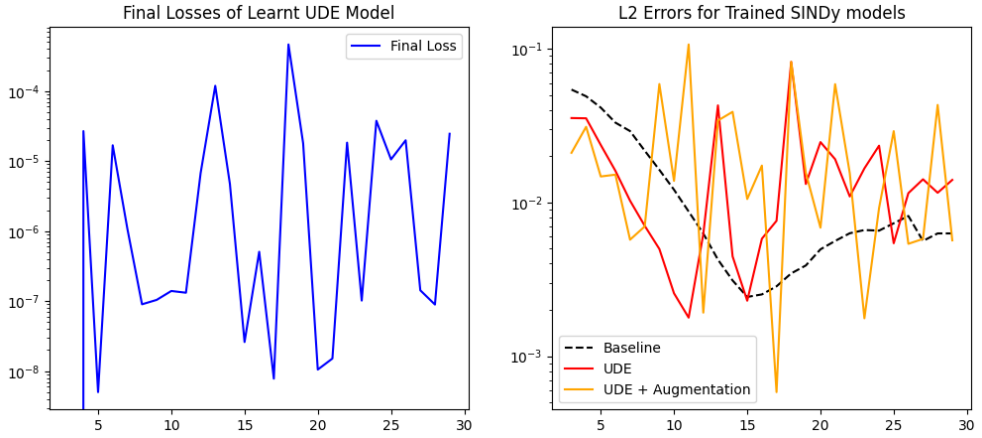


Figure 3.1.7: Results of third experiment, for $n = 3, \ldots, 29$

For the second case, we can see that from around 10 data points onward the UDE-enhanced method tends to destabilize, suggesting that the method works better for a moderate size of training set.

In summary, with this experiment we can conclude that the UDE-enhanced SINDy method can sometimes improve than the base model, although it is robust for a moderate amount of training samples and for different initial conditions of the model.

3.1.4 Experiment 4: Training UDEs with more Trajectories

In the previous two experiments, we have trained the UDE model using only one timeseries of the model: however, a scientist might have multiple trajectories of the same system. So an idea to create a more robust model, especially with respect to unseen initial

conditions, is to train the models with multiple trajectories of the model.

In this experiment, the goal is to (1) see how the UDE-enhanced SINDy method changes between the two training methods and (2) compare the extrapolated dynamics of the UDEs by comparing their trajectories with an unseen initial condition.

In this experiment we will consider the same coefficients of the first experiment, however we will consider three trajectories of the model with initial conditions $x_0 = 0.5, 1, 1.5$ with five data points each. Figure 3.1.8 shows the extrapolated dataset. Then we have trained

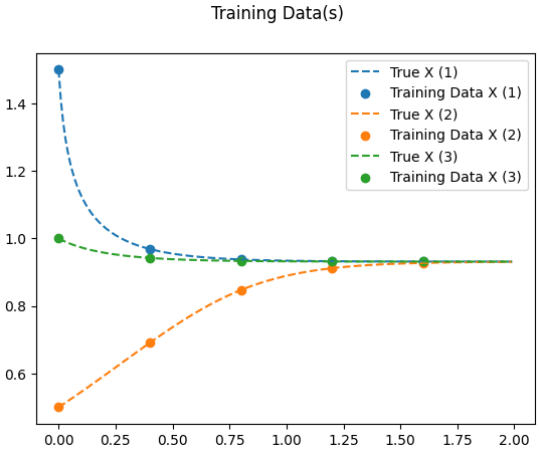


Figure 3.1.8: Experiment 4: Dataset. Parameters described in the text

two UDEs models: the first one serves as a baseline and is trained only on one of the trajectories, while the second model makes use all of the three trajectories in its training iterations. The first model was trained for 1000 epochs with a learning rate of 1×10^{-3} and the second one was trained for 5000 epochs with the same learning rate. The reason that the second model has more training epochs is that it might require more time to fit all of the trajectories simultaneously, compared to fitting only one. For both of the UDEs, the RK4 solver was used. The SINDy models are trained with only one of the trajectories, which in our case is the one with initial condition $x_0 = 1.5$.

Figures 3.1.9 shows the results of the model that has been trained with all 3 trajectories at the same time, and by observing its loss

plot we can confirm that the model requires more training iterations in order to converge to a minimum.

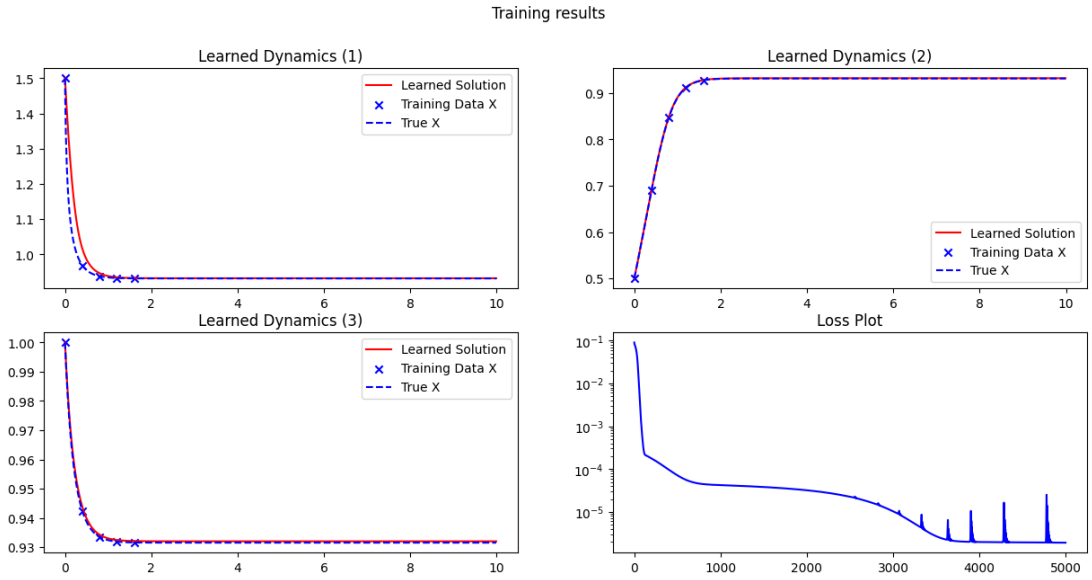


Figure 3.1.9: Experiment 4: Result (Model with more trajectories)

Then, figure 3.1.10 shows the result of the trained SINDy models. We can see that the two models have very similar performances, with the UDE-enhanced model that has been trained on multiple trajectories yielding an L^2 error of 2.065×10^{-2}, while the one trained on only one trajectory yielded an error of 2.408×10^{-2}. We can consider this to be only a very slight improvement of the UDE-enhanced SINDy method for the goal (1).

However, for objective (2), figure 3.1.11 points out that the base UDE (i.e. the one trained with only one trajectory) is not able to generalize the dynamics for unseen initial conditions as well as the other UDE trained with multiple trajectories; especially, we are considering the unseen initial condition $x = 0.01$ for both trained UDEs. Therefore, from this result we still can say that this form of training is an improvement of the original pipeline in terms of dynamic extrapolation and robustness. However, the only caveat lies in the increased computational expenses and requirements for more data.

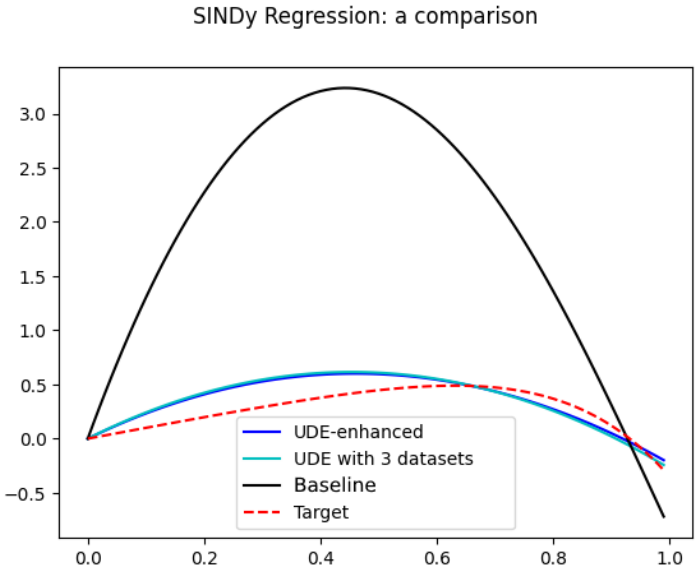


Figure 3.1.10: Experiment 4: SINDy models

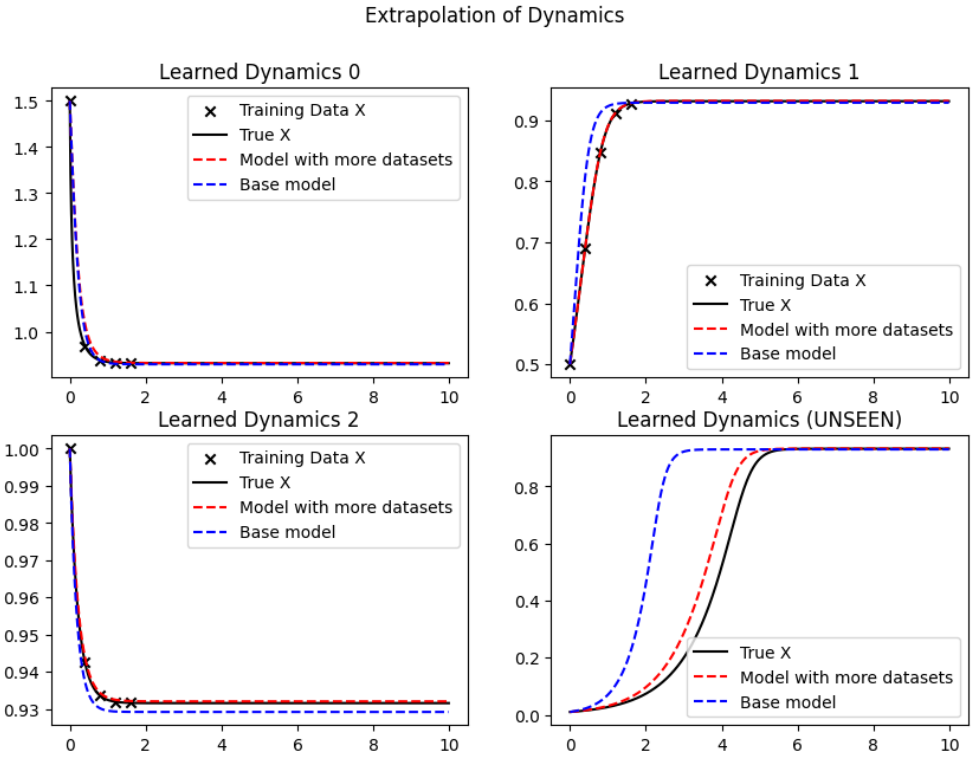


Figure 3.1.11: Experiment 4: Models Comparison

3.2 Noisy Measurements of the Logistic Equation Over Time

Up until now, we have assumed an "ideal" measurement of the population growth. However, in more realistic scenarios, there could be noise in the data due to various factors. For instance, they could be due to errors in measurements, uncontrolled and unknown factors or inherent stochasticity of the dynamics. In our case, we will consider random fluctuations which will be represented with random variables.

We will consider the same model of the previous section (eq. 3.3) from which we will extrapolate a training set $\{t_i, x_i\}_{i=1,\dots,N}$ and introduce a form of noise therein.

We will consider two types of noise: additive and multiplicative. The first type of noise consists of simply adding a term η_i, which is sampled from a probability distribution. Consider our dataset $\{t_i, x_i\}_{i=1,\dots,N}$ sampled from the generalized logistic model, we will turn it into a dataset with noise $\{t_i, z_i\}_{i=1,\dots,N}$ according to the system of equations 3.6, where $\mathcal{N}(\mu, \sigma^2)$ is the Gaussian distribution with mean μ and standard deviation σ and $\varepsilon > 0$ is an arbitrarily small number which is necessary to avoid the noise term setting some data points to a non-positive value.

$$\begin{aligned} \gamma_i &\sim \mathcal{N}(0, \sigma_\gamma^2) \\ y_i &= x_i + \gamma_i \\ z_i &= \max\{z_i - \varepsilon, 0\} = \texttt{ReLU}(z_i - \varepsilon) \end{aligned} \tag{3.6}$$

Similarly, multiplicative noise acts in an analogous manner as equation 3.6, but instead of adding the noise term it multiplies, which yields the following system of equations for the transformation:

$$\begin{aligned} \eta_i &\sim \mathcal{N}(1, \sigma_\eta^2) \\ y_i &= x_i \cdot \eta_i \\ z_i &= \max\{z_i - \varepsilon, 0\} = \texttt{ReLU}(z_i - \varepsilon) \end{aligned} \tag{3.7}$$

In this chapter, we will perform experiments where we re-run the pipeline defined in experiment 3.1.3 where instead of changing the

initial value x_0 or dataset size N, we will use the dataset with added noise, for changing values of $\sigma_\bullet$. The goal of this section is to experiment UDEs with noisy datasets, and observe how they perform relatively to the strength of the noise, represented with the variance σ.

3.2.1 Experiment 1: Additive and Multiplicative Noise

Here, we will use the same parameters of the logistic model in the previous section, i.e. the one with polynomial term of fourth degree with parameters $r_0 = 1, \underline{\rho} = [0, 0.25, 0.1, 0.001, 1]$. Also, the UDE is defined in the same manner, with the polynomial term being substituted by the neural network $U(x; \theta)$ with four hidden layers which use the hyperbolic tangent activation function.

The training set is extrapolated from the trajectory of the model with initial state $x_0 = 0.5$, from the time span $t \in [0, t_\infty]$ where t_∞ is the time where the steady state is considered to be reached, which is estimated with equation 3.5. Figure 3.2.12 shows the extrapolated dataset where we applied the two forms of noise, with different values of variance $\sigma^2 = 0.02, 0.05, 0.1$ So we ran the pipeline defined in experiment 3.1.3, where instead of changing the x_0 or N values, we changed the σ values in the set $\sigma \in \{0, 0.001, 0.01, 0.05, 0.1, 0.25, 0.5\}$. The case $\sigma = 0$ is used as a baseline case, from which we will compare with the cases where noise is applied.

Figures 3.2.13 and 3.2.14 report the final losses of trained UDEs and L2 errors of their respective SINDy-based models in a logarithmic plot, with the dashed gray horizontal lines in the figures that represent the values of the baseline case with $\sigma = 0$.

First of all, the final loss values tend to increase with σ faster than an exponential growth, since that we can see an exponential growth in the logarithmic plot. Of course, this was expected as the noise distorts the true dynamics of the training set significantly, which impairs the learning capabilities of the UDE.

However, if we observe the L2 errors of the trained SINDy models with additive noise, we can see that for small values of σ (until

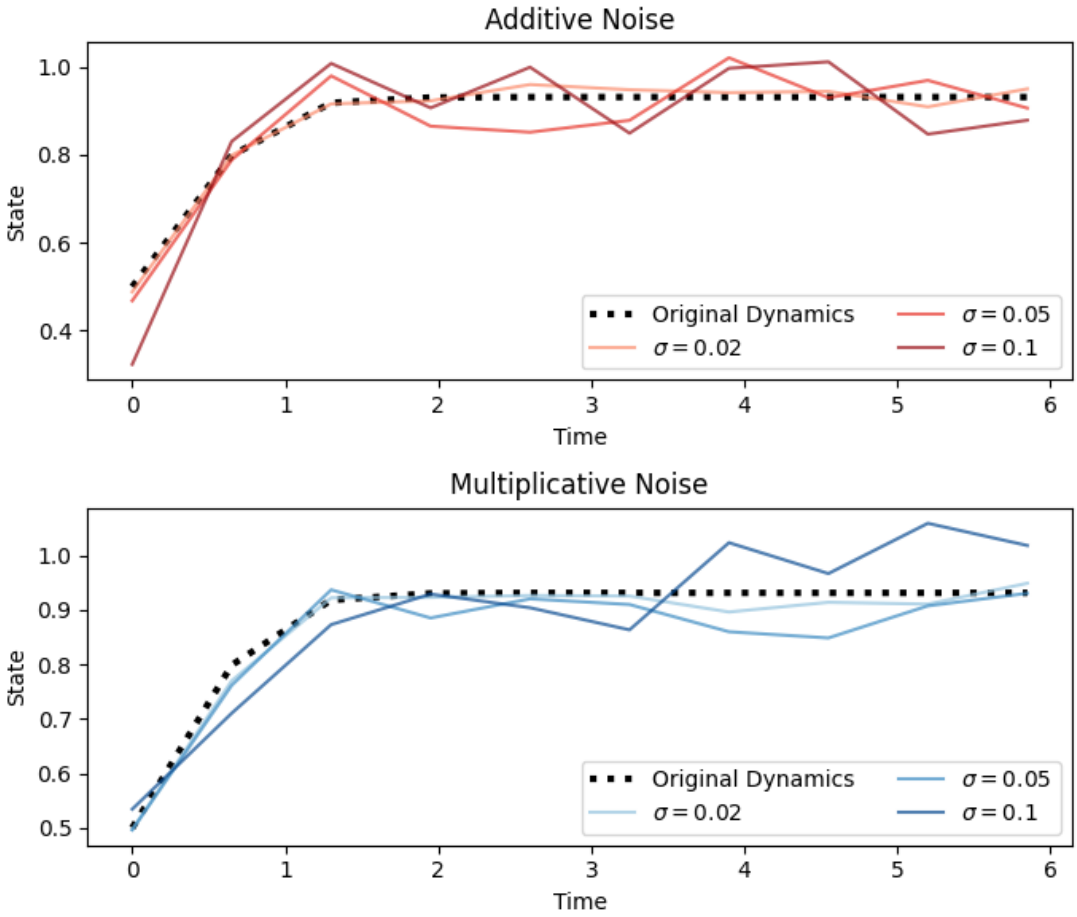


Figure 3.2.12: Dataset of the logistic model with noise. Lighter hues indicate lower amount of variance in the noise. Parameters described in the text

around $\sigma = 10^{-1}$) the UDE-based models are able to extrapolate the dynamics sufficiently well enough and better than the original SINDy model (the black line). Additionally, for the intermediate value of $\sigma = 0.05$, the UDE-based models perform nearly well as the baseline case (i.e. the model without noise). Then, the performance of the UDE-based SINDy models deteriorates for values after 2.5×10^{-1}, even exceeding the error of 1 and becoming worse than the original SINDy model

This phenomenon is also observed in the dataset with multiplicative noise, however even more strangely for all the cases with noise less than $\sigma = 0.01$ the SINDy based models perform better than the baseline model significantly, before deteriorating while still maintaining better values than the original SINDy model.

With these results, we can conclude that while the UDE models are not able to fit the trajectories with a significant value of noise applied, they are still robust enough for us to leverage them in extrapolating the dynamics of the model: in some cases, we were even able to get improved results.

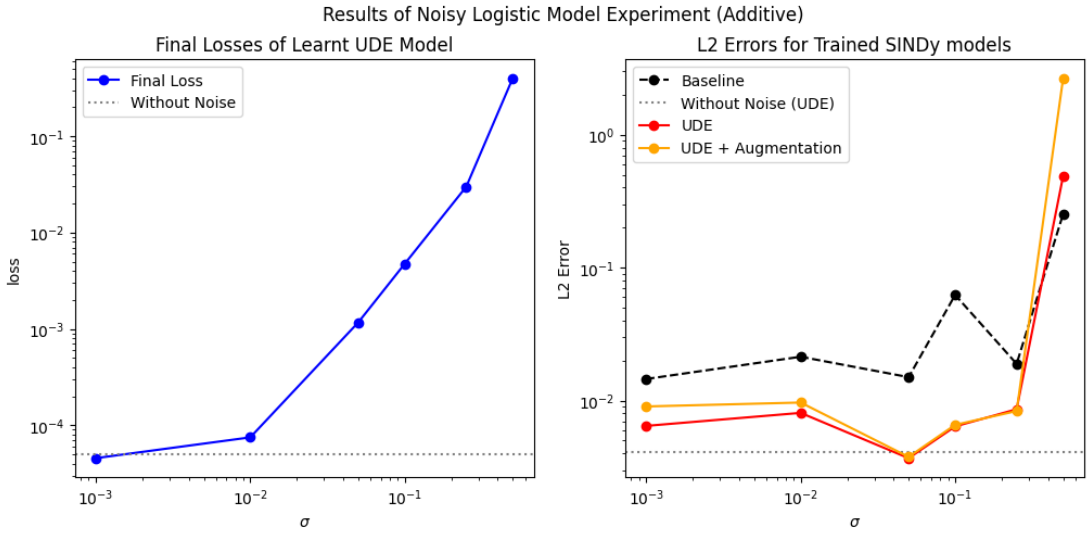


Figure 3.2.13: Results of experiment 1: additive noise

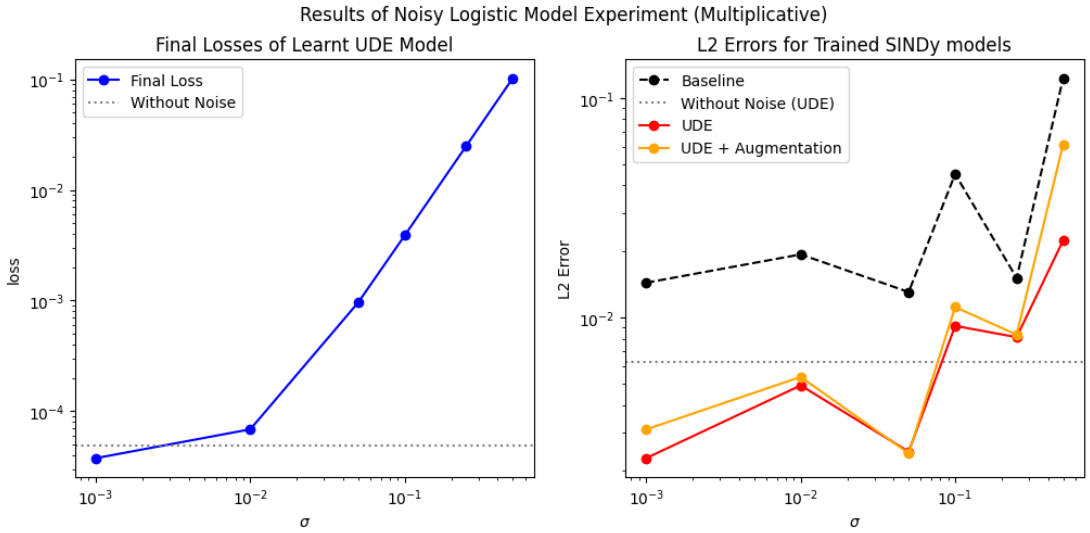


Figure 3.2.14: Results of experiment 1: multiplicative noise

Chapter 4

Advanced Case Study I: Neuron Modeling

Another important application of mathematical modeling is neurosciences: as we have seen in Chapter 1, there are some examples of models describing the mean activity of neurons. For example, we have the firing rate model (eq. 1.22) and the network of neuron populations model (eq. 1.26).

In this chapter, we will apply UDEs in neuron modeling. We will split up our analysis in two parts: in the first part we will be mainly concerned with the single firing rate populations, that is the model described in equation 1.22, and apply UDE in order to learn the key ingredients of said model, such as the external impulse function $I_{\text{ext}}(t)$ and the activation function Φ. Then in the second part we will transition towards the network of firing rate populations as described in the equation 1.26 with an analogous objective, with the extra task of learning the intra-network dynamics.

4.1 Single Firing-Rate Neurons

In this subsection, we will consider the firing rate model. Recall its equation (eq. 1.22):

$$\tau \frac{\mathrm{d}r}{\mathrm{d}t}(t) = -r(t) + \Phi(I_{\text{ext}}(t) + Jr(t)) \tag{4.1}$$

Where Φ is the activation function of the neuron, τ is the time constant and I_{ext} is the external impulse function. We will see three experiments, where in the first one we will use UDEs to learn the

impulse function, in the second one it will be used to learn the activation function instead and the external impulse will be just a constant instead, and in the third one we will use UDEs to learn both.

Across all of the three experiments, we will use the same activation function Φ, defined as the threshold function. Let us recall that the threshold function is defined as follows (eq. 1.23):

$$\Phi_T(I) := \max\{0, I - T\} \tag{4.2}$$

Also, we will set the same time constant $\tau = 1$.

Experiment 1: Learning External Impulses

In this experiment, we set up the original simulation with the following parameters:

- The neuronal population is excitatory, with $J = 1$
- The threshold of the firing rate function is set to $T = 1$
- The external impulse function is defined as the gaussian peak function, as in equation 4.3
- The initial condition is set to $r_0 = 1$

$$I_{\text{ext}}(t; \mu, \sigma) := 2e^{-\frac{(t-\mu)^2}{2\sigma^2}} \tag{4.3}$$

From this model, we obtained the training data by sampling the trajectory in the time span $t \in [0, 20]$ with constant step size $\Delta t = 0.5$. Then, to learn the external impulse function we have defined the following UDE:

$$\dot{r} = -r + \tilde{\Phi}\left(U(t; \theta) + r\right) \tag{4.4}$$

With the UDE being a neural network consisting of one input layer, one hidden layer with 16 neurons, two hidden layers with 32 neurons, one hidden layer of 16 neurons and one output layer, with each hidden layer using the hyperbolic tangent function. Also note that $\tilde{\Phi}(I)$ is a surrogate of the ReLU activation function; the necessity of such surrogate is due to the non-differentiability of the ReLU function at $I = 0$, which can cause issues with the automatic differentiation while training the model.

In order to choose an appropriate surrogate model, let us state the following lemma:

Lemma 1 *Define the parametrized softplus function as the following:*

$$\phi_\beta(I-T) := \frac{1}{\beta}\log(1+e^{\beta(I-T)}) \tag{4.5}$$

Then for $\beta \to +\infty$ the ϕ_β function uniformly converges to the ReLU activation function:

$$\phi_\beta(I-T) \overset{\beta\to+\infty}{\longrightarrow} \max\{0, I-T\} \text{ uniformly} \tag{4.6}$$

Intuitively, we can observe the plots of the graph of the parametrized ϕ_β function as in the figure 4.1.1. A formal demonstration of the uniform convergence is illustrated in the following proof.

Proof. Proving the limit 4.6 is equivalent to proving equality 4.7:

$$\sup_{I\in\mathbb{R}} \lim_{\beta} |\phi_\beta(I-T) - \max\{0, I-T\}| = 0 \tag{4.7}$$

Observe that the argument in the absolute value is bounded by $\frac{\log 2}{\beta}$, in fact for $I \leq T$ we have only $\phi_\beta(I-T) = \frac{1}{\beta}\log(1+\underbrace{e^{\beta(I-T)}}_{"\leq 1}) \leq \frac{\log 2}{\beta}$.

Conversely, for $I > T$ can observe that

$$\begin{aligned}
\phi_\beta(I-T) &= \frac{1}{\beta}\log(1+e^{\beta(I-T)}) \\
&= \frac{1}{\beta}\log(e^{\beta(I-T)}(e^{-\beta(I-T)}+1)) \\
&= \frac{1}{\beta}(\beta(I-T) + \log(1+e^{-\beta(I-T)}) \\
&= (I-T) + \frac{1}{\beta}\log(1+e^{-\beta(I-T)})
\end{aligned}$$

So we obtain the following:

$$\phi_\beta(I-T) - (I-T) = \frac{1}{\beta}\log(1+e^{-\beta(I-T)})$$

However this equation can be bounded by $\frac{\log 2}{\beta}$ in a similar fashion as above. Therefore we can deduce that

$$0 \leq |\phi_\beta(I-T) - \max\{0, I-T\}| \leq \frac{\log 2}{\beta}$$

By applying the squeeze theorem, we can conclude that the limit of the central argument goes towards zero, concluding the proof. ■

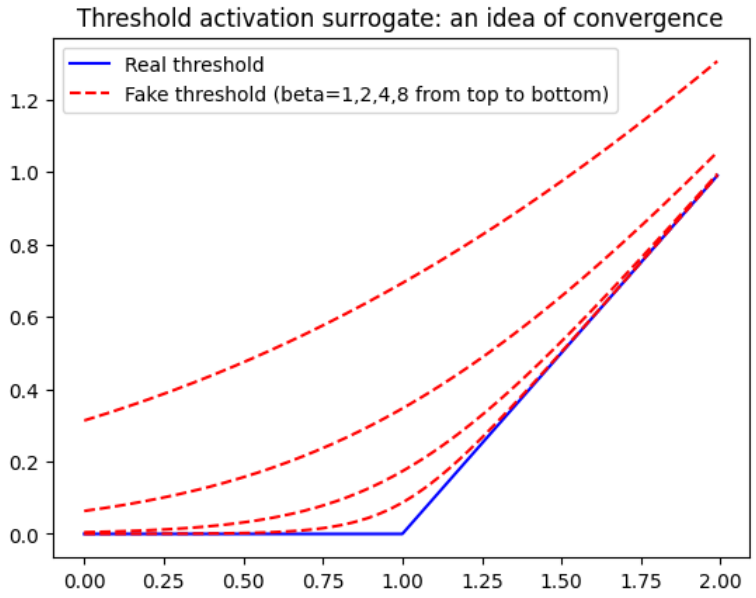


Figure 4.1.1: Plots of the parametrized softplus function

Therefore, we will choose our surrogate transfer function as the one defined in equation 4.5.

And so we trained the UDE for 1000 epochs with the ADAM optimizer with a learning rate of 3×10^{-3}, yielding a final loss of 1.452×10^{-6}. As we can see from figure 4.1.2, we can see that the UDE managed to replicate the dataset with a satisfactory accuracy, and also managed to identify the peak of the impulse function.

However, there is a discrepancy with the learned impulse function outside of the peak region, as in the original impulse function after the peak the value remained to be constantly around 0.25, whereas in the learned function is remains to be constantly zero.

This could be due to the non-identifiability of the system as we set the threshold value to $T = 1$, so any value below 1 in the impulse output could not have been "seen" by the model; so, in order to confirm or refute this conjecture we re-did the same experiment setting a lower threshold value, to see whether the missing part could be learnt or not.

As the same experiment was re-done with a lower threshold value of $T = 0.3$ and retrained the UDE for 2000 epochs, we have obtained

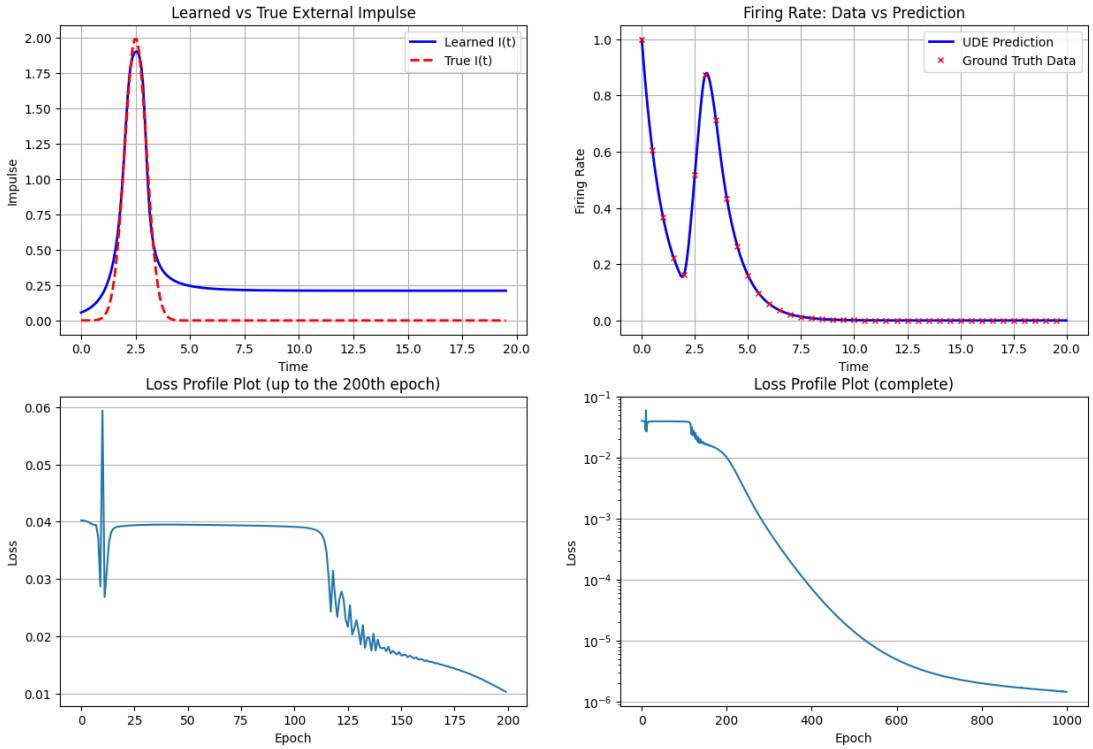


Figure 4.1.2: Results of the first experiment (neuron modeling)

an UDE which yields a final loss of 4.383×10^{-4}. We can note that here the UDE managed to replicate the external impulse function in a more faithful manner as we can observe in figure 4.1.3: this confirms the initial hypothesis of the non-identifiability of the problem due to a high threshold value.

Experiment 2: Learning Activation Functions

This time let us study the intrinsic dynamics by assuming that the external impulse function is a constant. We have defined the model with the following parameters:

- The impulse function is a constant set to $I_{\text{ext}} = 6$
- The threshold value is $T = 1$
- The neuron population is inhibitory with $J = -1$
- The initial value is $r_0 = 1$

The training set is a sample of the trajectory simulated with the time horizon $t \in [0, 3]$ and with constant step size $\Delta t = 0.2$. Then, we have defined the following UDE:

$$\dot{r} = -r + U(I_{\text{text}} + Jr; \theta) \tag{4.8}$$

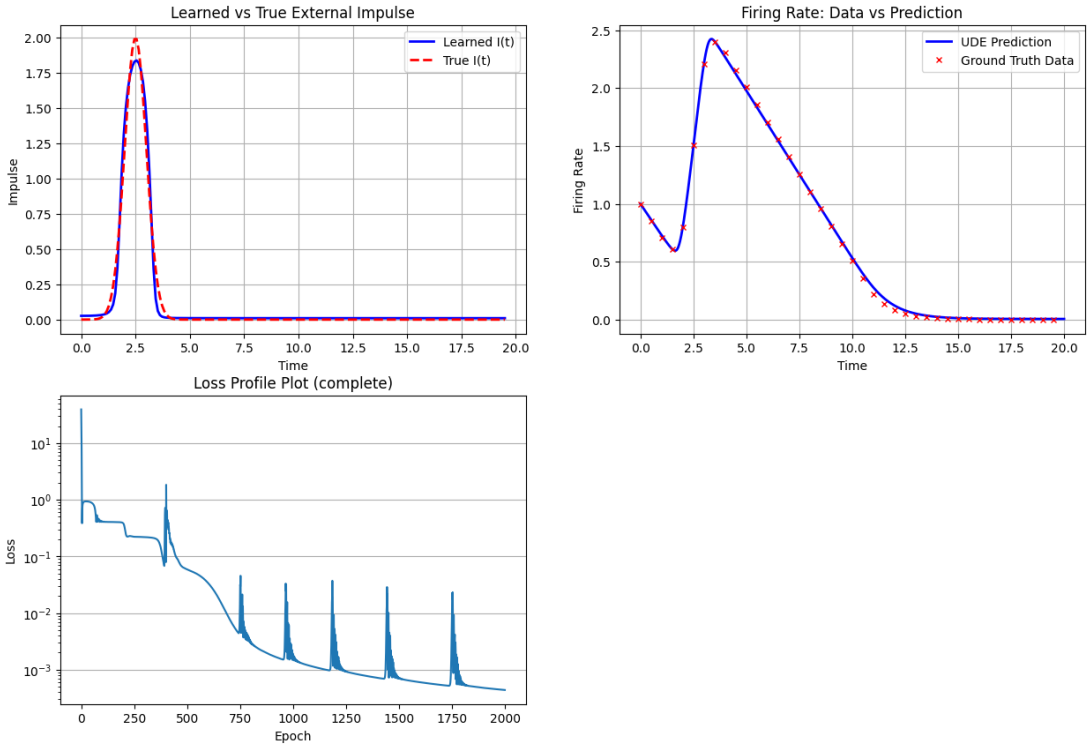


Figure 4.1.3: Results of the first experiment (neuron modeling) with a lower threshold value

Where $U(I;\theta)$ is a fully connected neural network consisting of one input layer, two hidden layers with 16 neurons and ReLU activation function and one output layer.

The goal of this experiment is to see whether the UDE 4.8 will be able to infer the activation function of the neuronal model.

The UDE has been trained for 200 epochs with the ADAM optimizer with a learning rate of 3×10^{-3}, yielding a final loss of 1.049×10^{-9}. Observing the plot of the predictions (fig. 4.1.4) we can see that the UDE is accurate in predicting the trajectory. However, one major shortcoming is that the activation function Φ was not learnt perfectly, as it resembled a ReLU function but presented major discrepancies in the linear part (i.e. after 0).

This could have been due to the fact that the UDE's NN architecture was not designed as well as to learn the function efficiently; to improve this result, we will use hyperparameter optimization techniques to find the best-suited NN architecture for this problem.

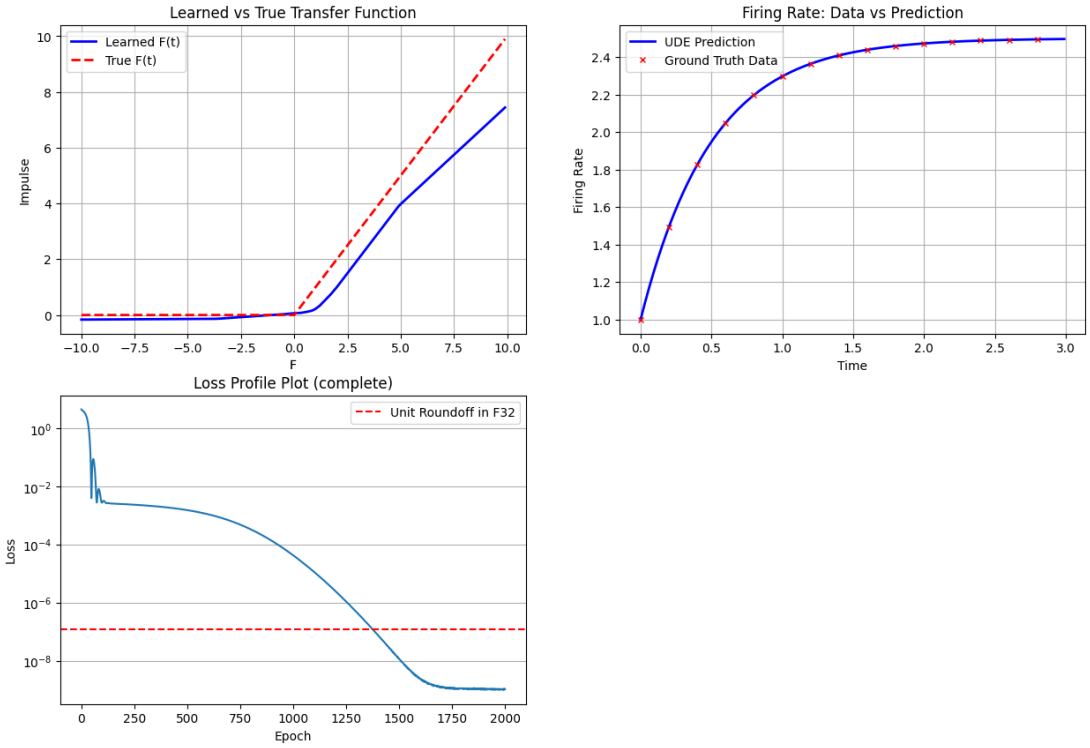


Figure 4.1.4: Results of the second experiment (neuron modeling)

To perform the hyperparameter optimization, we used the following procedure:

1. Obtain the dataset as done in this experiment, but also add a validation set which is obtained by sampling the time trajectory for a longer time horizon $t \in [0, 5]$ with a constant step size $\Delta t = 0.3$
2. We built the UDE 4.8, with the hyperparameters taken from the table 4.1.1
3. The UDE is trained for 1000 epochs with the ADAM otimizer, using a learning rate of 0.005
4. The validation loss is taken as the score metric for the hyperparameter optimizer

Hyperparameter	**Search Space**
Number of hidden layers	$1, 2, \ldots, 8$
Hidden units per layer	$4, 5, 6, \ldots, 128$
Activation function (each layer)	ReLU, GELU, Leaky ReLU, SiLU, ELU, CELU, SELU, Mish

Table 4.1.1: Hyperparameter search space for UDE's Neural Network of Experiment 2.

As we ran the network optimization pipeline, we obtained the UDE 4.8 whose neural network is composed of two hidden layers, the first one with 120 neurons and the second one with 50 neurons. The activations functions of the two layers are respectively the leaky ReLU and the ReLU function. This choice was mainly motivated by the rank plots generated by Optuna's (the optimization hyperparameter tool, a dedicated discussion is dedicated in appendix 7) dashboard functionality (fig. 4.1.5, 4.1.6).

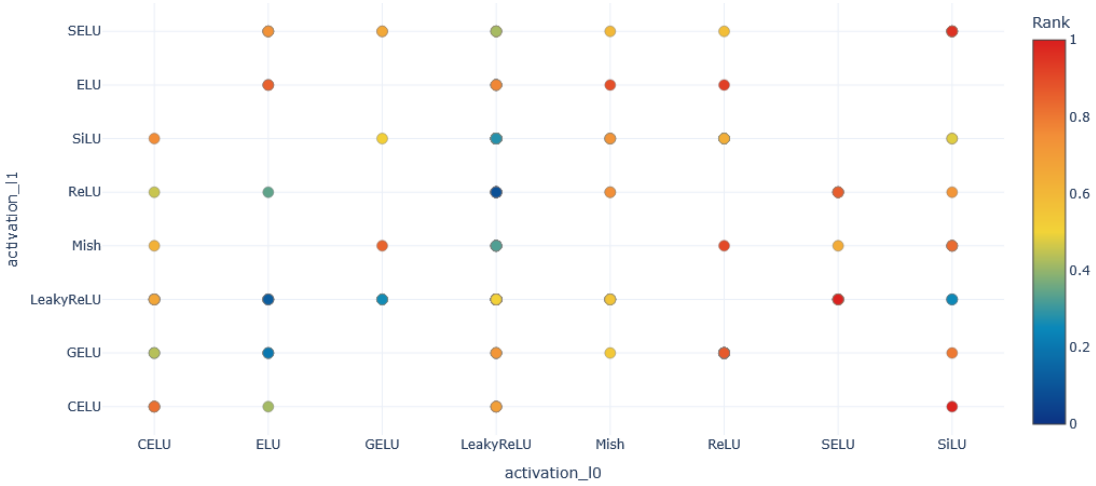


Figure 4.1.5: Rank Plot of Experiment 2: Activation Functions

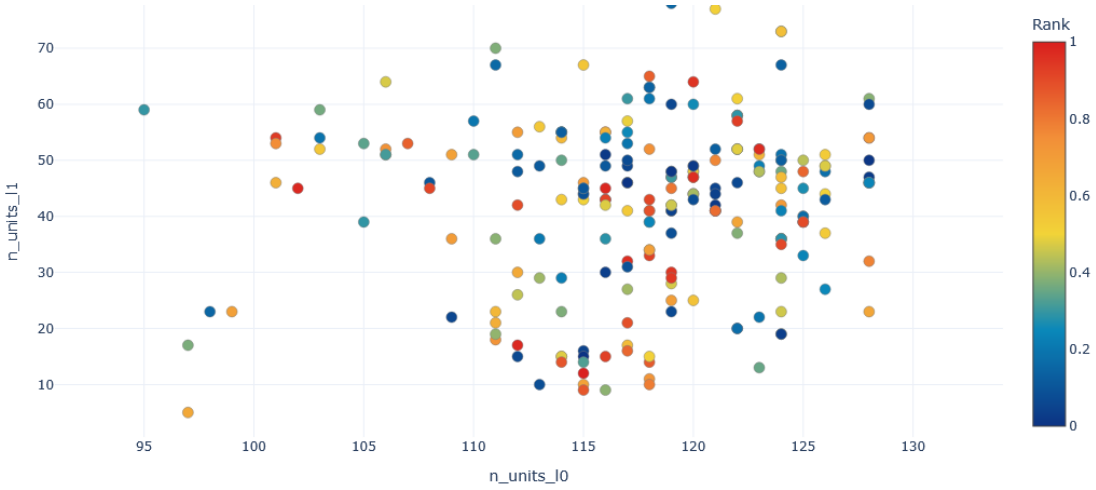


Figure 4.1.6: Rank Plot of Experiment 2: Layer Sizes

As we retrained the UDE with ADAM for 2000 epochs using a learning rate of 0.004, we obtained a model that yields a final loss of 5.636×10^{-12} on the training set. However, as we plotted the learned activation function of the neuron, we obtained a strange

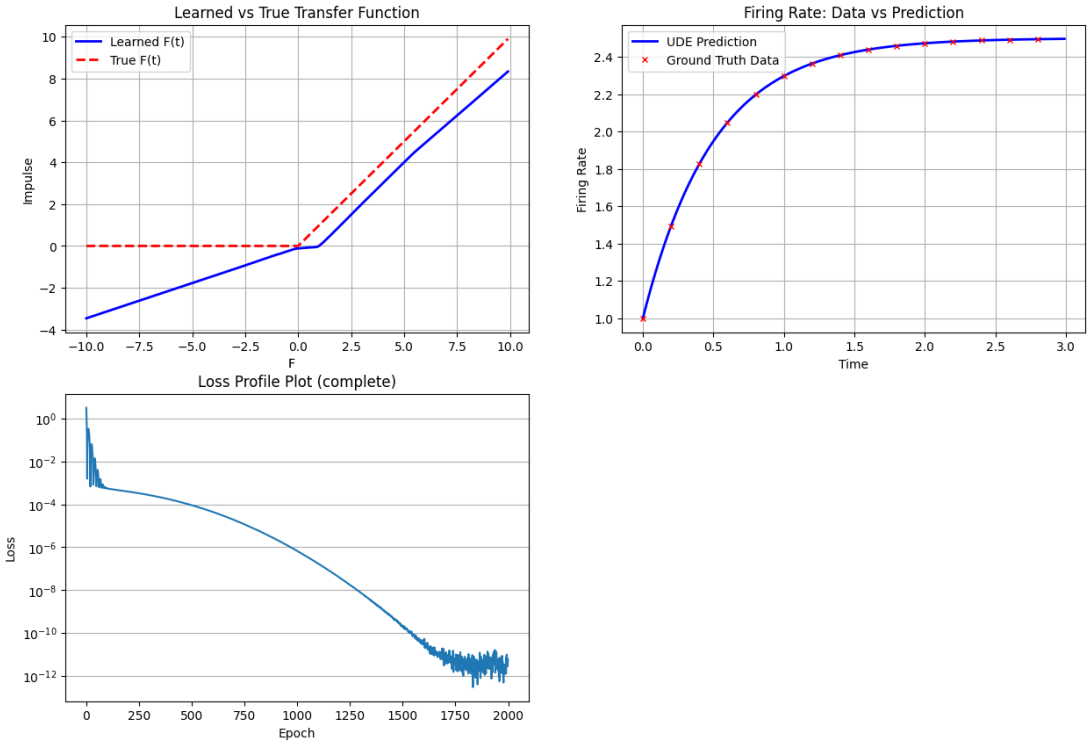


Figure 4.1.7: Experiment 2 Follow-up: Results

result: the right side had been sightly corrected, however the left side of the function presented a meaningful discrepancy with the original function (fig 4.1.7). In a way, with this technique we only managed to "flip" the side of the function that presented the major errors.

Experiment 3: Learning Both

Now, the idea of the experiment is to take the previous two experiments and combine their problems into one problem: that is, we will use UDEs to learn both the impulse function as a time-valued function and the activation function.

For this experiment, we have defined the following parameters of the model:

- The initial condition is set to $r_0 = 0$
- The threshold is set to $T = 1.5$
- The impulse function is a time-valued function defined as the gaussian peak function as in equation 4.3
- The neuronal population is inhibitory with $J = -1$

Then we simulated a trajectory of the model with the time horizon $t \in [0, 12]$, from which we obtained the training data by sampling the trajectory with constant step size $\Delta t = 0.5$. To learn both the impulse function and the activation function, we set the UDE as the following:

$$\dot{r} = -r + U_1([U_2(t; \theta_2) + Jr]; \theta_1) \tag{4.9}$$

Where U_1, U_2 are the neural networks as defined initially in the previous two experiments.

We trained the UDE with the ADAM optimizer for 2000 epochs with a learning rate of 5×10^{-4} for the first 1500 epochs and then with a learning rate of 1×10^{-3} for the remainder of the epochs. As a result, the model yielded a final loss of 5.0369×10^{-7} with its results depicted in figure 4.1.8.

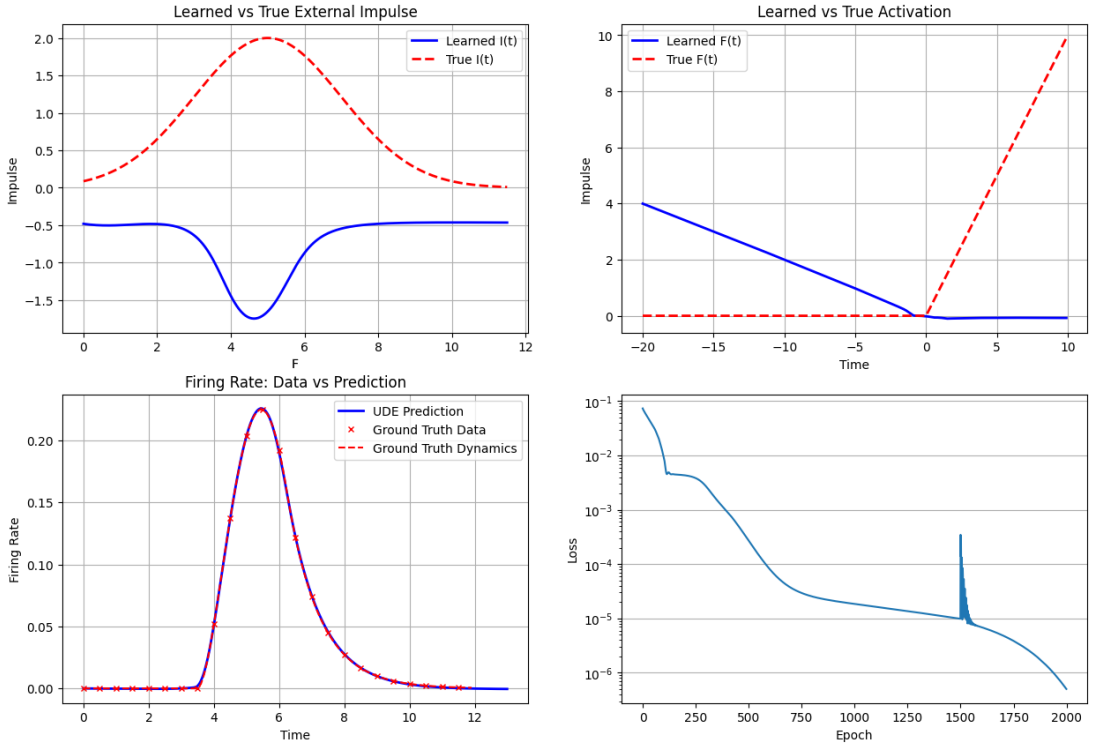


Figure 4.1.8: Results of the third experiment (neuron modeling). Let it be noted that the spike in the loss curve caused by a technical issue, that is resetting the state of the optimizer.

Other than the fact that the learn trajectory fits the original dynamics almost perfectly, let us observe that the learned functions

are strangely "symmetrically" incorrect; that is, both the learned impulse function and the activation function are the "flipped" versions of their respective original functions. This could be also due to another non-identifiability problem, where a high degree of freedom in the choice of the functions potentially led to two different solutions.

In an attempt to correct the obtained solution, we trained another model where we enforced a positivity constraint on the neural network that represents the impulse function (U_2) by placing a softplus function onto the output layer.

Unfortunately, as we can see in figure 4.1.9 the UDE has still incorrectly learned both the activation function and the external impulse; in particular, it still "insists" on the shape of a "flipped" gaussian peak function. This result might point towards the non-identifiability of the problem, due to the fact that the UDE attempts to learn both the transfer function and the external impulse function.

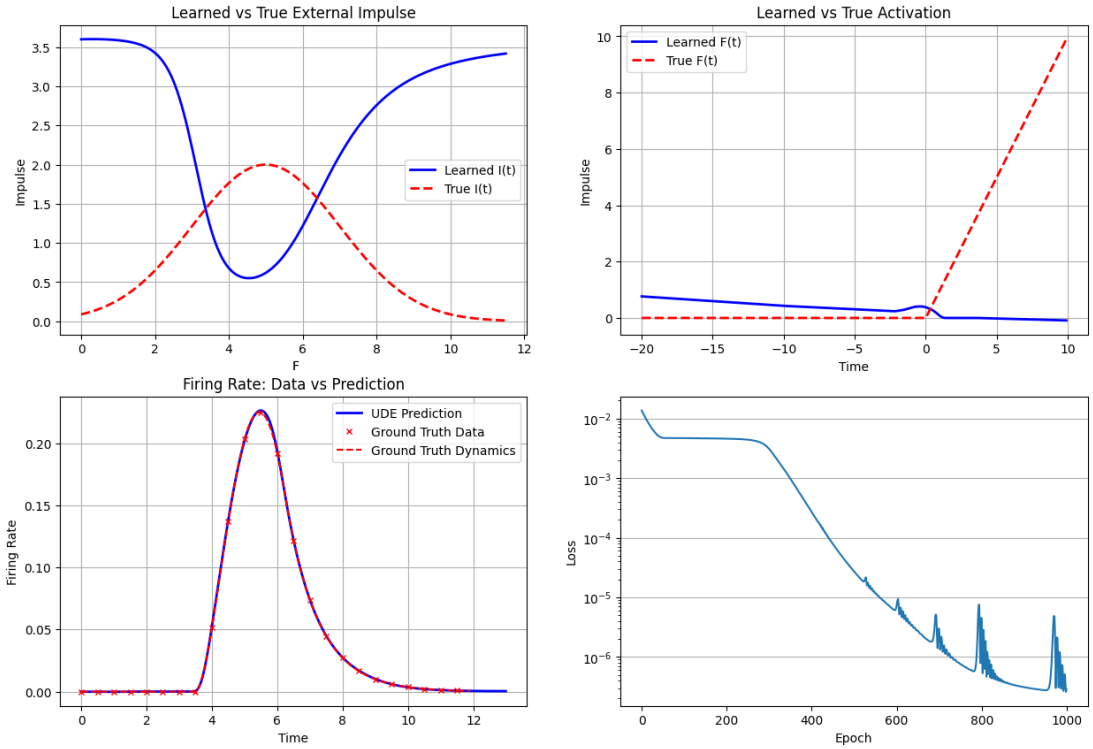


Figure 4.1.9: Results of the third experiment (neuron modeling), with forced positivity constraint on the impulse NN

4.2 Network of Firing-Rate Neurons

Up until now, we have only treated single population of neurons. In this section, we will transition towards the network of neuron populations, as proposed in [16]. Let us recall the equations of coupled neuron populations (as in 1.26):

$$\tau \frac{\mathrm{d}r_n}{\mathrm{d}t}(t) = -r_n(t) + \Phi\left(\sum_{m=1,\dots,k} w_{nm} r_m(t) + I_n(t)\right), \forall n = 1, \dots, k \tag{4.10}$$

With the model described in equation 4.10, we will apply UDEs with the objective to learn the intrinsic dynamics of the populations and the interactions between them. To this end, we performed experiments on the following two scenarios:

1. In the first scenario we will consider the so-called "winner-take-all" mechanism as proposed in [43] and see in which case the UDE is able to learn the dynamics better
2. In the second scenario we will focus on the more general case of neuron networks, with a significantly bigger size of linked populations. Here, we will have three experiments, one dedicated on learning solely the network parameters $w_{\bullet,\bullet}$, another for learning both the network weights and the activation function, and lastly an experiment focused on learning the activation function and the vector of external impulses.

In the experiments on both of the scenarios, we will assume the following:

- The time constant is set to $\tau = 1$ for simplicity
- The activation functions are shared between the populations, in our case it will be the sigmoidal function defined as $\sigma(x) := (1 + e^{-x})^{-1}$
- The external impulses are constants, but can vary across the populations; as stated in the beginning, we will focus only on the intrinsic dynamics

4.2.1 Winner-take-all Mechanism

Recall the equations of the winner-take-all mechanism as in [43] and in equation 1.28:

$$\begin{aligned} \frac{\mathrm{d}r_1}{\mathrm{d}t} &= -r_1 + \sigma(I_0 - Jr_2) \\ \frac{\mathrm{d}r_2}{\mathrm{d}t} &= -r_2 + \sigma(I_0 - Jr_1) \end{aligned} \tag{4.11}$$

It can be shown that the model of equation 1.28 exhibits two attractors: the first one being the homogeneous steady state, which always exists. Then if the inhibitory term J is sufficiently large, then the sub-population with the highest initial mean activity will prevail, while the other one gets suppressed ([43], fig. 1.5.14). So we will perform two experiments, one for each scenario.

Experiment 1: Homogeneous Steady State

Consider the equation 4.11 with $I_0 = 2, J = 2, (r_1(0), r_2(0)) = (0.99, 1.01)$. As we can see in figure 1.5.14, both of the populations converge to the same steady state on the long term.

So from the trajectory over the time span $t \in [0, 25]$ we extracted our traning dataset by performing a sampling with a constant step size of $\Delta t = 0.3$ limited to the time horizon $t = 3.5$. Then we defined the following UDE, which will attempt to learn the dynamics of the model and extrapolate the activation function:

$$\begin{aligned} \dot{r}_1 &= -r_1 + U(I_0 - Jr_2; \theta) \\ \dot{r}_2 &= -r_2 + U(I_0 - Jr_1; \theta) \end{aligned} \tag{4.12}$$

Where $U(\bullet; \theta)$ is a fully-connected neural network consisting of one input layer, one hidden layer of 16 neurons and one output layer. In the hidden layer we have applied the hyperbolic tangent activation function and in the output layer we have applied the softplus activation function.

The UDE 4.12 was trained for 1000 epochs with the ADAM optimizer using a learning rate of 1×10^{-3}, which resulted in a model that has a final loss (L2 MSE) of 1.962×10^{-4}.

As we can see in figure 4.2.10, the model is able to extrapolate the dynamics on the long term accurately, and the learned transfer function coincides with the true sigmoidal function in the central part of the function. The fact that the true transfer function was not learned in a totally accurately manner could be due to a poor choice in the architecture of model, as we chose a shallow neural network.

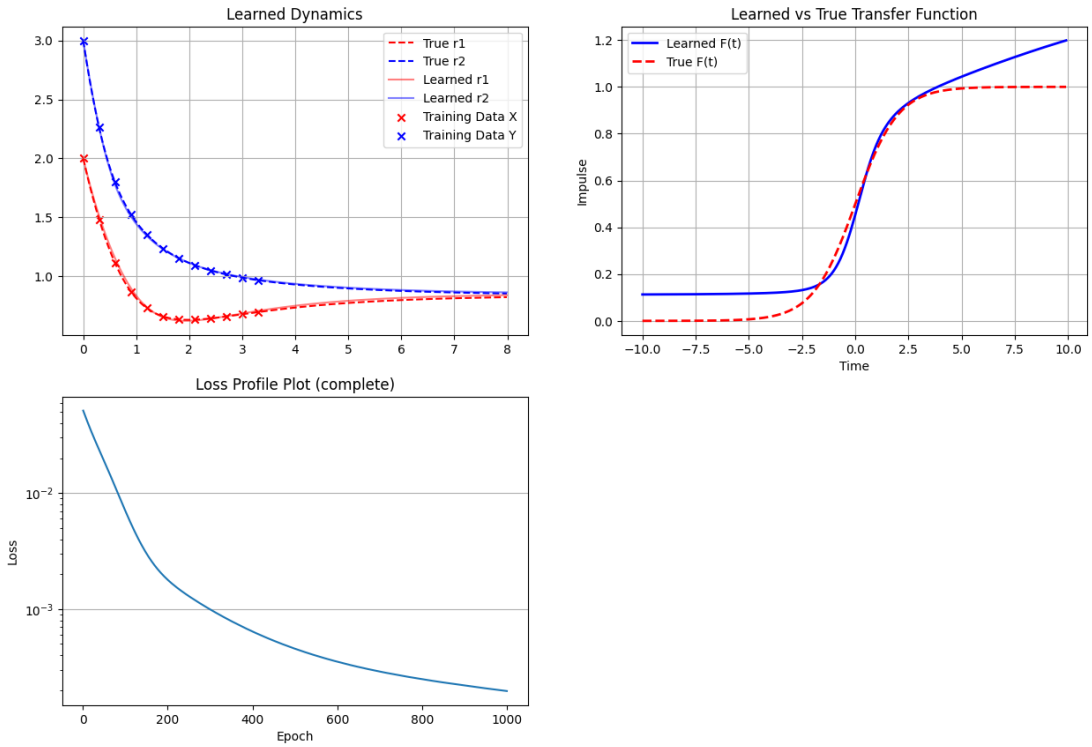


Figure 4.2.10: Winner-Take-All Mechanism Experiment 1: Results

Experiment 2: Dominance State

In this experiment, we will take the following values: $I_0 = 2, J = 10$ and the initial conditions same as the previous experiments. As we can observe in figure 1.5.15, with this set of parameters the steady state where the subpopulation with the highest initial state dominates over the other population, that is instead suppressed.

As in the previous case, we extracted our training dataset from the trajectory over the time span $t \in [0, 12]$ by performing a sampling with a constant step size of $\Delta t = 1$ limited to the time horizon $t = 4.5$. Also, we added the data point at $t = 8$. Then we defined

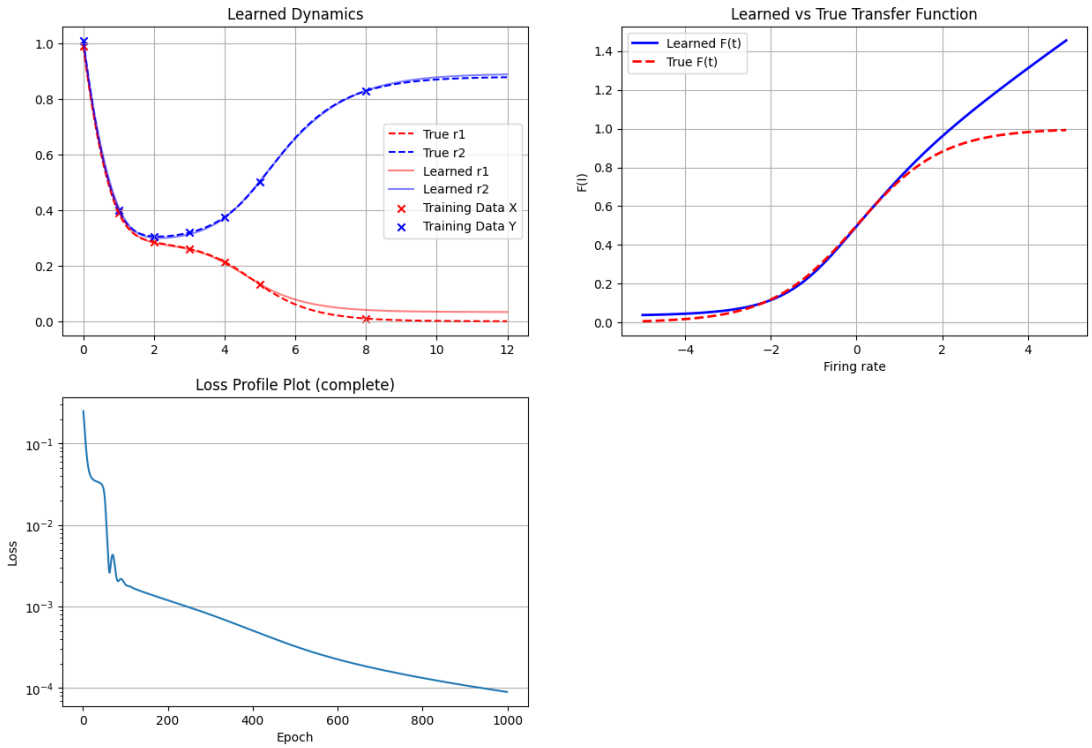


Figure 4.2.11: Winner-Take-All Mechanism Experiment 2: Results

the same UDE as in 4.12.

The UDE 4.12 was trained for 1000 epochs with the ADAM optimizer using a learning rate of 5×10^{-3}, which resulted in a model that has a final loss (L2 MSE) of 8.973×10^{-5}.
Similarly to the previous case, in figure 4.2.11 we can observe that the trained UDE is able to extrapolate the dynamics over the long term in a fairly accurate manner; however, in this case there's also a slight discrepancy with the learned transfer function. In this case, they seem to coincide in the left-side of the function, while the difference becomes clear in the right side.

Experiment 3: Network Hyperparameter Optimization

The two previous experiments yielded fairly satisfactory results, especially in terms of replicating the trajectories of the model. But, one major shortcoming of the results is in the identification of the activation function; in both cases, the UDE managed to replicate the central part of the function, but not the right side in both cases, or even the left side in the first experiment. As explained before,

this could have been due to a poor lack in the neural network architecture of the UDE 4.12.

One way of improving this result is by using hyperparameter optimization techniques on the neural network as done previously with the single firing rate neurons. To perform the hyperparameter optimization process, we used the following procedure:

- Obtain the dataset as done in the previous experiments; the dataset of the first experiment will be used for training the UDE, and the dataset of the second experiment will be used as a validation metric for the hyperparameter optimization algorithm
- Build the UDE 4.12, with the hyperparameters of the network sampled from the table 4.2.2
- Train the UDE for 300 epochs with the ADAM optimizer, using a learning rate of 0.005

As before, we used Optuna to implement the pipeline and also to visualize the results of the optimization process. We then re-trained the UDE that presented the best validation loss for 1000 epochs with the ADAM optimizer using a learning rate of 10^{-3}.

Hyperparameter	**Search Space**
Number of hidden layers	$1, 2, \ldots, 6$
Hidden units per layer	$4, 5, 6, \ldots, 128$
Activation function (for all layers)	ReLU, Leaky ReLU, SiLU, Tanh, Sigmoid

Table 4.2.2: Hyperparameter search space for Winner-Take-All UDE of Experiment 3.

Figure 4.2.12 presents the learned trajectory of the model with the initial conditions of the training dataset and the learned activation function. As we can see the model managed to learn the trajectory very accurately, even better that in the previous experiment (cf. fig. 4.2.11); at the same time, the learned activation function managed to improve relatively to the previous experiments. However, there is still the discrepancy in the right side of the activation function.

In order to investigate this question in further detail, figure 4.2.13 presents the learned trajectories on "extreme" unseen initial con-

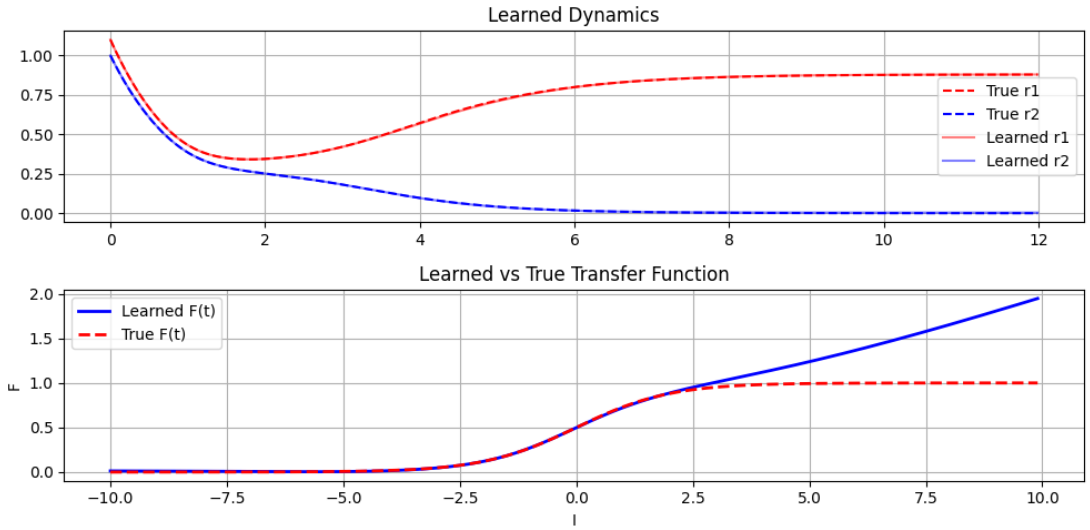


Figure 4.2.12: Learnt trajectories of Winner-Take-All UDE and Activation Function

ditions as a some sort of "stress test" on the trained UDE. In the top figure, the initial conditions are taken to be $(r_1(0), r_2(0)) = (50, 40)$ and we can see that the reproduction of the trajectory is quite faithful. However, on the other hand, in the bottom figure where the initial condition was taken to be negative values with $(r_1(0), r_2(0)) = (-1, -1.1)$, we can see that the learned and the true trajectories are completely different.

This would suggest that the UDE fails on negative values of the firing rates r_1, r_2; however, of course r_1, r_2 cannot be negative as they represent the mean activity of a neuron population. This confirms the intuition where the model managed to learn the activation function only on the values which were passed, which in our case are negative as the neurons are recurrently inhibitive.

4.2.2 General Neuron Networks

Now, let us consider the general case of coupled neuron populations as described in equation 4.10. With this model, we performed three experiments with different goals each:

1. In the first experiment we will design a "UDE" which attempts to learn only the weight matrix
2. In the second experiment we will use UDEs to learn both the

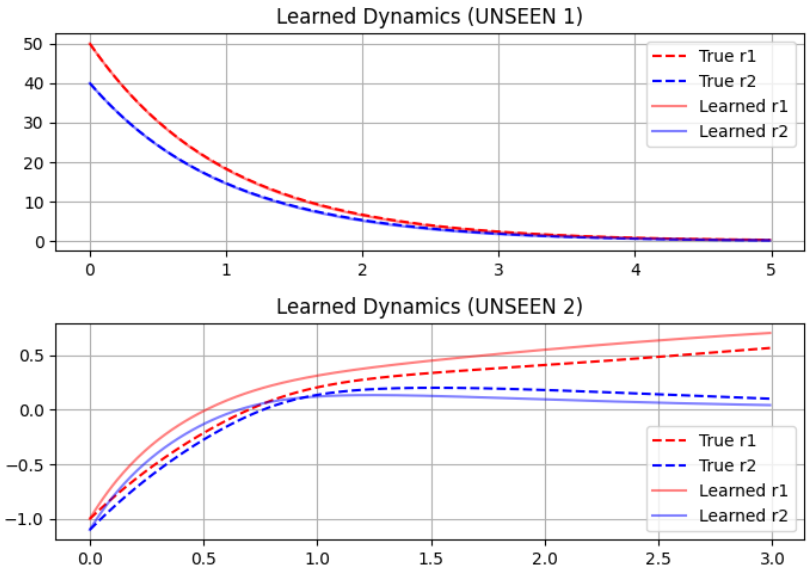


Figure 4.2.13: Learnt trajectories of Winner-Take-All UDE: Unseen trajectories. Parameters (initial conditions) described in the text

weight matrix and the activation function

3. In the third and final experiment we will use UDEs to learn both the external impulses of each neuron and the activation function

In all of the tree experiments, both the initial impulses and the weight matrices will be randomly generated.

Experiment 1: Learning Matrix Weights

In this experiment, we considered five population of neurons coupled with each other. We defined the following UDE, in order to learn the randomly generated weight matrix $W \in \mathbb{R}^{5\times5}$:

$$\forall n = 1, \dots, 5 : \dot{r}_n = -r_n + \sigma\left(I_n + \sum_{m=1,\dots,5} W_\theta[n,m] r_m\right) \tag{4.13}$$

Where $W_\theta \in \mathbb{R}^{5\times5}$ is learnable matrix of parameters. We trained the UDE 4.13 for 30 000 epochs with the ADAM optimizer using a learning rate of 0.05, and we measured both the loss of the UDE with respect to the training data (which consisted a sampling of the trajectory of the simulation from $t = 0$ to $t = 10$ with constant step size $\Delta t = 0.5$) and the Frobenius distance between the trained matrix and the real weight matrix, defined as in equation 4.14. The training set and the weight matrix are illustrated in figure 4.2.14.

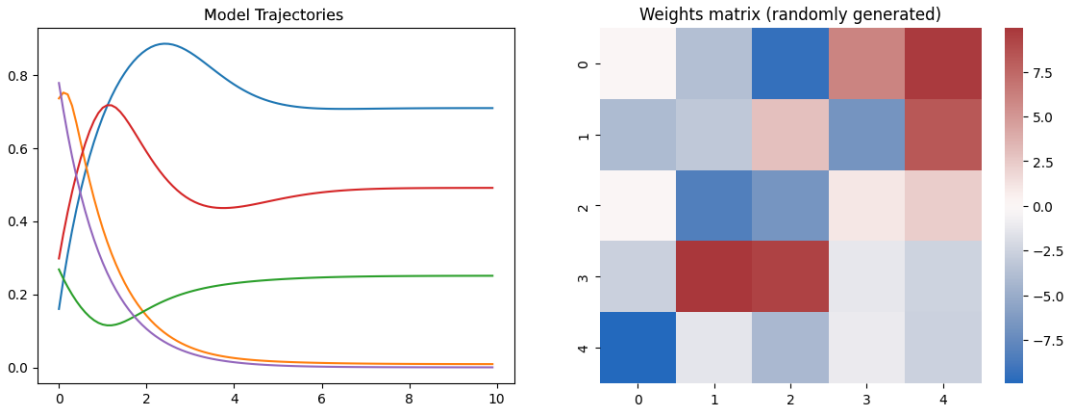


Figure 4.2.14: Dataset of Experiment 1 (left) and Weight Matrix (right). Other parameters described in the text.

$$d_{F(\mathbb{R}^{n\times n})}(A,B) := \sum_{i=1,\dots,n} \sum_{j=1,\dots,n} (A[i,j] - B[i,j])^2 \tag{4.14}$$

The final model yielded a loss of 1.956×10^{-8} and is able to extrapolate the dynamics in a very accurate manner (fig. 4.2.15)

However, the Frobenius distance between the matrices reached the value of 14.082, which normalized by the matrix size is 0.563. As we can also observe in the differences between the weights (fig. 4.2.15), we can see that unfortunately the model did not manage to learn the matrix weights in a faithful manner, as there are some elements of the learned weights with significant discrepancies.

Moreover, we can observe a dual behavior between the profiles of the loss and the Frobenius distances (fig. 4.2.15). On on side, at a certain point the loss becomes noisy and starts oscillating between the values of 10^{-8} and 10^{-6}; on the other hand, the Frobenius distance kept decreasing very slowly, albeit without any form of noise as the loss profile started to present the noise.

Experiment 2: Learning Matrix Weights and Activation Function

In the previous experiment, we attempted to learn the weight matrix by assuming that we know the transfer function σ. With this experiment, we want to take a more realistic assumption by not knowing such function and making it a learnable dynamics for the UDE. In this experiment, we will consider a network of three coupled firing rate neurons and a sparse weight matrix.

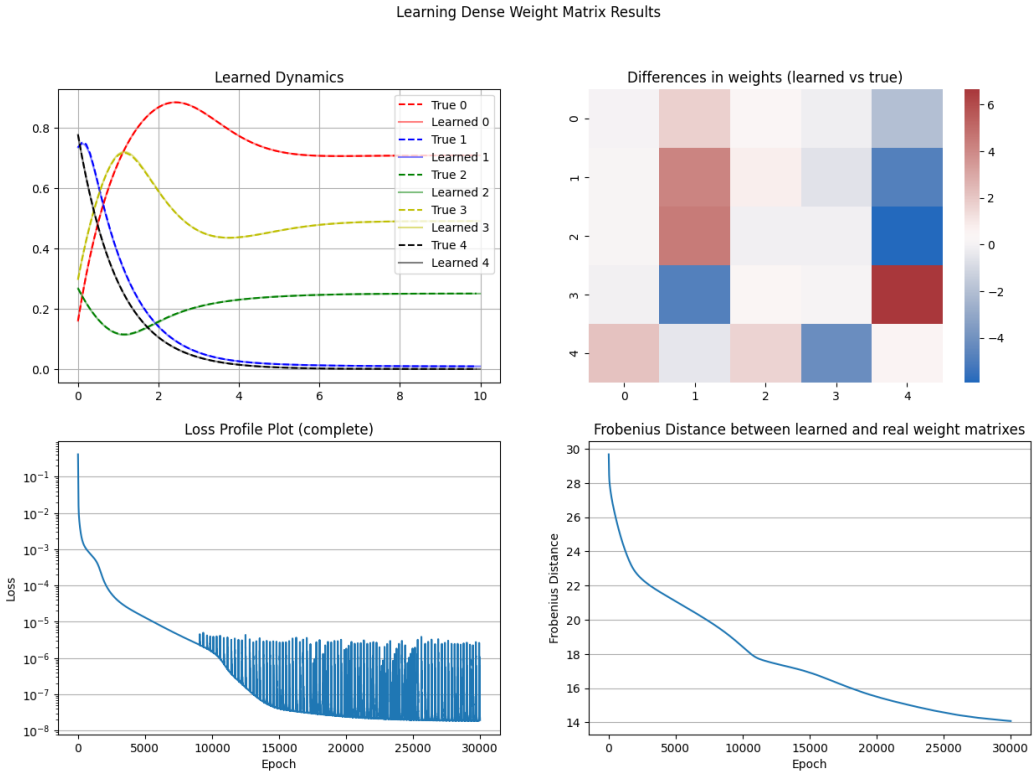


Figure 4.2.15: General Neuron Networks Experiment 1: Results

We extracted the dataset from a sampling of the simulated trajectory in the time span $t \in [0, 10]$ with constant step size $\Delta t = 0.5$; the dataset is shown in figure 4.2.16 The UDE is defined as in 4.15 where W_{θ_1} is the learnable weight matrix and U_{θ_2} is the learnable activation function. In particular, U_{θ_2} consists of an input layer, four hidden layers with 64 neurons each and an output layer. For each hidden layer we applied the Leaky ReLU activation function and on the output layer we applied the softplus function.

$$\forall n = 1, 2, 3 : \dot{r}_n = -r_n + U_{\theta_2}\left(I_n + \sum_{m=1,2,3} W_{\theta_1}[n, m] r_m\right) \tag{4.15}$$

We trained the UDE 4.15 with the ADAMax algorithm using a learning rate of 0.01 for the first 10 000 epochs, and then with a learning rate of 0.05 for the next 10 000 epochs and then applied a sparsity regularization term in the loss for the last 5000 epochs. The sparsity regularization term is defined as in equation 4.16

$$R(\theta_2) := 10^{-7}\left(\sum_{i,j \in \{1,2,3\}^2} |W[i, j]|\right) \tag{4.16}$$

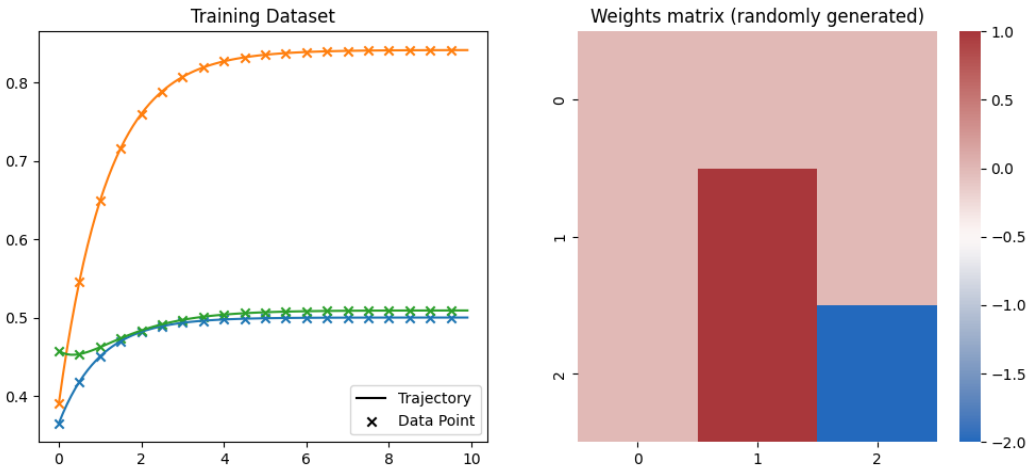


Figure 4.2.16: Dataset of Experiment 2. A similar description can be found in figure 4.2.14. Parameters described in the text.

In a similar manner to the previous experiment, we tracked both the loss and the 2-norm distance between the learned weights and the actual weight matrix. Similarly to what we observed in the previous experiment, the model is able to replicate the trajectories very well (fig. 4.2.17), but at the same time the learned matrix weight differed from the actual one in a significant manner. Also, the dual behavior of the loss profile and the 2-norm distance profile is even more accentuated in this case (fig. 4.2.17).

Moreover, we can observe that the learned activation function coincides with the real one only at a certain point; a similar effect was also obtained in some of the experiments of the previous subsection.

In an attempt to improve the results, we applied the dropout regularization technique ([17]) on the neural network. In our first approach, we attempted to reuse the previously trained model by injecting dropout layers on all hidden layers with dropout probability $p = 0.6$ and trained the model further for 3000 epochs with a learning rate of 10^{-5}. In a certain sense, we wanted to fine-tune the model where the final model is re-trained for deeper adaptation on the dataset with regularization techniques. Figure 4.2.18 illustrates the result of the re-training and while we can see that the 2-norm based distance between the matrices managed to lower to a value of around 1.1, unfortunately after the 2000th epoch it started to rise again. Moreover, the performances in learning dynamics are

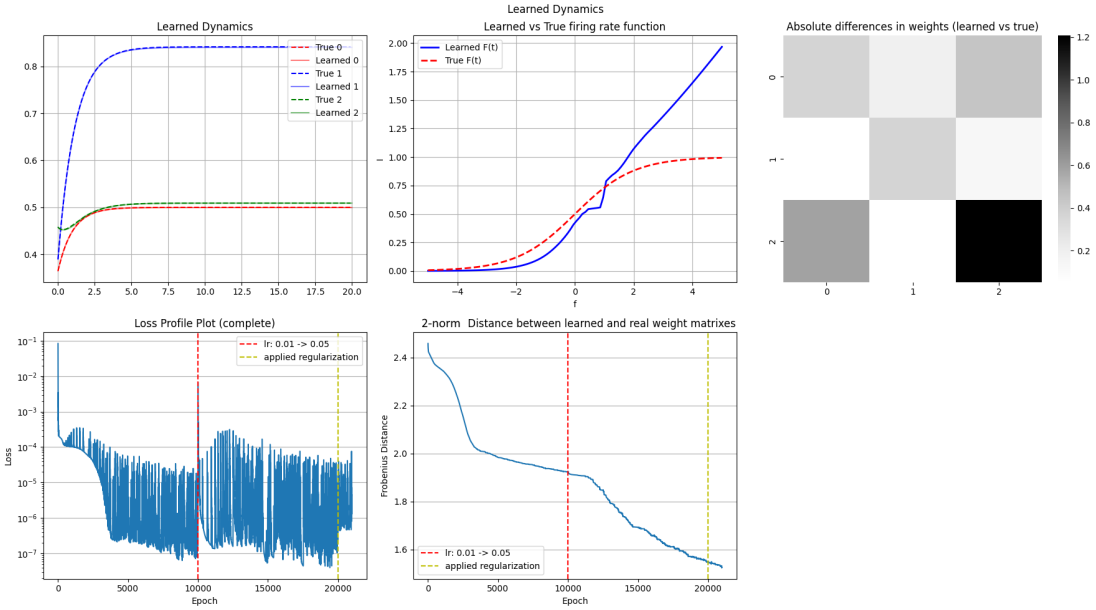


Figure 4.2.17: General Neuron Networks Experiment 2: Results

degraded as both the learned trajectories and activation function became fit their respective original forms worse.

So, as the previous approach might not have worked, we re-trained the UDE 4.15 from scratch, applying the dropout layers in the completely same manner as before, for 10 000 iterations with the ADAMax optimizer using a learning rate of 0.01. Also, we changed the dataset by limiting its time horizon to $t_{\max=7}$ (from $t_{\max=10}$) but decreasing the time-step to $\Delta t = 0.05$ (from $\Delta t = 0.1$).

This yielded a final model with a loss of 2.778×10^{-5} and 2-norm difference of 1.426, and figure 4.2.19 shows the results. Unfortunately, did still not fix the issue of identifying the activation function; rather, it seems that the learned activation function is even more inaccurate than the training without the dropout layers. Moreover, the loss profile plot entered in a "noisy" state earlier than the previous case. Rather, the true advantage of applying dropout might be in the fact that the 2-norm distance reached a lower value more quickly, as with dropout regularization the distance reached a value of around 1.4, while in the original case it never reached such value.

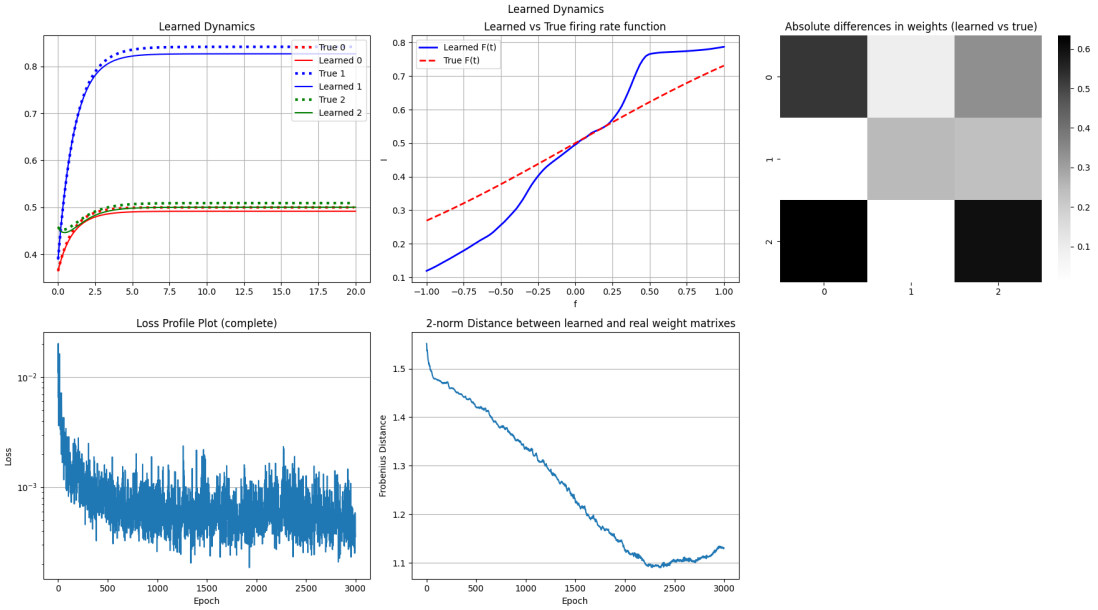


Figure 4.2.18: General Neuron Networks Experiment 2 Follow-Up: Results (fine-tuning)

Experiment 3: Learning Impulses and Activation

In the last experiment, we decided to see whether UDEs would be able to learn a simpler parameter describing the network, that is the external impulses described as a vector $I \in \mathbb{R}^n$.

In this experiment, we considered a network of 50 coupled neuron populations with a randomly generated matrix and a randomly generated impulse constant vector. From this, we extracted the training set by simulating the trajectories in the time span $t \in [0, 2]$ and taking a sample valued in the time stamps $t = 0, 1, 2$. Figure 4.2.20 contains the generated dataset and the weights matrix.

The UDE is defined as in equation 4.17, where $U_1(\bullet; \theta_1)$ is the neural network that learns the activation function and $U_2 \in \mathbb{R}^{50}$ is a learnable parameter which represents the external impulse constants of each neuron.

$$\forall n = 1, \dots, 50 : \dot{r}_n = -r_n + U_1 \left(U_2[n] + \sum_{m=1,\dots,50} W[n, m] r_m; \theta_1 \right) \tag{4.17}$$

We trained the UDE 4.17 for 2000 epochs with the ADAM optimizer using a learning rate of 0.003. During the training process, we

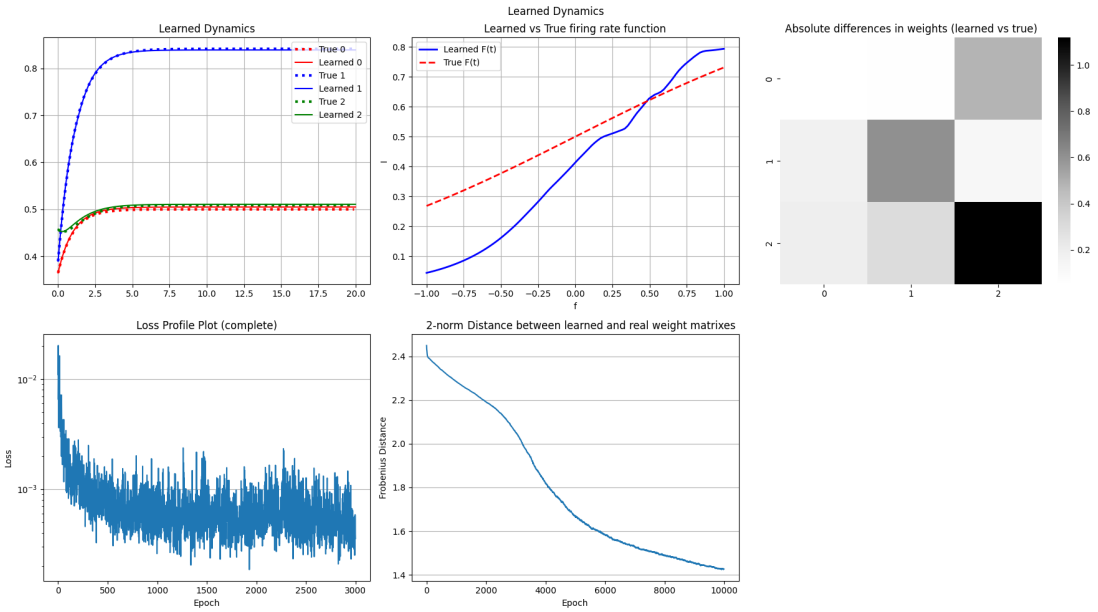


Figure 4.2.19: General Neuron Networks Experiment 2 Follow-Up: Results (retraining from scratch)

tracked both the losses as the L2 MSE loss and the 2-norm difference between the learned and the actual impulse vectors (as in eq. 4.18).

$$d_2(\underline{x}, \underline{y}) := \sum_{i=1,\dots,n} (x[i] - y[i])^2 \tag{4.18}$$

The final model yielded a loss of 1.156×10^{-5} and final 2-norm difference of 3.186.

Like in all of the previous cases, the model is able to replicate the trajectory very accurately with a very small amount of training data and even extend it on longer time horizons (fig. 4.2.21). Surprisingly, the model also successfully recreates the activation function very accurately (fig. 4.2.21), unlike in the other cases.

However, as seen in the previous experiments, the model failed to learn the parameters that characterize the dynamics of the network, which in this case is the impulse vector I. Also, we can still observe the dual behavior in the plots of the losses and the 2-norm differences, where the losses suffer from a form of noise but the 2-norm differences do not (fig. 4.2.21).

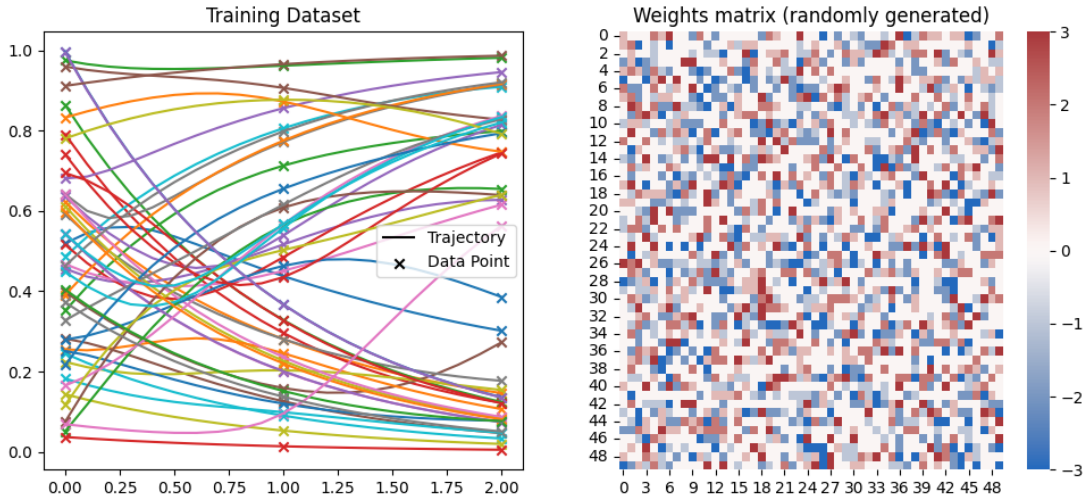


Figure 4.2.20: Dataset of Experiment 3. A similar description can be found in figure 4.2.14. Parameters described in the text.

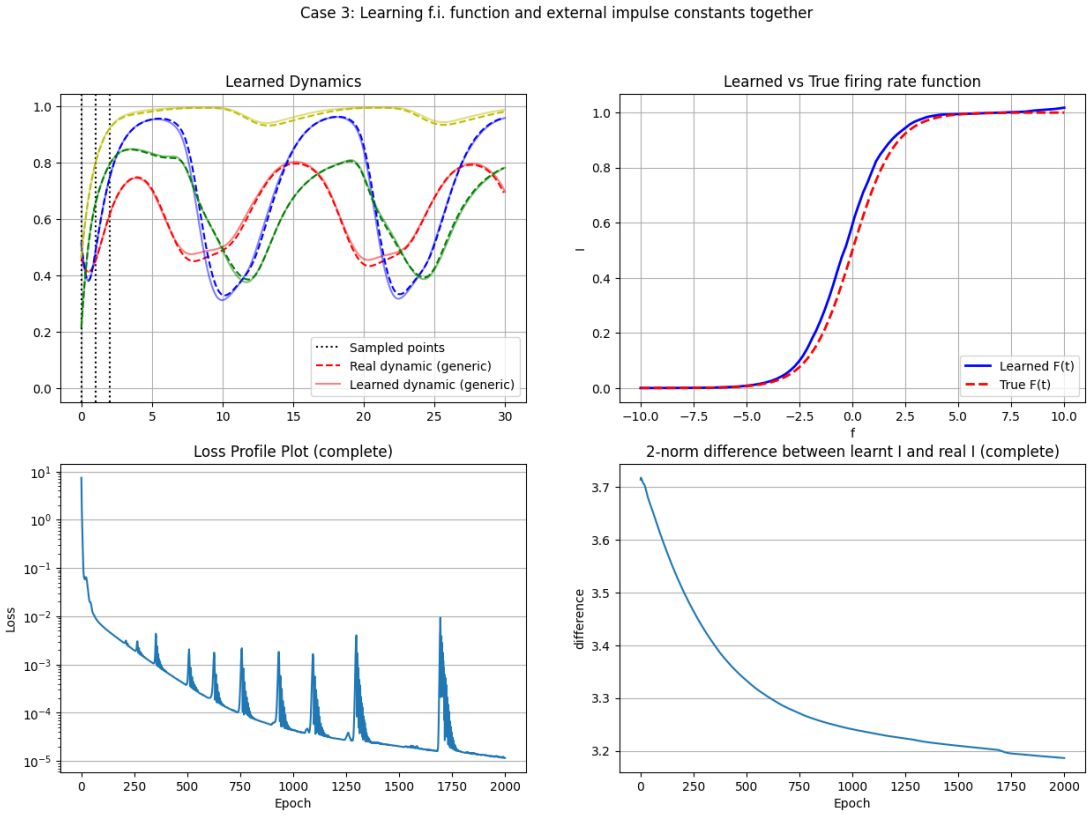


Figure 4.2.21: General Neuron Networks Experiment 2: Results

Chapter 5

Advanced Case Study II: Chua Model, a Prototypical Chaotic Model

This chapter illustrates the results of the applications of the methods explained in the previous chapters, and of other methods to the chaotic Chua Model.

5.1 Basic concepts on chaotic systems

Chaos is one of the most interesting phenomenons which can be observed with dynamical systems. As the mathematician James A. Yorke (who also coined the term "chaos") would put it, chaotic dynamical systems are dynamical systems that exhibit an "erratic kind of motion"; in other words the trajectory of a chaotic dynamical system is a bounded and non-periodic orbit [8]. One of their distinguishing properties is the sensitive dependence on initial conditions [25], which consists of exponential divergence between two trajectories starting from slightly different initial conditions. Namely, an important quantity associated to chaotic systems is the Maximum Lyapunov Coefficient λ, which is the value such that the follow approximation holds:

$$\|\delta(t)\| \sim \|\delta_0\| e^{\lambda t} \tag{5.1}$$

Where $\delta(t)$ is a tiny separation vector between the two initial conditions (say, for example $\|\delta_0\| = 10^{-15}$). In a certain sense, chaos acts as a sort of interface between determinism and randomness: from deterministic equations (ODEs) we derive in trajectories that are hard to predict due to their sensitivity to initial conditions.

One among the most important and simplest models that are endowed of chaotic behavior is the mathematical model of the Chua's circuit [6]. This model is extremely important for three reasons:

1. This is a model that is extremely rich in behavior and a large literature exists on it
2. The Chua circuit allowed the first experimental proof that Chaos exists and it is not only a mathematical hypothesis: through oscilloscopes one could observe the chaotic attractors [5] [10]
3. The electric circuit is very easily and inexpensively implementable: as a matter of fact, in 2005 K. O'Donoghue et al. [10] was able to to implement the circuit with around €5

The dimensionless Chua model reads as follows 1.30:

$$\begin{aligned} \dot{x} &= \alpha(y - g(x)) \\ \dot{y} &= x - y + z \\ \dot{z} &= -\beta y - \gamma z \end{aligned} \tag{5.2}$$

In our case, we will consider $g(x)$ be a cubic polynomial

$$g(x) := ax^3 + cx$$

The most typical chaotic attractor generated by the above model is the so-called *Double Scroll* (DS) strange attractor, as initially observed in [6]. Figure 5.1.1 illustrates two instances of the DS attractor, which is generated by simulating the model over a time horizon of $t \in [0, 35]$ with parameters $\alpha = 10, \beta = 16, \gamma = 0, a = 1, c = -0.143$ (the same as proposed in [6]) and initial states $\underline{x_0}^{(1)} = (0.0001, 0.1, 0)^T; \underline{x_0}^{(2)} = (0.0001, 0.1, 0.1)^T$.

In this chapter, we will apply the UDE approach on a sampling of data generated from the model 5.2, in an attempt to (1) learn the real trajectory of the model (especially DS), and then to (2) apply symbolic regression and identify the equations of the model with only the sample.

5.2 Learning the Double Scroll Attractor

In this section, we will focus on learning the dynamics of the Double Scroll attractor.

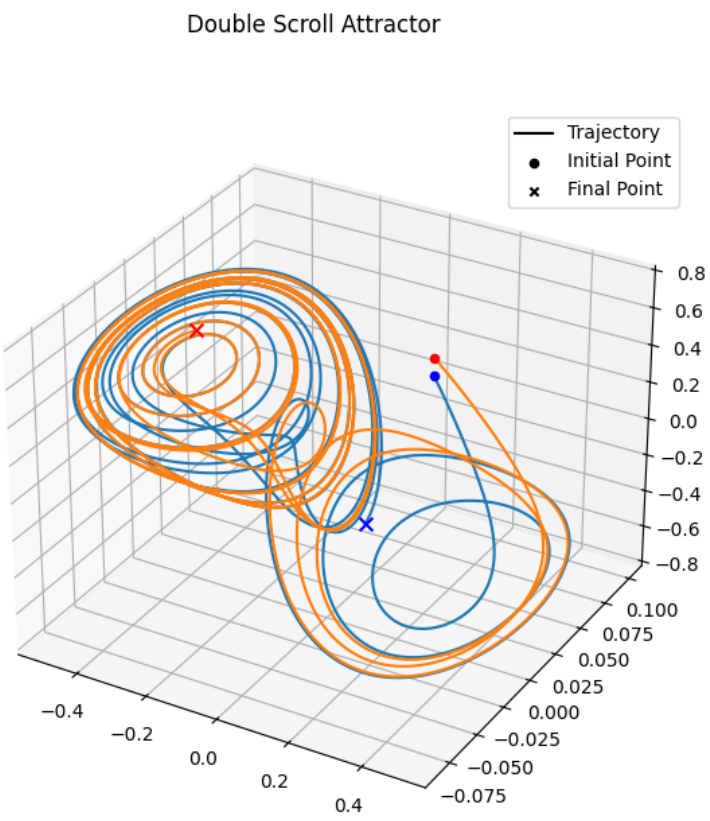


Figure 5.1.1: Two trajectories of the Double Scroll attractor. Parameters described in the text.

5.2.1 Experiment 0: Calculating Lyapunov Exponents

In this experiment, rather than focusing on applying UDEs to the Chua electric circuit model, we will make some preliminary calculations which characterize some specific chaotic attractors. In particular, we will consider the Double Scroll attractor which is obtained by setting the parameters $\alpha = 10, \beta = 16, \gamma = 0, a = 1, c = -0.143$ (same as the one used to produce figure 5.1.1) and we will calculate its characteristic time τ which is the inverse of the maximal Lyapunov exponent given by considering the distance

$$\delta(t) := \|\mathbf{x}(t) - \mathbf{y}(t)\|,$$

where $\mathbf{x}(t)$, $\mathbf{y}(t)$ are two trajectories of the model with different initial conditions.

To calculate τ, we considered the initial condition $\mathbf{x}_0 = (0.0001, 0.1, 0)$ and its small perturbation $\tilde{\mathbf{x}}_0 = \mathbf{x}_0 + \delta_0 \mathbf{v}$, where $\mathbf{v}$ is a random unit

vector in $\mathbb{R}^3$ and $\delta_0 = 10^{-10}$. From these two initial conditions we obtained the trajectories $\mathbf{x}(t), \mathbf{y}(t)$, from which we calculated $\delta(t)$. Then we found the best fit of λ for $\delta(t) \approx \delta_0 e^{\lambda t}$ by fitting a linear regression on the logarithm of the distances (on a limited dataset, as at a certain point the distance starts oscillating) and setting λ as the first coefficient directly. In the end, we obtain τ as $\tau = \frac{1}{\lambda}$.

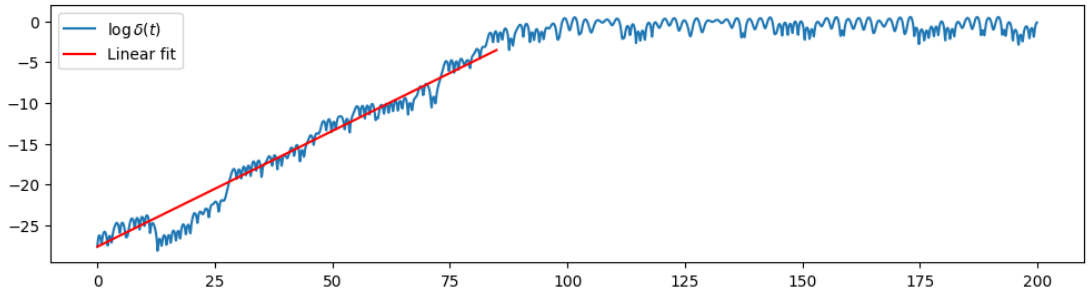


Figure 5.2.2: Experiment 0: Linear regression on the logarithmic norm-difference

As a result, we obtained the value

$$\lambda = 0.28587651, \tau = 3.4980139.$$

5.2.2 Experiment 1: Model Comparisons

OBJECTIVES In this experiment, we will train more models on a dataset sampled from the DS attractor and compare their performances in order to understand which model is able to handle better chaoticity. In the end, we will also study the generalization capacities of the best UDE model.

METHODOLOGY The models used for training are the following:

1. A Neural Ordinary Differential Equation, which serves as a baseline reference for the following UDEs
2. A Universal Differential Equation where the polynomial term $g(x)$ is assumed to be unknown and replaced by a neural network $U(x;\theta)$, as in equation 5.3. $U(x;\theta)$ is a fully-connected neural network with 5 hidden layers, each with 128 neurons and using the ReLU activation function.
3. A PEM-UDE as proposed by Anthony G. Chesebro et al. [51]. The UDE is analogous to the previous one, i.e. the neural network is the same and "replaces" the polynomial term. The new

part consists of the linear interpolation being used to estimate $\hat{x}, \hat{y}, \hat{z}$ and setting the hyperparameter $K = (K_x, K_y, K_z) = (0.3, 0.1, 0.1)$. The PEM-UDE model reads as in equation 5.4.

$$\begin{aligned} \dot{x} &= \alpha(y - U(x;\theta)) \\ \dot{y} &= x - y + z \\ \dot{z} &= -\beta y - \gamma z \end{aligned} \tag{5.3}$$

$$\begin{aligned} \dot{x} &= \alpha(y - U(x;\theta)) + K_x \varepsilon_x(t) \\ \dot{y} &= x - y + z + K_y \varepsilon_y(t) \\ \dot{z} &= -\beta y - \gamma z + K_z \varepsilon_z(t) \\ \varepsilon_x(t) = \hat{x}(t) - x; \varepsilon_y(t) &= \hat{y}(t) - y; \varepsilon_z(t) = \hat{z}(t) - z \end{aligned} \tag{5.4}$$

To solve the UDEs defined above, the Tsitouras' 5/4 method [15] was employed. The models were trained with the ADAM optimizer for 500 epochs with a learning rate of 0.01.

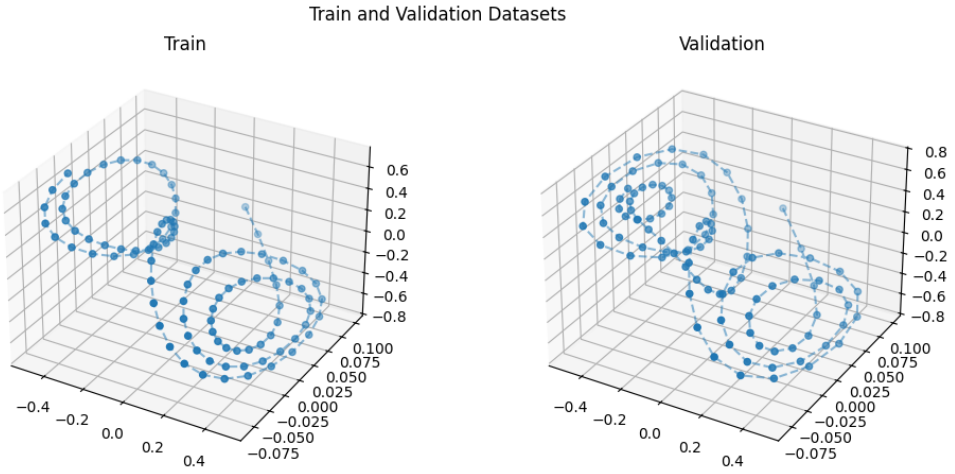


Figure 5.2.3: Experiment 1: Dataset. Parameters are the same of the trajectories presented in figure 5.1.1

RESULTS AND DISCUSSION Figures 5.2.4 and 5.2.5 illustrate the loss curves of each model. As we can see from the first figure (fig. 5.2.4), the classic UDE approach is particularly instable for approximately the first 100 epochs, even hitting a peak of 3.5220×10^{16} at the 57-th iteration, which then dropped to 2.2744×10^{0}. This highlights the shortcomings of the classical UDE approach in learning

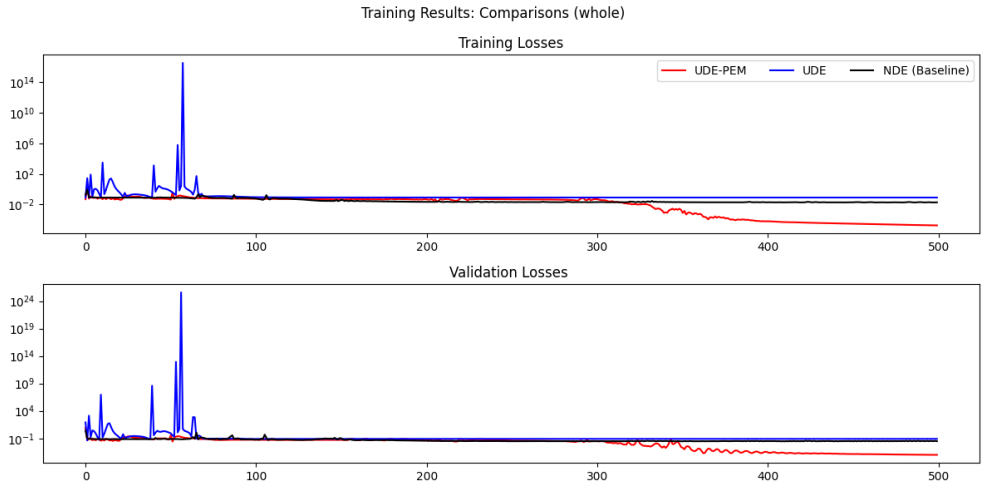


Figure 5.2.4: Experiment 1: Results. The blue line is the loss curve of the original UDE model, the red line is the loss curve of the PEM-UDE variant model, and the black line is the loss curve of the baseline mode. Upper panel: training losses. Lower panel: validation losses.

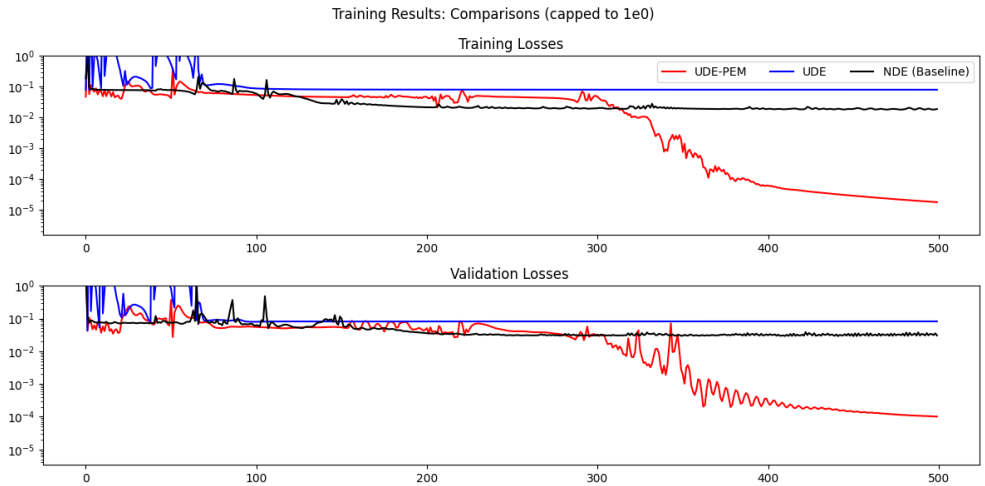


Figure 5.2.5: Experiment 1: Results (zoomed). For a description of this figure refer to the caption of the figure above.

from certain scenarios (in particular chaotic models), as it shows extreme instability in the training phase and it fails to improve the baseline metrics with its training loss converging to a value of the order of 10^{-1}.

On the other hand, the UDE-PEM variant shows a marked improvement with respect to the classical UDE method, as it reached a loss value of the order of 10^{-4} in both training and validation datasets. From this result, we can therefore conclude that the UDE-PEM variant is the best model to fit on datasets derived from the chaotic attractors of Chua circuit model.

This difference in performances can be more evidently visualized by comparing the trajectories of the original UDE and the PEM-UDE model as shown in figures 5.2.6 and 5.2.7. On one hand, the original UDE is not even able to reproduce the oscillations in the training set; while, on the other hand, the PEM-UDE is able to even slightly generalize the trajectories, although the trajectories start to diverge very quickly.

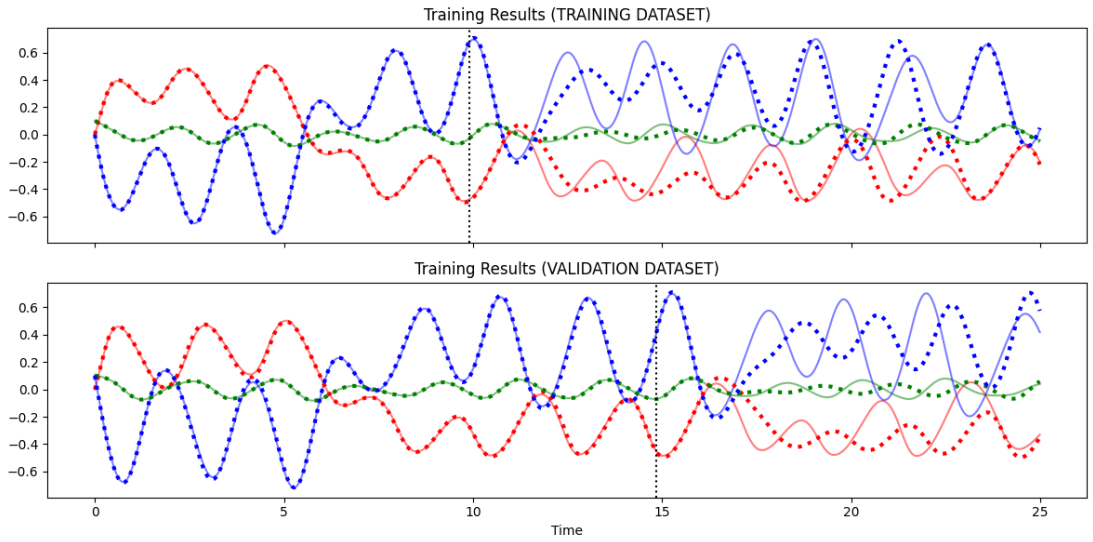


Figure 5.2.6: Experiment 1: Predictions beyond the datasets (of the PEM-UDE). The continuous and transparent lines represent the predicted trajectory, the dotted lines represent the ground truth. The vertical lines represent the time instances where the training or validation dataset end.

To quantify the idea of a "period" of convergence duration, we estimated t_{eff} as the timestamp where the norm of the absolute differences is greater than 10^{-1} and divided this quantity by the charac-

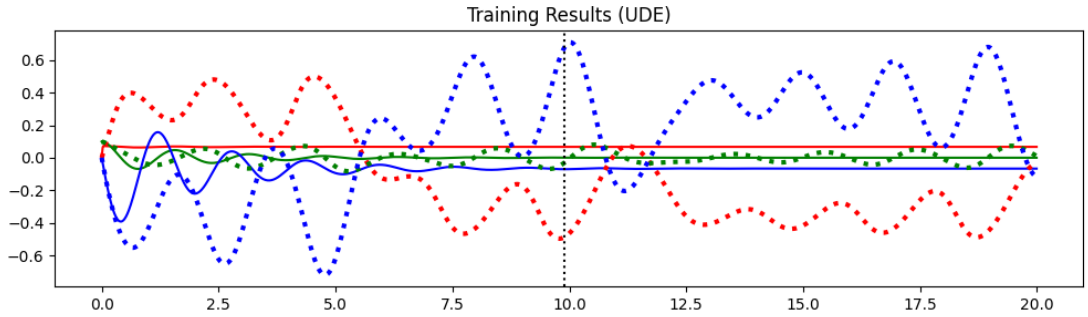


Figure 5.2.7: Experiment 1: Predictions beyond the datasets (of the original UDE). For a description of this figure refer to the caption of figure 5.2.6

Initial Condition (x_0, y_0, z_0)	$\frac{t_{\text{eff}}}{\tau}$
(Training $\mathbf{x}_0$) $(0.0001, 0.1, 0)$	42.023%
(Validation $\mathbf{x}_0$) $(0.0001, 0.1, 0.1)$	176.385%
$(0.1, 0.1, 0.1)$	72.326%

Table 5.2.1: Experiment 1: Calculated characteristic time proportions

teristic time τ. In a certain sense, the proportion $\frac{t_{\text{eff}}}{\tau}$ measures how many Lyapunov times beyond the training horizon the model can predict. This idea has been introduced in the literature on machine learning applications to chaotic dynamical systems as an assessment metric for the models [46].

The values are indicated in table 5.2.1. In most cases, the model is able to generalize up to the 42.023% of the characteristic time. However, notice also that the result is also significantly better with the initial state of the validation dataset, as it generalizes the 176.385% of the characteristic time. In any case, these values indicate mild or decent generalization abilities of the model.

5.2.3 Experiment 2: Modeling the unknown term

INTRODUCTION In the previous experiment, we assumed that the UDE was defined as in equation 5.3, where $U(x;\theta)$ is a scalar fully-connected neural network, that is a parametrized function of form $U(\bullet;\theta) : \mathbb{R} \longrightarrow \mathbb{R}$. Of course, this is done under the assumption that the modeler knows that the non linear terms only depends on x.

However, there are some cases where this prior knowledge is not

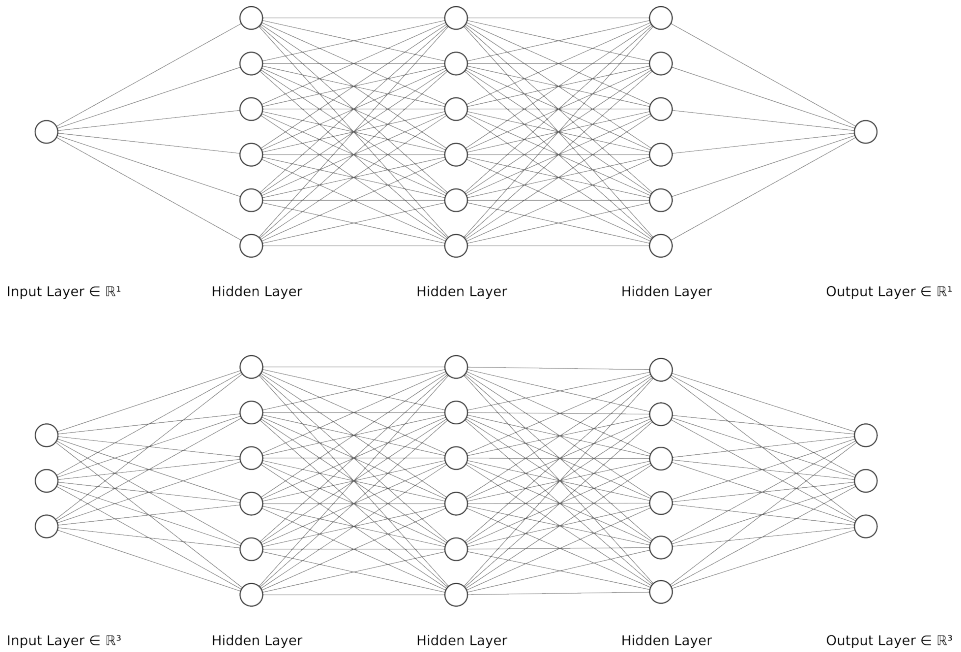


Figure 5.2.8: A qualitative idea on the forms of the neural networks of the UDEs. The actual size of the implemented neural networks do not coincide with this figure for graphical reasons. Upper panel: UDE 5.3. Lower panel: UDE 5.5.

available or unclear. So, we performed an additional experiment where we replaced the universal approximator term of UDE 5.3 with a fully-connected neural network $U(\bullet, \bullet, \bullet; \theta) : \mathbb{R}^3 \longrightarrow \mathbb{R}^3$, of which we only consider one term. In a certain sense, the ANN elaborates on the entire dynamics and then we only consider its "calculations" along the x variable. So, the UDE 5.3 becomes the following:

$$\begin{aligned} \dot{x} &= \alpha(y - U_x(x, y, z; \theta)) \\ \dot{y} &= x - y + z \\ \dot{z} &= -\beta y - \gamma z \end{aligned} \tag{5.5}$$

METHODOLOGY For the dataset we considered the same parameters and initial conditions of the previous experiment, but changed the time ranges of the training and validation datasets respectively to $t \in [0, 25]$, $t \in [0, 37.5]$ as well as the sampling step sizes $\Delta t = 0.01, 0.02$ (for the numerical resolution the initial step size was fixed to 0.01 in both cases). Both the UDEs 5.3 and 5.5 (actually the PEM-UDE variants, in this experiment we will automatically assume the presence of the PEM correction term) were trained for

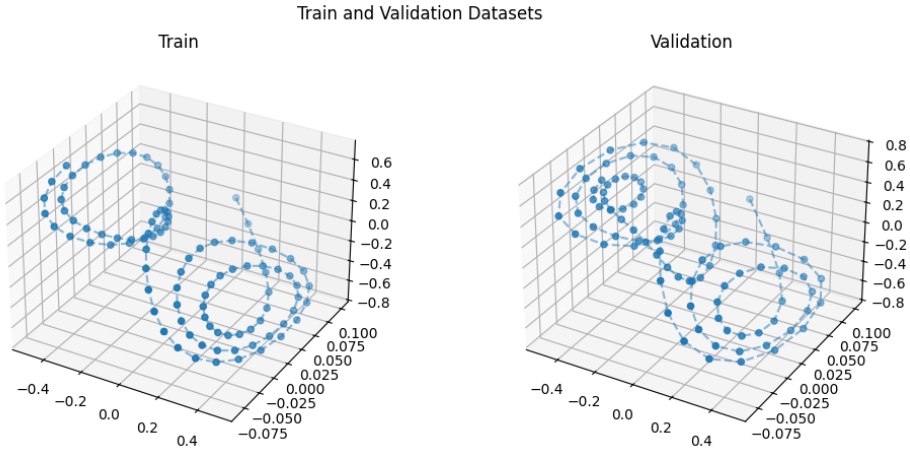


Figure 5.2.9: Experiment 2: Dataset. Parameters described in the text.

1000 iterations with the ADAM optimizer using a learning rate of 0.001, evaluating the validation loss every 100 epochs to lighten the computational costs.

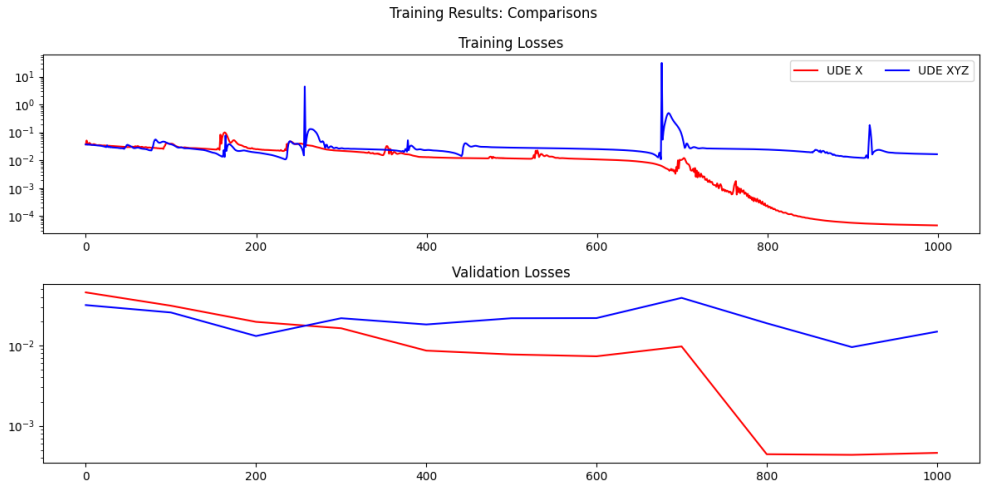


Figure 5.2.10: Experiment 2: Loss curves. Upper panel: Training loss. Lower panel: Validation loss. The red lines indicate the losses of UDE with one input variable, whereas the blue lines indicate the losses of the UDE with three input variables.

RESULTS AND DISCUSSION Figure 5.2.10 illustrates and compares the training and validation losses of the UDEs throughout the training process.

From the results we can clearly see that the UDE that uses only the x variable (UDE 5.3) outperforms the other UDE which uses all

of the states (UDE 5.5): the former manages to converge towards a training loss of 4.6176×10^{-5}, while the latter conveges towards the value in training loss of 1.6503×10^{-2}. Hence we can see that the designation of the unknown term is very important for fitting the dynamics, as we can obtain significant differences in the fitting capabilities of the model.

5.2.4 Experiment 3: Dynamics Extrapolation

Up until now, we have only trained the UDEs and analyzed how well they generalize by simply solving them for longer time intervals or unseen initial conditions.

OBJECTIVES In this experiment, we will also consider applying symbolic regression (SINDy) to extrapolate dynamics of a chaotic model from the dataset. As done similarly with the generalised logistic model and with the Lotka-Volterra equations (see chapters 2, 3), we will train three types of SINDy models: one is the "baseline" model which will not use the UDE in any way, while the other two leverage UDEs in an attempt to get a more accurate SINDy model.

METHODOLOGY In light of the results of the previous two experiments, for this experiment we will be using a PEM-UDE with a scalar neural network which "replaces" the unknown term $g(x)$. The model will be trained on a longer training set, which is extrapolated from a simulation of the model using the same parameters as of experiments 1 and 2, setting instead the time window to $t \in [0, 60], [0, 90]$ (training and validation). Moreover, the trajectories were sampled with a constant step size $\Delta t = 0.25, 0.2$ (training and validation).

Before training the PEM-UDE effectively, we used hyperparameter optimization tools to figure out the best model architecture for learning the dataset presented in figure 5.2.11. In particular, we focused on the PEM-UDE's hyperparameter K (see eq. 1.14), which represents a smoothing factor on the loss landscape as explained in [51]. The hyperparameter search space is described in table 5.2.2. The best model, with a training and validation loss of the order of

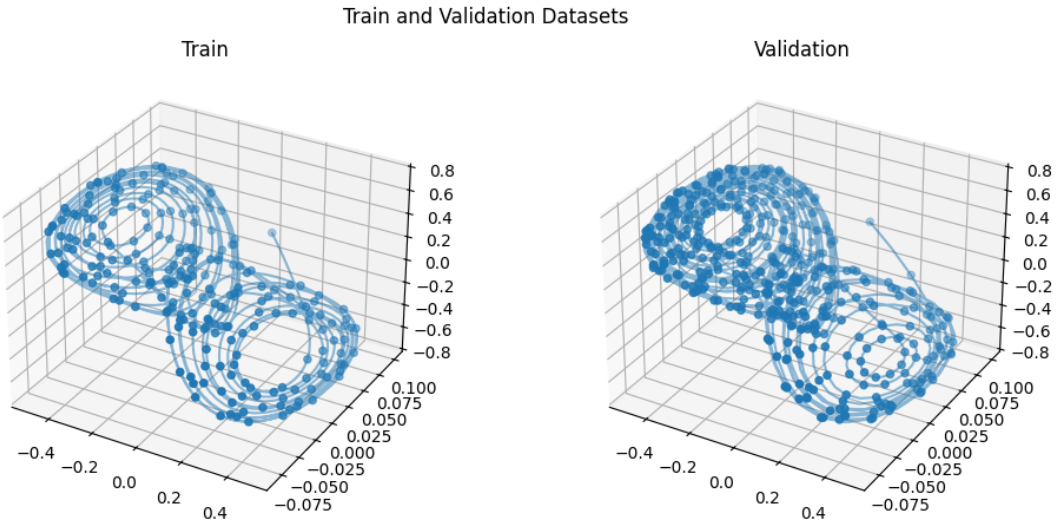


Figure 5.2.11: Experiment 3: Dataset. Parameters described in the text.

approximately $10^{-4.503}$, used the values $K = [0.9, 0.6, 0.6]$ with the swish activation function.

Hyperparameter	**Search Space**
K	$\{0, 0.1, 0.3, 0.6, 0.9\}^3$
Zero-initialize the last layer	Yes or No
Activation function (each layer)	ReLU, SiLU (swish), Leaky ReLU, Tanh, Sigmoid

Table 5.2.2: Experiment 4: Hyperparameter search space

RESULTS AND DISCUSSION The PEM-UDE was trained for 1000 epochs with ADAM, using a learning rate of 0.001. Figure 5.2.13 (left) illustrates the loss curves produced during the training process, which shows that the model managed to learn the training and validation trajectories with a satisfying accuracy. The only issue with the training process is that, as also observed in some experiment of the previous sections, the training loss curve presents some a form of noise with peaks appearing around every 400 epochs.

Then we also plotted the learned nonlinear term $U(x;\theta) \approx g(x)$ with figure 5.2.13 (right), from which we can see that the model manages to learn the true nonlinear term $g(x)$ accurately inside the range of our training dataset along the variable x. But, outside the dataset range the model fails to generalize well the nonlinear term.

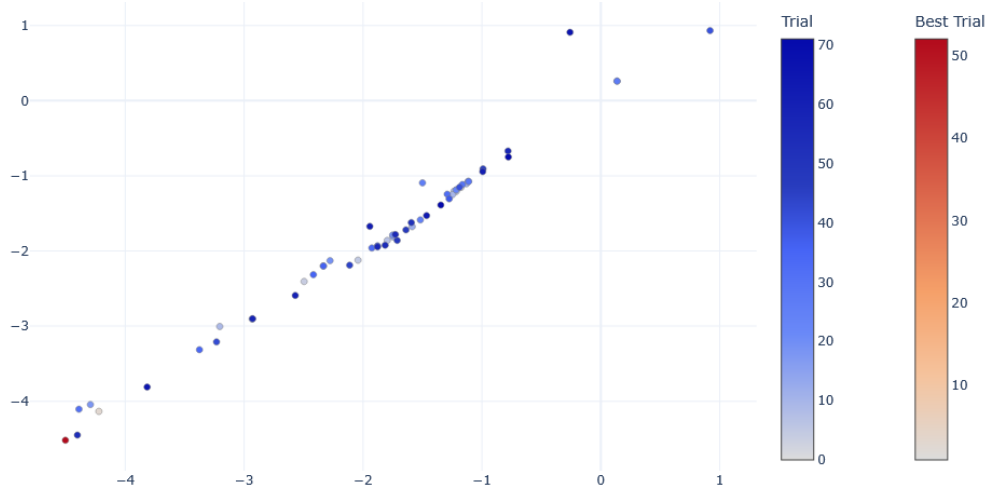


Figure 5.2.12: Experiment 3: Various results of trials done during the hyperparameter optimization process. Horizontal axis: $\log_{10}$ of the training loss. Vertical axis: $\log_{10}$ of the validation loss.

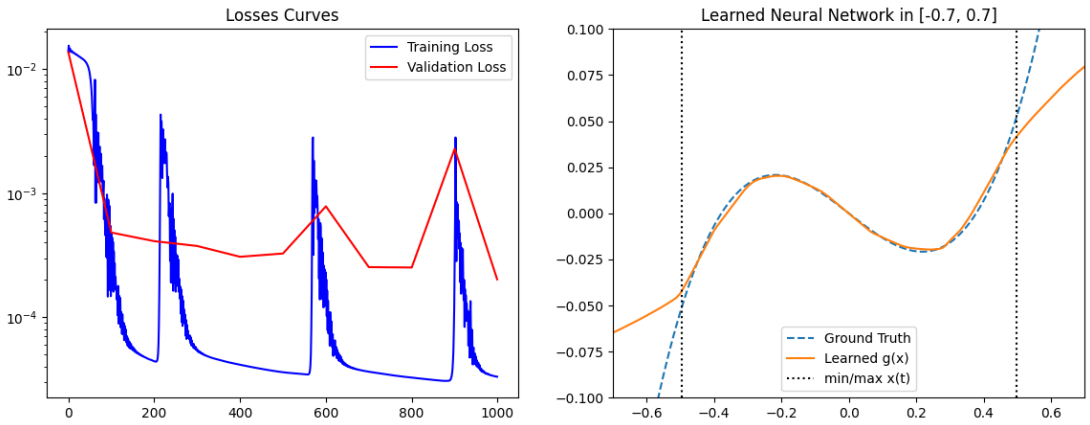


Figure 5.2.13: Experiment 3: Results. Left panel: Training and validation loss curves. Red line is the validation loss, blue curve is the training loss. Right panel: Plot of trained neural network, compared with the ground truth. Orange line is the learned function $U(x;\theta)$, the dotted blue curve is the real cubic polynomial.

However, as we plotted the learned trajectory with figure 5.2.14, we saw that the model is unable to replicate the exact trajectory already before the last timestamp of the trajectory set $t_{\max} = 60$, and rather starts experiencing a "delay" from the true trajectory starting from $\tilde{t} \approx 28$. This observation will be important for motivating the definition of the two UDE-enhanced SINDy models. We then trained three SINDy models, using a candidate library consisting of

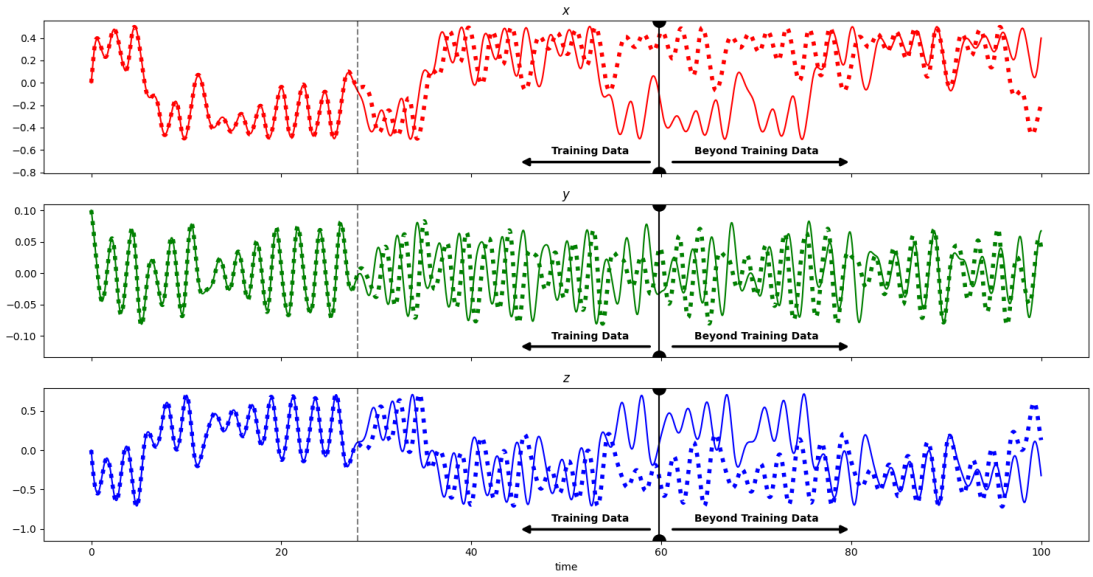


Figure 5.2.14: Experiment 3: Plot of the time series of trained UDE's trajectory, compared with the ground truth (trajectory of the true model). The continuous line of each row is the trajectory of the UDE, while the dotted line represents the true trajectory of the model. The dotted grey horizontal line (at $t = 28$) indicates the time stamp where the learned UDE starts to diverge from the true trajectory before the training data in a significant manner. The horizontal black line at $t = 60$ indicates the last time stamp of the training dataset point.

polynomials in $\mathbb{R}[x, y, z]$ up to the fifth degree excluding the bias term, which are the following:

- The baseline SINDy model, does not use UDEs in any way
- A UDE-enhanced SINDy model, trained on the entire training dataset and uses the UDE to approximate the derivative
- Another UDE-enhanced SINDy model, trained on the truncated training set to the time window $t \in [0, 28]$ but at the same time also expanded with more data points, with one data point for each constant step size $\Delta t = 0.01$

We will refer to the baseline model simply as the baseline model, and the two UDEs models with numbers (1) for the UDE-based model with the original traning dataset and (2) for the other UDE-based model with the modified training dataset.

To train the SINDy models, we used the STLSQ algorithms to solve the optimization problem, with different threshold values. The models with their recovered coefficients are illustrated in table 5.2.3. As we can see, all of the models managed to identify the significant

Chua Model (Ground truth)	$1.43x + 10y - 10x^3$	$1x - 1y + 1z$	$-16y$
Name	**Recovered $\dot{x}$**	**Recovered $\dot{y}$**	**Recovered $\dot{z}$**
Original SINDy	$1.428x + 9.575y + -14.769x^3 + -8.115x^2z + -3.190xz^2$	$0.9x - 0.911y + 0.903z$	$-14.709y$
UDE-enhanced SINDy (1)	$1.34x + 10.077y + -11.683x^3 + -3.846xz^2 + -1.619xz^2$	$x - y + z$	$-16y$
UDE-enhanced SINDy (2)	$1.302x + 9.497y + -9.084x^3 + 0.179xz^2$	$1.003x - 0.015y + 1.003z$	$-16.023y$

Table 5.2.3: Experiment 4: SINDy-recovered coefficients

terms of the coefficient correctly, but they also added extra terms which were not used in the original equations. This is especially verified for the baseline model, which introduced the polynomial terms x^2z, xz^2 with coefficients $-8.115, -3.190$. The model that minimizes this behavior is the UDE-SINDy model (2), with only the extra term x^2z being introduced with coefficient 0.179.

To obtain some clear quantitative metrics on the coefficients' accuracy, we calculated the mean squared error between the true coefficients and the learned coefficients of the models; the results are reported in table 5.2.4. Also, figure 5.2.15 shows a heatmap of the absolute differences between the coefficients of the original model and of the learned SINDy models. As we can see from both the table and the figure just mentioned, the UDE-SINDy model (2) minimizes the discrepancies the most.

SINDy Model	**MSE error in coefficients**
Baseline SINDy	6.1×10^{-1}
UDE (1)	1.227×10^{-1}
UDE(2)	5.410×10^{-3}

Table 5.2.4: MSE errors in coefficients

At the same time, it would be also appropriate to have a qualitative view on the learned trajectories with the SINDy models. Figures 5.2.16, 5.2.18 and 5.2.19 illustrate a comparison of the SINDy-learned trajectories with the true one. Also, in figure 5.2.17 we simulated the baseline SINDy model where we replaced the equations

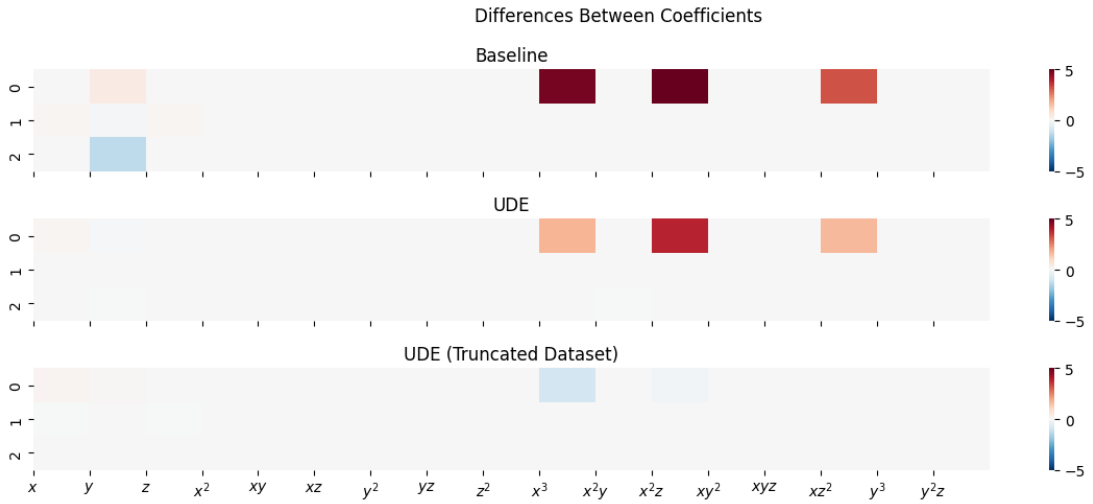


Figure 5.2.15: Experiment 3: Three heatmaps of absolute distances between the true coefficients and the learned coefficients, one for each SINDy model. Each square indicates the distance between the difference in the coefficients, with the "columns" being the term of the coefficient and the "rows" being the one of the specific equations (e.g. 0-th row indicates the equation for the $\dot{x}$ variable) N.B. The horizontal axis is truncated due to the large selected candidate library.

for $\dot{y}$, $\dot{z}$ with the ground truth. That is, the following equation:

$$\begin{aligned}\dot{x} &= 1.428x + 9.575y - 14.769x^3 - 8.115x^2z - 3.190xz^2 \\ \dot{y} &= x - y + z \\ \dot{z} &= -16y\end{aligned} \tag{5.6}$$

The idea behind this equation is to only take the estimate of the $\dot{x}$ equation, as it is the "hardest part" of the model to infer.

For starters, we can see that all of the SINDy models fail to generalize the model on relatively short times; the baseline model starts to diverge from the true model at around $t \approx 12.5$, while the model as described in equation 5.6 and the UDE-based SINDy models start to diverge at around $t \approx 5$. More evidently, the major shortcoming of the UDE-based models is the fact that the learned trajectories differ significantly from the ground truth with the shape of the attractor: rather than representing a Double-Scroll form, the UDE-enhanced models have the form of a Single-Scroll attractor. This is worse than the baseline model, which on the contrary still maintains the Double-Scroll form.

In order to fix this problematic aspect of the UDE-based SINDy

models, we optimized the recovered coefficients of the UDE-based model (2) by solving a least-squares problem on the residual defined as $r(\theta) := \sum_t \|\hat{\mathbf{x}}(t;\theta) - \mathbf{x}(t)\|^2$ where $\hat{\mathbf{x}}(t;\theta)$ is the simulated trajectory with parameters θ. We solved the problem with the Levenberg–Marquardt algorithm; we opted to use the Levenberg-Marquardt algorithm over simpler methods such as Gradient Descent or ADAM as they are either unstable or converge very slowly and require many iterations. Either way, this optimization process yielded the final model with a loss of 1.101×10^{-9}. In particular, the final model is obtained as the following:

$$\begin{aligned}
\dot{x} &= 1.429998x + 9.99957y - 10.0000974x^3 - 0.00183684xz^2 \\
\dot{y} &= 0.999927x - 0.99981y + 0.99994870z \\
\dot{z} &= -15.99916y
\end{aligned} \tag{5.7}$$

Which is almost perfect, as the coefficients relative to the true terms are almost identical and the coefficient relative to the only "extra" term is minimized to around 1.8×10^{-3}. By plotting the two trajectories over the time period $t \in [0, 60]$ (fig. 5.2.20), we can see that the model managed to recover the Double-Scroll attractor form, and they diverge at $t \approx 26$, which is nearly the double of the SINDy baseline model. However the divergence was expected, as we are handling a chaotic model and we have very slight discrepancies in the coefficients and the additional term.

In the end, to have a quantitative evaluation on the divergence we calculated the maximal Lyapunov exponents (MLEs) that characterise the errors between the two trajectories over time. Figure 5.2.21 represents a time series of the log of the norm errors between the trajectories, from which we can see that the values start saturating from approximately $t = 30$.

So we fit the linear regression $\hat{\delta}(t) = \beta_0 + \beta_1 t$ of each time series in $t \in [0.01, 30]$ with the fixed bias term $\beta_0 = \delta(0.01)$. Note that we excluded $t = 0$, as the two trajectories in that time instance are equal and thus their logarithmic difference is $\log 0$, which is not defined. The yielded MLE was 0.4838, which is close to the Lyapunov coefficient of the Chua's circuit $\lambda = 0.28587651$.

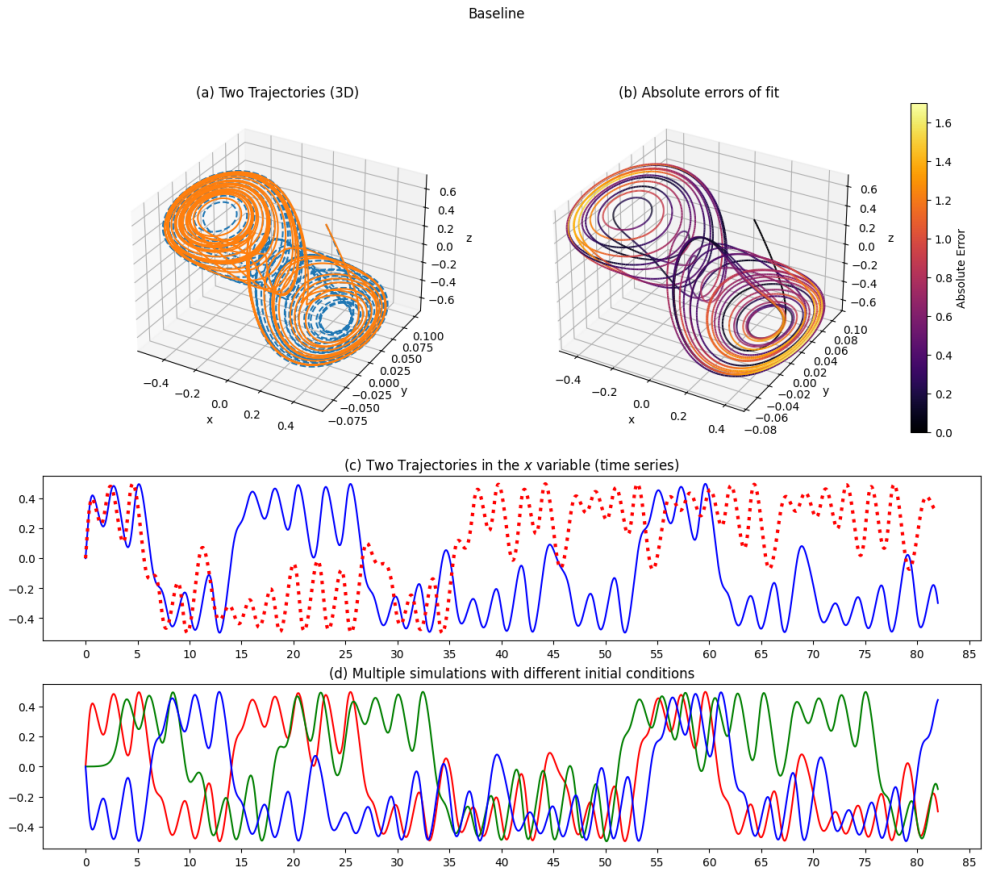


Figure 5.2.16: Experiment 3: Baseline SINDy, comparison with the true trajectory. (a) Learned vs true model trajectories, with the continuous orange line representing the learned model and the dashed blue line the true model. (b) Absolute fit errors, with the true trajectory colored by its distance from the learned trajectory. (c) Time series along the x variable, where the continuous blue line is the learned model and the dotted red line the ground truth. (d) Time series of the learned model for different initial conditions: red for $\mathbf{x}_0 = (0.0001, 0.1, 0.1)$, green for $\mathbf{x}_0 = (0.0001, 0, 0)$, and blue for $\mathbf{x}_0 = (0.0001, -0.1, 0)$.

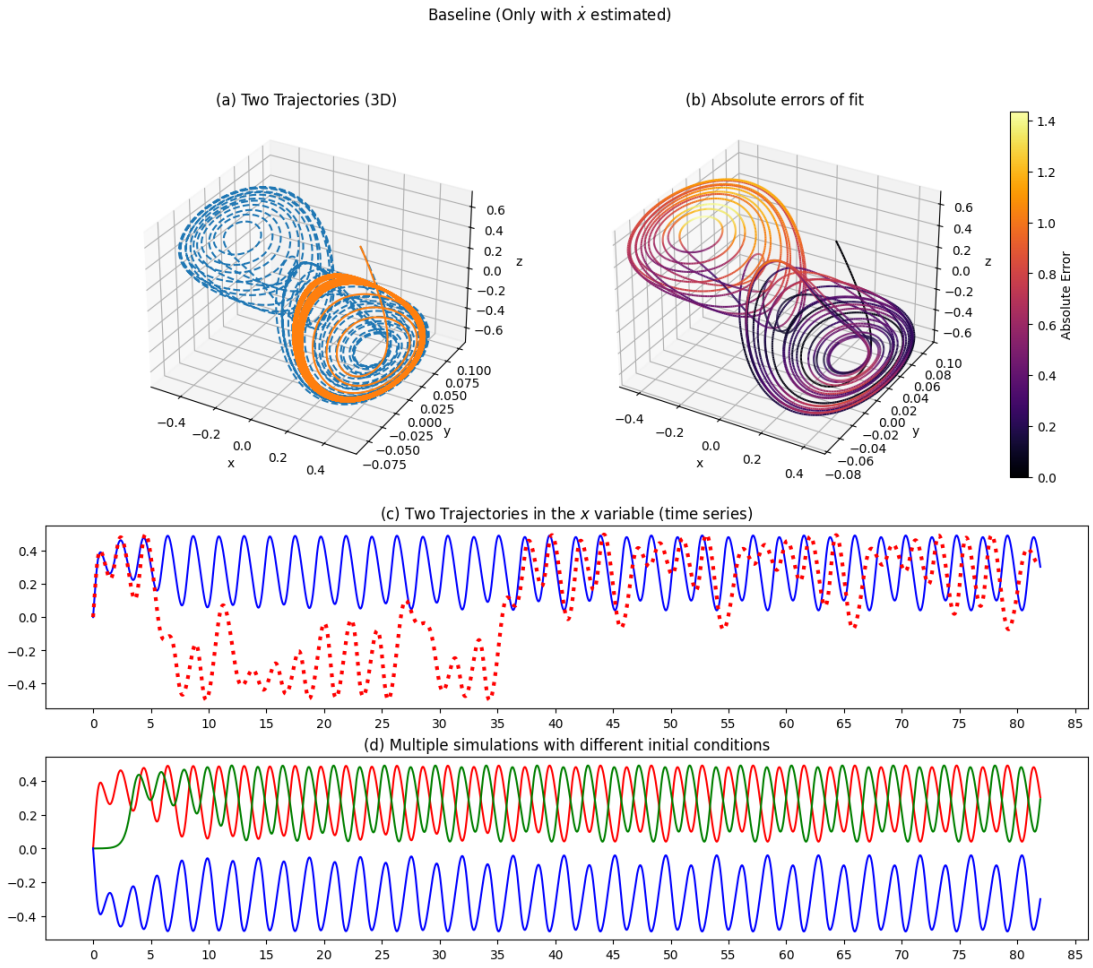


Figure 5.2.17: Experiment 3: Baseline SINDy only for $\dot{x}$, as in equation 5.6. For a description of this figure, refer to caption of figure 5.2.16

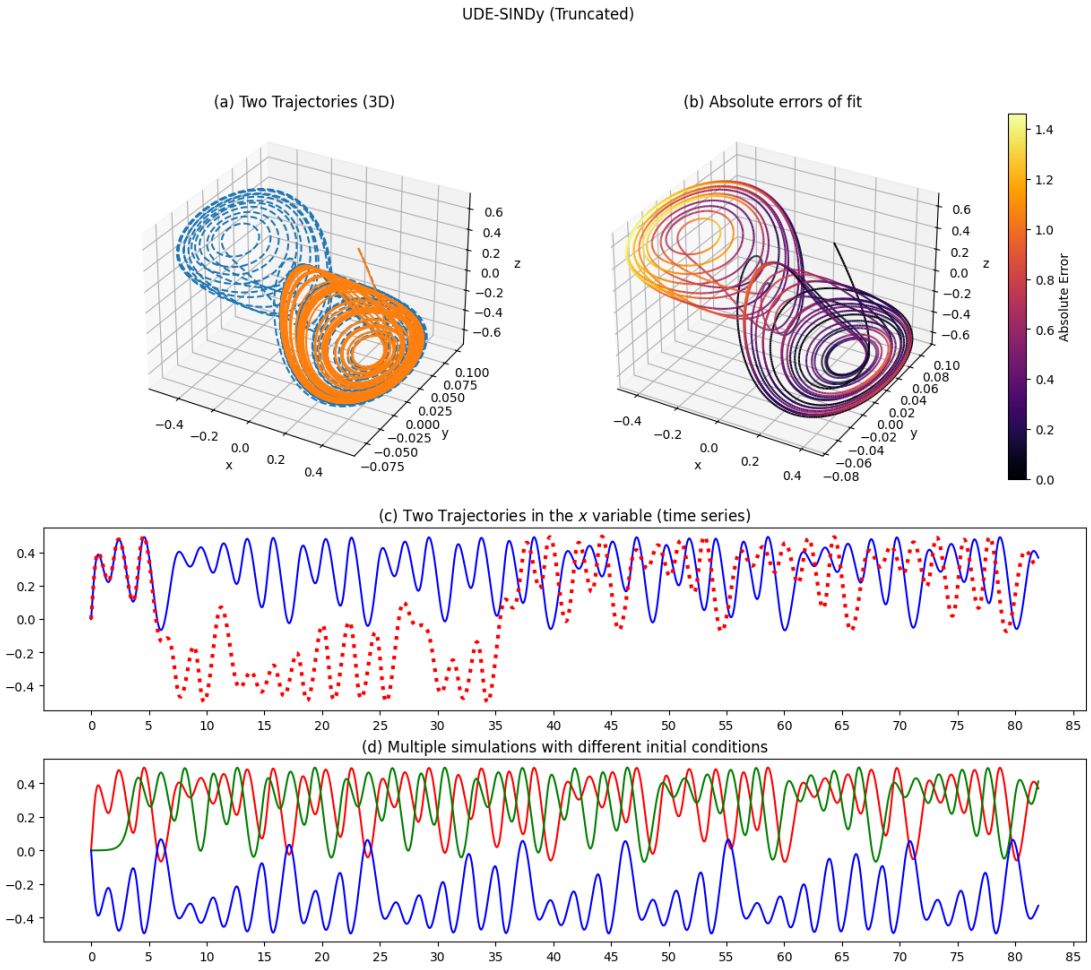


Figure 5.2.18: Experiment 3: UDE-based SINDy (1), comparison with true trajectory. For a description of this figure refer to the caption of figure 5.2.16.

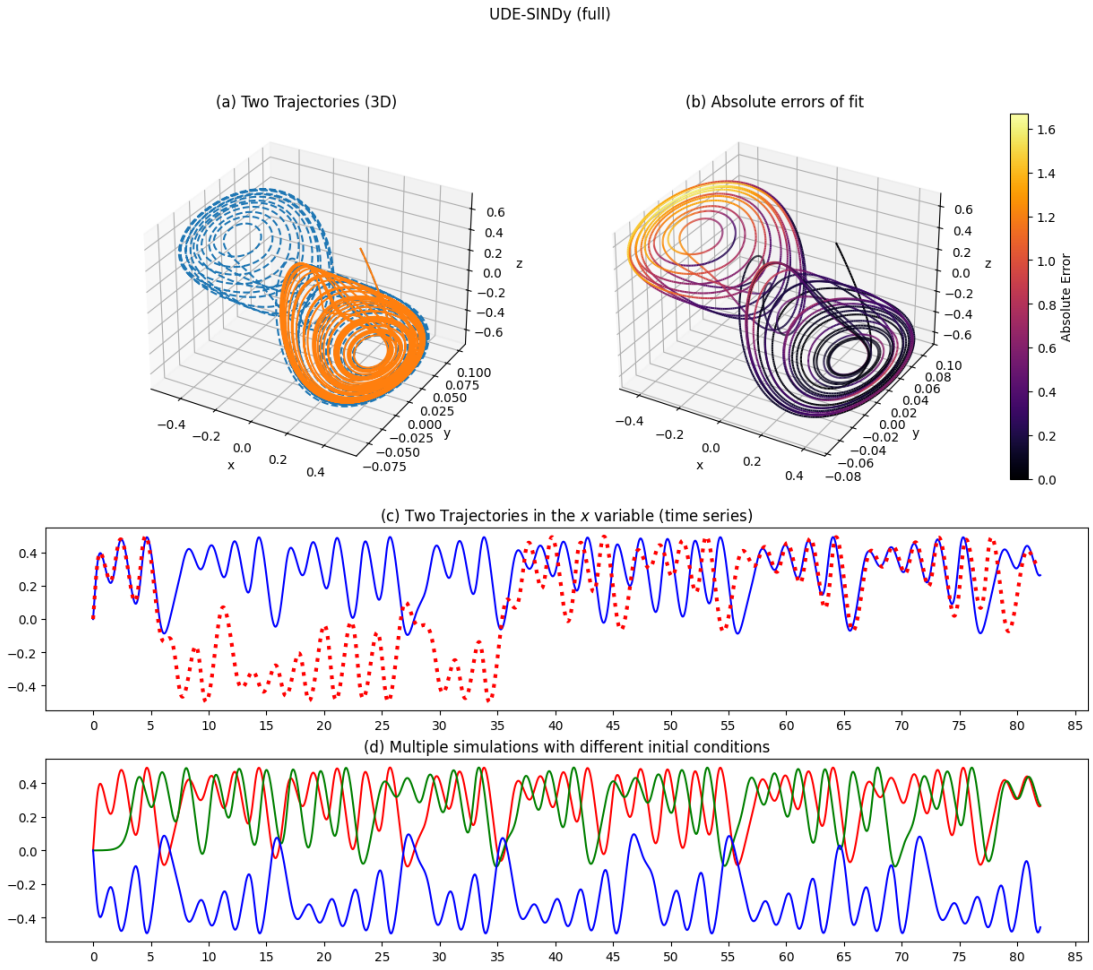


Figure 5.2.19: Experiment 3: UDE-based SINDy (2), comparison with true trajectory. For a description of this figure refer to the caption of figure 5.2.16.

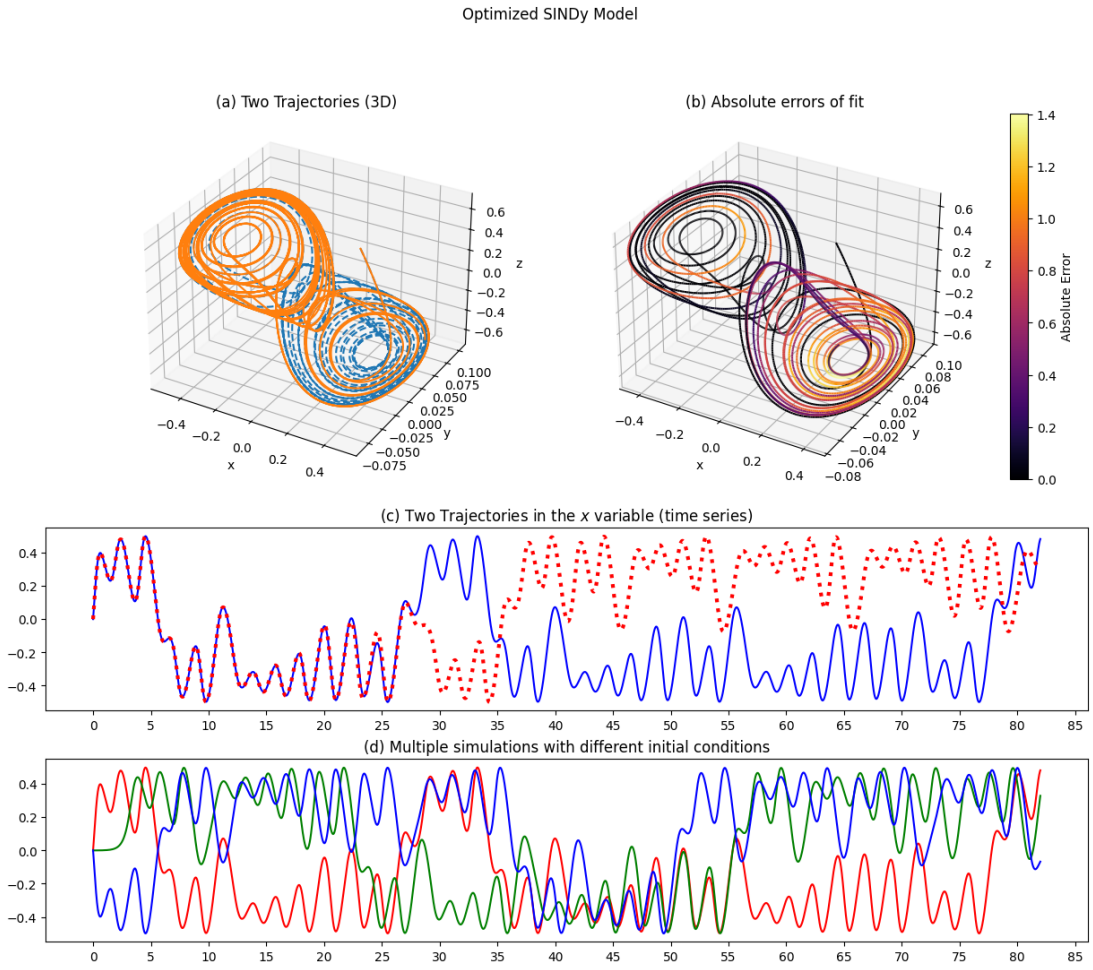


Figure 5.2.20: Experiment 3: Optimized UDE-based SINDy (2), comparison with true trajectory. For a description of this figure refer to the caption of figure 5.2.16.

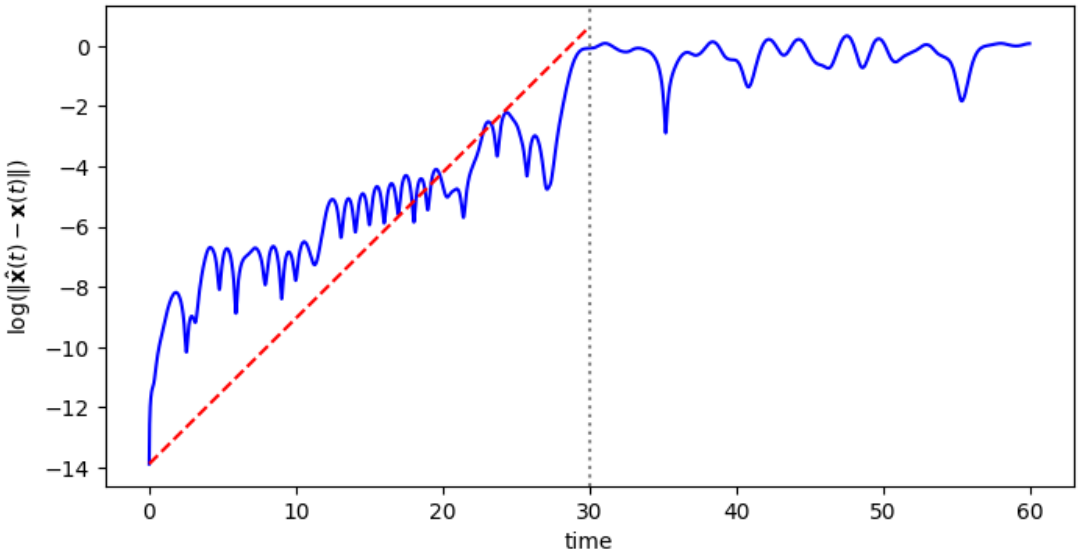


Figure 5.2.21: Experiment 3: Plot of the logarithmic norm difference between the simulated trajectory of the optimized SINDy model and the ground truth. Blue continuous line indicates the logarithmic norm difference, the red line represents the linear regression performed in $t \in [0, 30]$ and the vertical grey line indicates the maximum time instance for the linear regression.

5.2.5 Experiment 4: Training on multiple trajectories

INTRODUCTION One idea to potentially improve the results obtained in the previous experiment is to train our PEM-UDE model on multiple trajectories, as done in the last experiment of chapter 3.1.4. This might potentially improve the performances in generalizing the model as the chaotic model is highly sensitive to initial conditions, and thus by leveraging multiple trajectories from different initial conditions the model might be able to learn the dynamics better.

METHODOLOGY The dataset of this experiment is obtained by simulating three trajectories with the same parameters of the previous experiments, and using the initial conditions $\mathbf{x}_0 = (0.0001, 0.1, 0)$ and $\mathbf{x}_0 = (-0.3, 0, 0.3)$ for the training set and $(0.0001, 0.1, 0.1)$ for the validation set. The trajectories were simulated in the time window $t \in [0, 60]$ and then we extrapolated the training and validation sets from them with a constant step size $\Delta t = 0.25$.

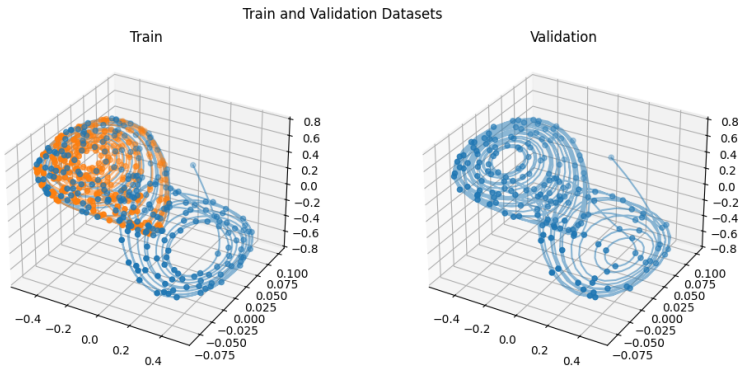


Figure 5.2.22: Experiment 4: Dataset. Parameters described in the text.

We trained two PEM-UDEs on the same dataset, one leverages both of the training trajectories and another leverages only the first one.

RESULTS AND DISCUSSION From figure 5.2.23 we see their loss curves during the training phase, from which we can see that the two PEM-UDEs have the same performance in traning losses, however the PEM-UDE trained with both trajectories is able to slightly generalize better with a final validation loss of 2.6883×10^{-3}, compared to the classical counterpart with 5.9460×10^{-3}. We then

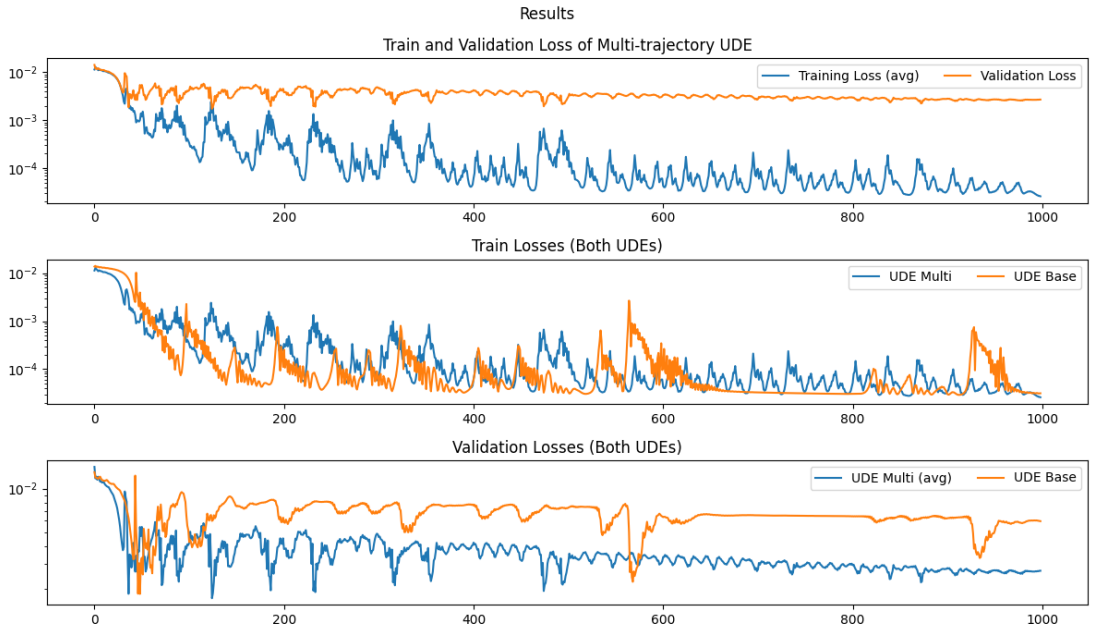


Figure 5.2.23: Experiment 4: Results and Comparison

repeated the same and exact procedures to train a SINDy model based on the trained UDE for extrapolating the dynamics of the model, obtaining the following final discovered equation:

$$\begin{aligned}\dot{x} &= 1.42938x + 10.00037y - 9.9975x^3 - 0.001107x^2z \\ \dot{y} &= 0.99980x - 0.999722y + 0.9987z \\ \dot{z} &= -16.00065y \end{aligned} \tag{5.8}$$

Unfortunately, from a quantitative point of view these results cannot definitively conclude that there is a clear improvement nor a worsening of the performances from the results of the previous experiment.

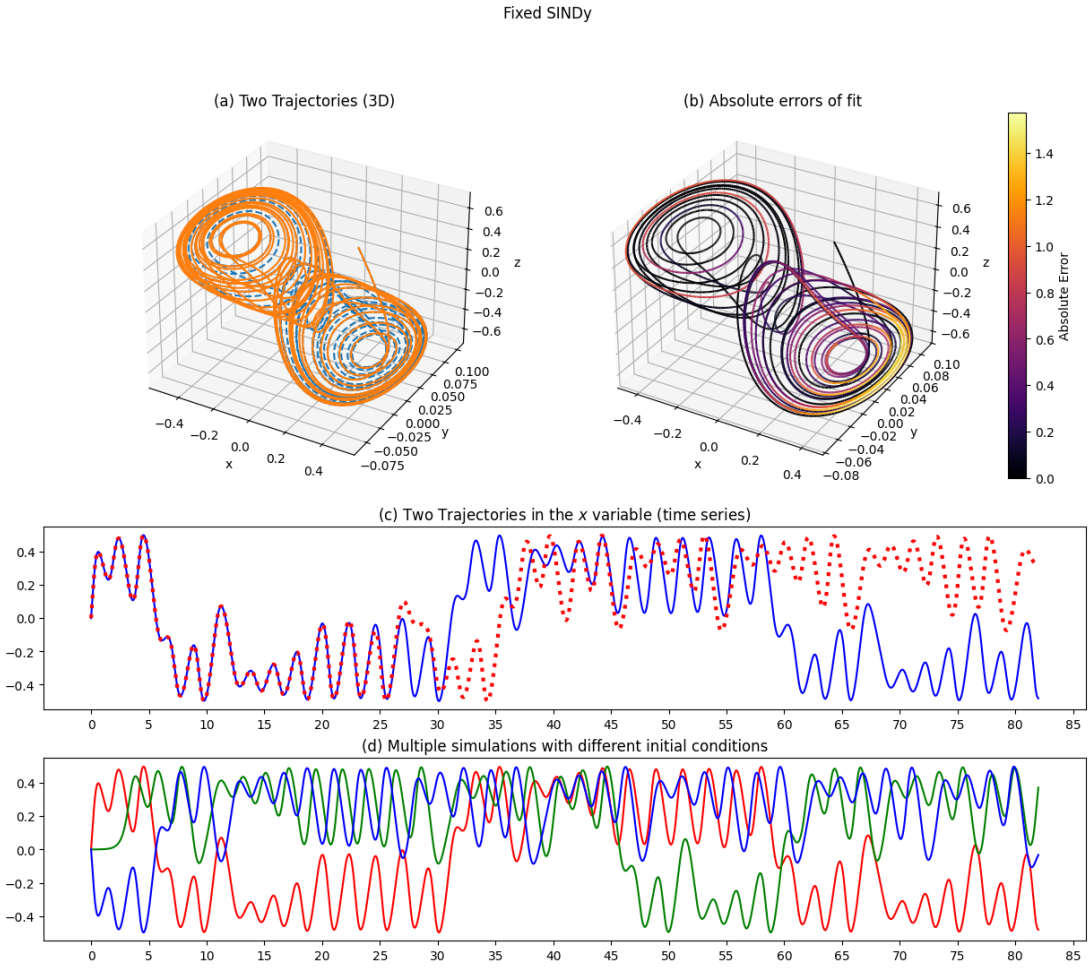


Figure 5.2.24: Experiment 4: Learned trajectory of the final UDE-enhanced SINDy model, compared with the ground truth. For a description of this figure refer to the caption of figure 5.2.16.

Chapter 6

Neural Challenging Classical Chaotic Models: Transient Chaos and Strange Attractors switching

In this chapter we illustrate some results worth of interest on the application of the UDE method to the Chua chaotic model.

6.1 Models with Transient Behavior

A fascinating feature of some chaotic dynamical systems is their transient behavior. That is, a model might temporarily exhibit a chaotic behavior until a certain time step is reached, after which it more or or less slowly transitions towards a non chaotic attractors, such as an equilibrium point or a limit cycle.

Establishing a-priori, by simple inspection of the differential equations, if a model has a chaotic transient or not is not possible, given the extreme dependence of the solutions of a chaotic model on both the parameters and the employed initial conditions.

In the first chapter of this section we will illustrate as the UDE approach in some important cases can be considered as a "Transient Chaos Neural generator".

In the second section we will show an even more surprisingly result: the generated transient chaos is such that during the transient interval the hybrid perfectly reach a Chua Strange attractor, but then it does not switch to an equilibrium point or to a limit cycles:

it switches to a different well-known strange attractors called the Rössler attractor.

The result of the first section is new in the literature on UDEs and other hybrid models, but the result of the second section is new also for the general literature on chaotic dynamics systems.

REMARK *This chapter and some sections of the previous chapter are being employed as the main kernel for a scientific manuscript to be submitted to an international peer reviewed journal.*

6.2 A UDE-based Model that generates Classical Transient Chaos

In the previous chapter, we have defined UDEs by assuming that the parameters of the underlying model are partially known, whereas – of course – we assumed that the function $g(x)$ as unknown. In particular, so far we have assumed the exact values of α, β, γ. However, in some cases we might not even have this sort of knowledge, rather we might only known the fact that there are some certain values of α, β, γ in the equations.

INTRODUCTION So, in the experiment of this section we will train a PEM-UDE that assumes the following parametric system of differential equations:

$$\begin{aligned} \dot{x} &= \theta_\alpha(y - U(x;\theta)) \\ \dot{y} &= x - y + z \\ \dot{z} &= -\theta_\beta y - \theta_\gamma z \end{aligned} \tag{6.1}$$

Where $U(x;\theta)$ is a learnable neural network and $\theta_{\alpha,\beta,\gamma}$ are learnable parameters. Moreover, this time we will experiment with a different configuration of the Chua's model, where we will set $\gamma > 0$. The objective is to see how the model learns the learnable parameters and measure its performance and generalization abilities.

METHODOLOGY For starters, we simulated a trajectory of the Chua model up to time $t_{\max} = 30$ using the following parameters: $\alpha =$

$20, \beta = 30, \gamma = 0.32, a = 0.03, c = -0.2$. This choice was inspired by [12], where a series of chaotic attractors were observed with this choice of parameters. Moreover, we used the initial condition $\mathbf{x}_0 = (0, 0.1, 0.3)$ for an analogous reason. Then we extracted the training dataset by sampling the trajectory with a constant step size $\Delta t = 0.5$, and we also created a validation dataset by using a different initial condition $\mathbf{x}_0 = (0.1, 0.2, 0.4)$, a different sampling step $\Delta t = 0.3$ and by extending the simulation time to $t_{\max} = 45$ (the initial time steps for the numerical simulations are fixed to 0.01 in both cases).

Then we defined the PEM-UDE model as defined in equation 5.3; the parameters $\theta_\alpha, \theta_\beta, \theta_\gamma$ are randomly initialized following the gaussian distribution $\mathcal{N}(0, 1)$.

The model was trained for 10000 epochs using the ADAM optimizer with a learning rate of 0.02.

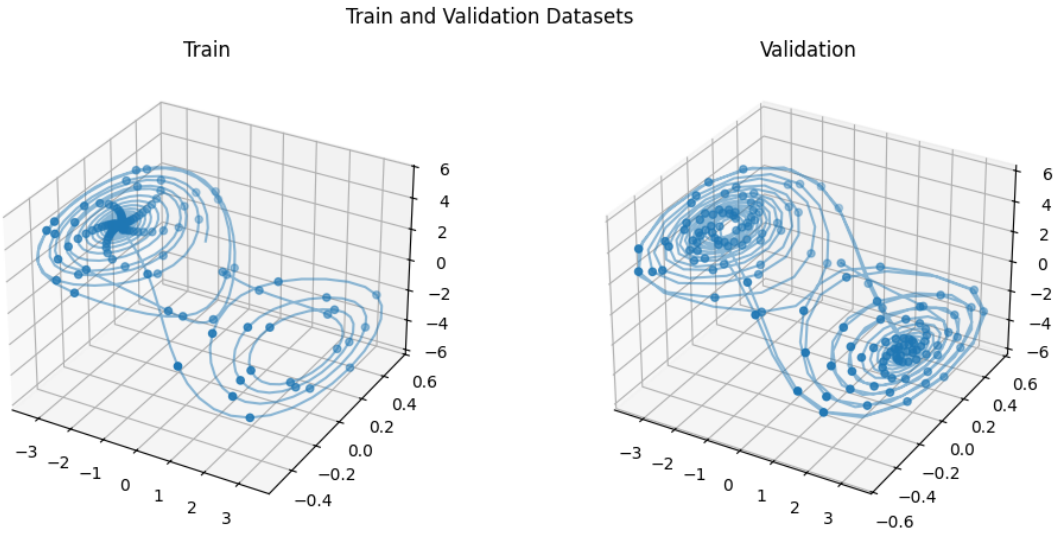


Figure 6.2.1: Experiment 1: Dataset. Parameters described in the text

RESULTS AND DISCUSSION Figure 6.2.1 shows the loss curves, the learned network and the evolution of the parameters $\theta_\alpha, \theta_\beta, \theta_\gamma$ over the training epochs.

From figure 6.2.2 we can see two difficulties of training parametrized UDEs: the first one being a clear mismatch in the learned function

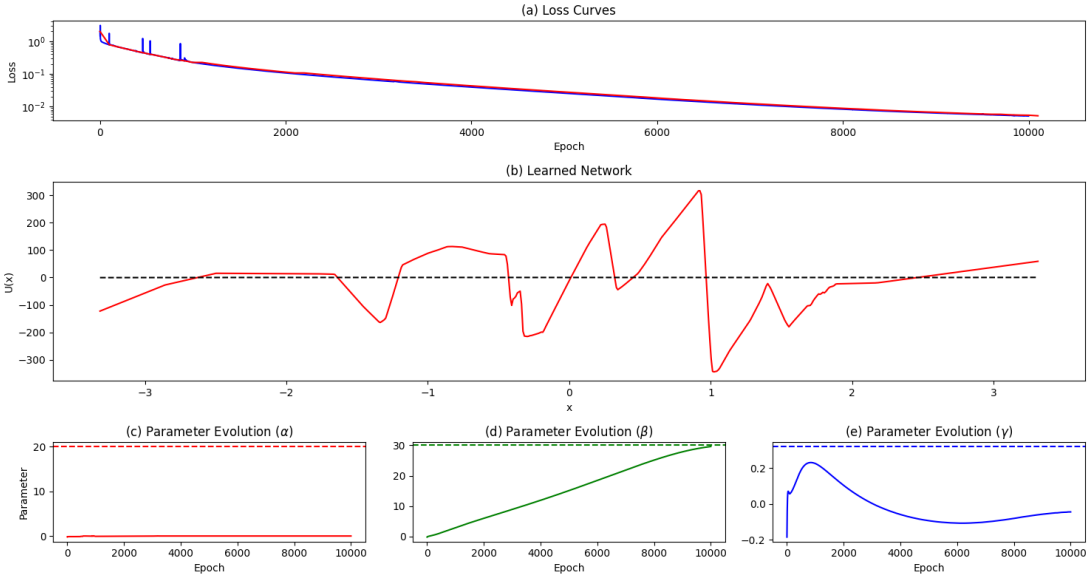


Figure 6.2.2: Experiment 3: Results (1). (a): Training and validation loss curves of the model. The red line represents the validation losses and the blue line represents the training losses. (b): Learned neural network. The red line represents the learned network of the PEM-UDE with known parameters and the dotted black line is the true cubic nonlinearity. (c), (d), (e): Evolution of the parameters $\theta_\alpha, \theta_\beta, \theta_\gamma$ over the training iterations. The continuous line represents the parameters, the dotted horizontal lines represent the true values for α, β, γ which are $\alpha = 20, \beta = 30, \gamma = 0.32$.

$U(x)$, which differs from the true form in the range $[-3, 3]$ in the order of hundreds, and the second one being the non-convergence of the parameters $\theta_\alpha, \theta_\gamma$; the only parameter which managed to converge its true value in ten thousand epochs is θ_β, although the converge was reached just for the last few epochs.

Then, we also simulated the model extending the time span to $t_{\max} = 60$ and we plotted its trajectory in figure 6.2.3, alongside with the true trajectory.
Here we can see an interesting phenomenon: until the maximum time of the training dataset (that is $t = 30$), the model's trajectory is able to replicate the true one accurately enough. However, from $t = 30$ on wards the model stabilizes towards a steady state instead of continuing to replicate a chaotic trajectory. This is a first instance of a PEM-UDE that creates a transient model, switching from a chaotic state to a steady state.

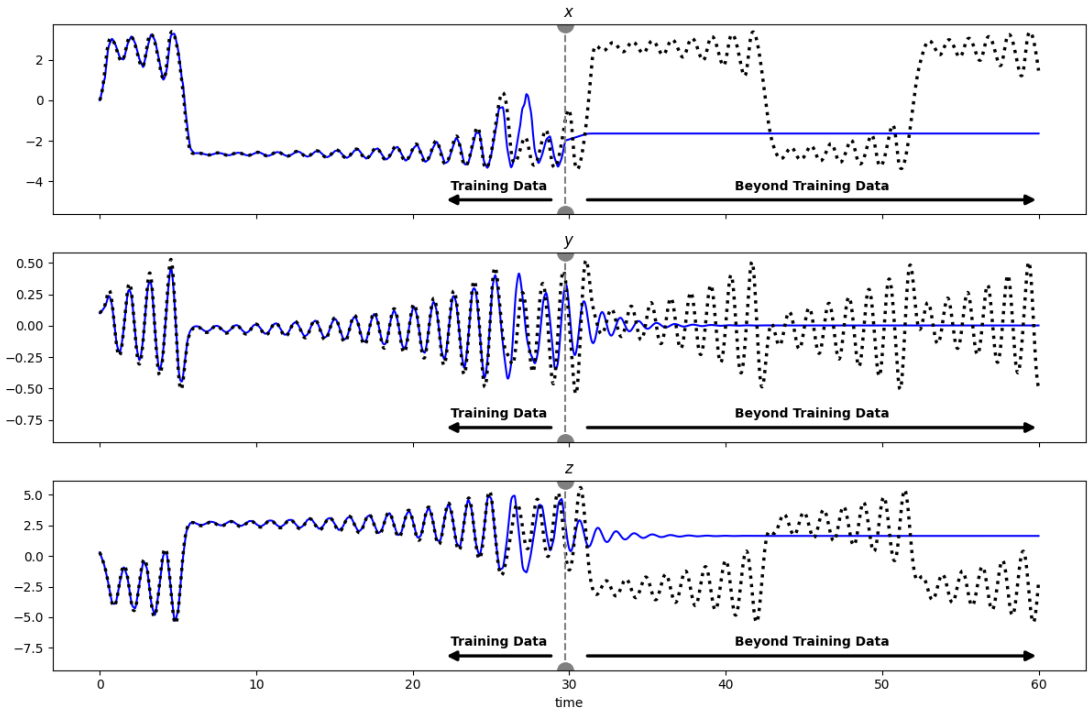


Figure 6.2.3: Experiment 3: Results (3). Time series of the predictions of the models, for each variable. The blue continuous line represents the predictions of the other PEM-UDE model, and the black dotted line represents the ground truth, i.e. the trajectory of the original circuit. The horizontal grey line represents the last time instance of the training dataset.

6.3 Transient Chua Chaos followed by Rössler Chaos

INTRODUCTION In all of the previous experiments, we have trained the UDEs on relatively short times, such as $t = 10, 30, 60$. However, it might be a better idea to train them on longer time series as the models could potentially explore the "features" of the Double Scroll attractor from which a better performance could be obtained. In this experiment, we will be training a PEM-UDE on a training trajectory with time span $t \in [0, 750]$ and be analyzing its subsequent results.

METHODOLOGY In this experiment, we will be training a PEM-UDE as defined in equation 5.3; that is, we will be using the neural network to approximate the cubic nonlinearity and the parameters α, β, γ are known. The dataset which we will be training our UDE on is sampled from a trajectory of the Chua's model with parameters $\alpha = 10, \beta = 16, \gamma = 0, a = 1, c = -0.143$ in the time range $t \in$

$[0, 750]$ with constant step size $\Delta t = 0.5$. Then, the model is trained for 1000 epochs with ADAM using a learning rate of 0.001.

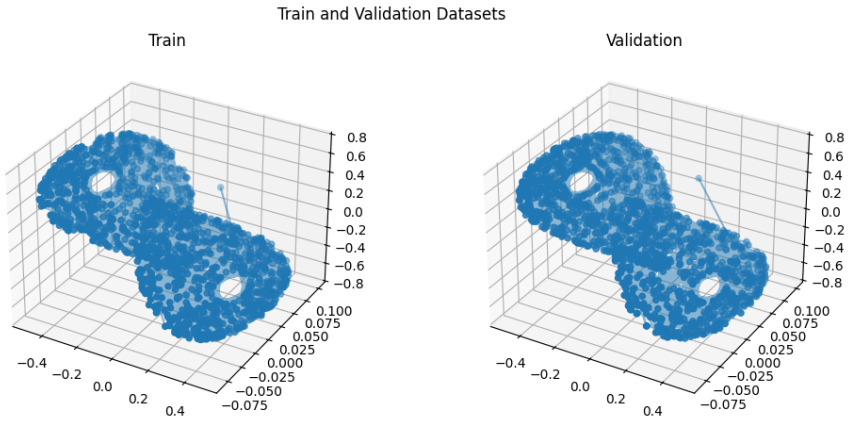


Figure 6.3.4: Experiment 2: Dataset. Parameters described in the text

RESULTS AND DISCUSSION Figure 6.3.5 reports the training results of the model; from what we can see, the model managed to reach a satisfactory training and validation loss of 3.5164×10^{-4} and 3.6123×10^{-4}. Moreover, the learned functional form $U(x)$ manages to resemble to the original cubic nonlinearity, especially closer to the values of $x = 0$.

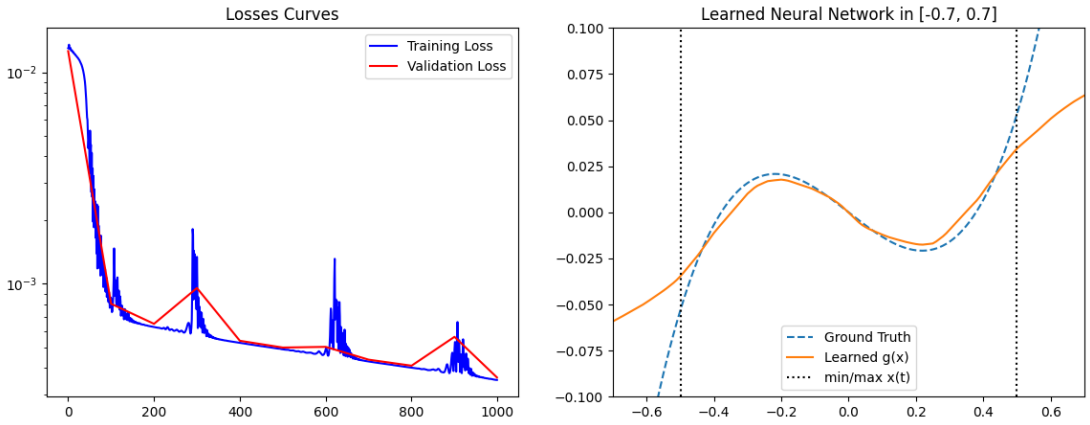


Figure 6.3.5: Experiment 2: Training results. Left panel: loss curves, blue represents training losses and red represents validation losses. Right panel: Visualization of the trained neural network $U(x)$ (continuous orange line), comparison with the ground truth (dotted blue line). The dotted vertical black lines indicates the minimum and maximum values in the x variable in the training set.

Figure 6.3.6 reports the time series of a simulation of the trained

UDE extended on the time horizon $t \in [0, 1500]$, alongside with some 3D plots on selected time intervals. Although we can qualitatively see that the trained trajectory starts to stray away from the actual trajectory very early (at around $t = 50$), we have to remember that the Chua's electric circuit is a chaotic model, meaning that a very small imprecision on the learned $U(x)$ term can lead to significant differences in trajectories. Moreover, part (b) of this figure highlights that the trained model on the "incorrect" part still behaves like a Double-Scroll oscillator.

Instead, we would like to focus on another very interesting aspect of the model that can be deduced from the figure: the model behaves behaves like the Double-Scroll attractor until the time of the last training point, from which the model changes its form into a Single-Scroll or a Rössler-like attractor.

This is a remarkable result, as with the tools of Machine Learning we have obtained some sort of transient chaotic model; rather than converging towards a steady state like in the previous experiment, this model is always chaotic but switches attractors with the last training set point.

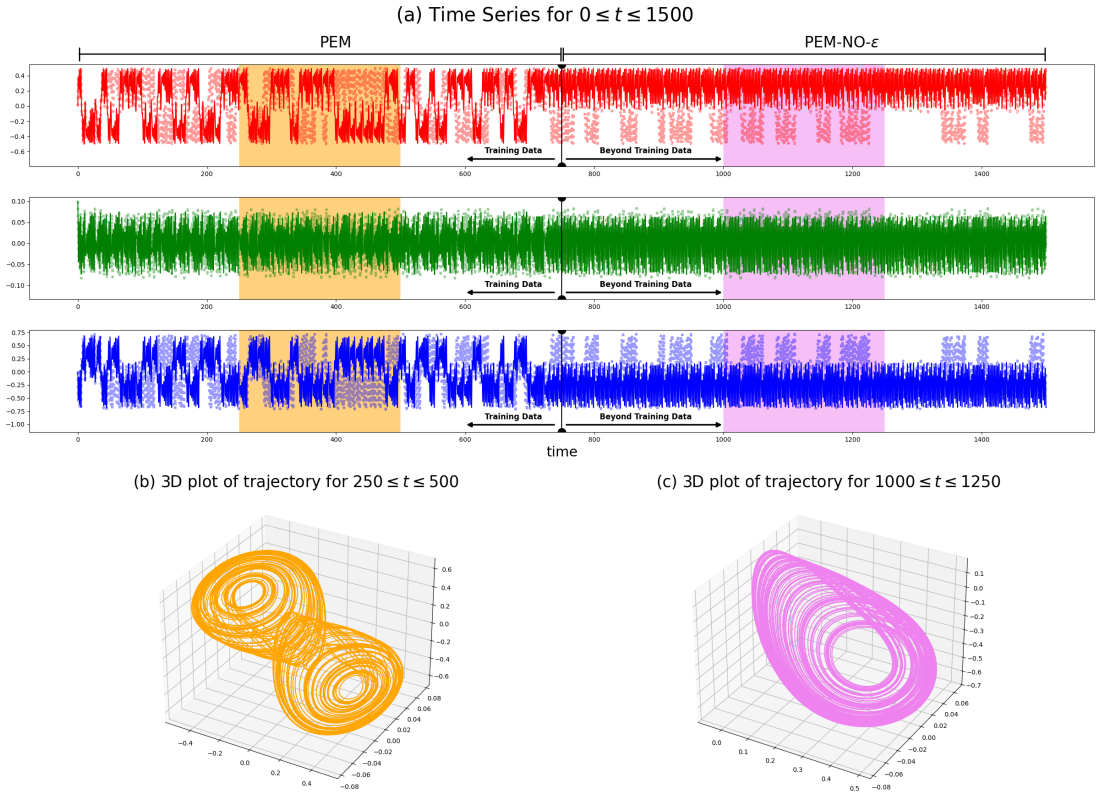


Figure 6.3.6: Experiment 2: Simulation of the trained model over the time horizon $t \in [0, 1500]$. (a): Time series for each variable, the continuous lines represent the learned trajectory and the transparent dotted lines represent the true trajectory. N.B. The text "PEM" refers to the part where the PEM-UDE made the predictions in the training range with the $K\varepsilon(t)$ term, and "PEM-NO-ε" refers to the predictions on the training range without the $K\varepsilon(t)$ term. The orange and violet areas highlight the time ranges $[250, 500]$ and $[1000, 1250]$. The vertical black line indicates the last training set point, which is $t = 750$ in our case. (b), (c): 3D plots of the trajectories in the indicated areas of part (a).

6.4 Effects of the Prediction-Error Method in the UDEs and their Learned Attractors

INTRODUCTION Recall that in all of the previous experiments, we have considered the PEM-UDE variant of the Universal Differential Equations; that is, other than the universal approximator, we are also adding an error term $\varepsilon(t)$ defined in equation 1.14 which in our case becomes as in equation 6.2

$$\begin{aligned} \dot{x} &= \alpha(y - U(x;\theta)) + K_x\varepsilon_x(t) \\ \dot{y} &= x - y + z + K_y\varepsilon_y(t) \\ \dot{z} &= -\beta y - \gamma z + K_z\varepsilon_z(t) \\ \varepsilon_x(t) = \hat{x}(t) - x; \varepsilon_y(t) = \hat{y}(t) - y; \varepsilon_z(t) &= \hat{z}(t) - z \end{aligned} \tag{6.2}$$

This idea had been presented by Chesebro et al. [51], and the main idea of adding the prediction-error term was to "ease" the training process of the UDEs by smoothing the loss landscape. Effectively, this result was verified in the first experiment of the previous section.

However, a natural question arises: what happens if given a trained PEM-UDE we remove its "PEM" part (i.e. the prediction-error term $\varepsilon(t)$) and simulate its trajectories? In this experiment, we will consider the model trained in the previous experiment (section 6.3), remove the prediction-error part and see how its trajectories change.

RESULTS AND DISCUSSION Figure 6.4.7 shows the plot of the trajectories as done analogously with figure 6.3.6. As we can see, by removing the prediction-error on the final model changes its behavior drastically: now it produces only the Rössler attractor as shown previously, even before the training set point.

This results shows us the importance of term that is proportional to the error $\varepsilon(t)$ in the PEM-UDE model: not only it is useful to ease the training process, but it also influences the outputs of the predictions significantly. In fact, even though we trained the PEM-UDE to replicate the Double-Scroll attractor, we still ended up with a final UDE that is able to only reproduce the Rössler attractor by itself.

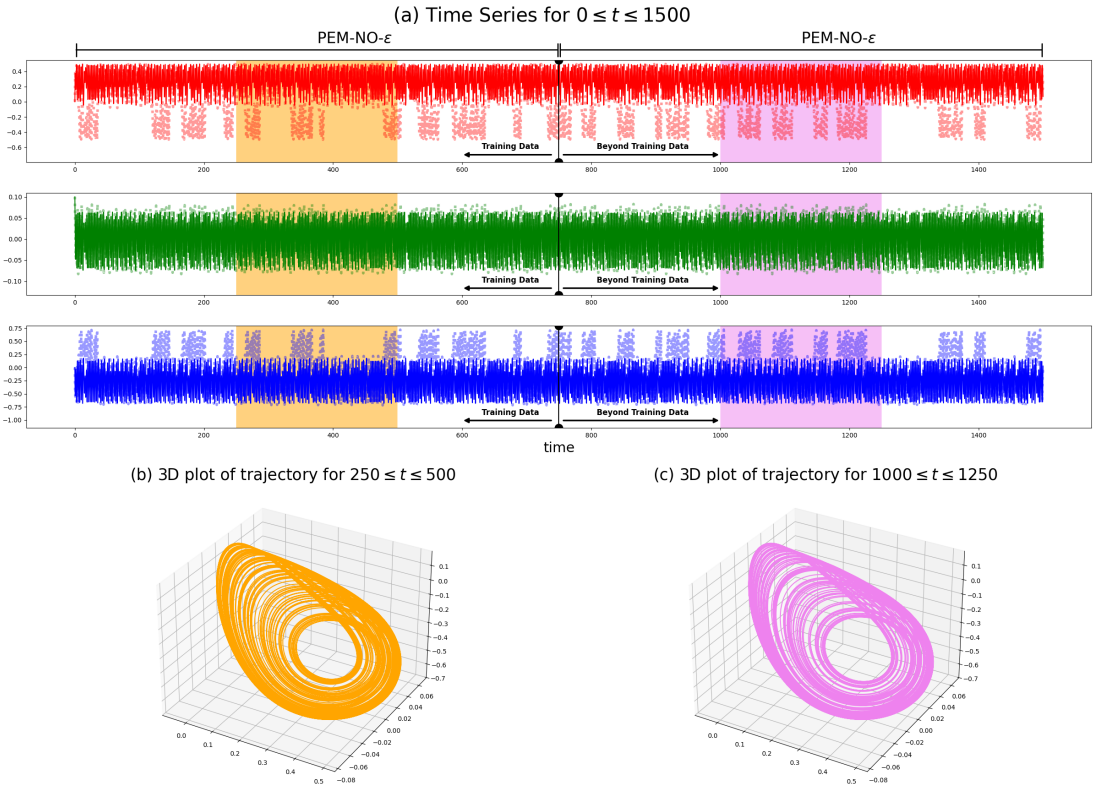


Figure 6.4.7: Experiment 3: Simulation of the trained model over the time horizon $t \in [0, 1500]$ with the prediction-error part removed. See figure 6.3.6 for a complete description of this figure.

Chapter 7

Implementation Details

In this chapter, we will discuss the tools used to implement the experiments done in this work.

7.1 UDE Modeling

To implement all of the experiments of this thesis, we used Python as the only programming language. Python is a general-purpose programming language with countless libraries for a diverse range of contexts.

To implement the UDE models, we have mainly used two frameworks: one is the PyTorch framework [28] and another one is the JAX framework [22].

We used PyTorch all of the experiments in chapters 2, 3 and 4. In particular, we used the `torchdiffeq` library [23] to use the differentiable solvers compatible with PyTorch. Defining a UDE in this framework is straightforward, as we could directly use the `nn.Module` class and use the `forward` method to output the derivative of the UDE. An example of a defined UDE class is given in listing 1.

However, the library used with PyTorch only provided some relatively simple solvers, which made it ineffective to solve complex models such as the chaotic Double Scroll attractor of the Chua's circuit model. Instead, for this part (chapters 5 and 6) we used the JAX library [22] with an ecosystem designed for efficient computing, with the main advantage being JIT compilation of Python

```python
class universal_budworm(nn.Module):
    def __init__(self, x0, r0, K):
        super().__init__()
        self.r0 = r0
        self.K = K
        self.x0 = x0

        self.net = nn.Sequential(
            nn.Linear(1, 32),
            nn.SiLU(),
            nn.Linear(32,32),
            nn.SiLU(),
            nn.Linear(32, 1)
        )

    def forward(self, x, t):
        # returns dx/dt
        return self.r0*x*(1-x/self.K) - self.net(x)
```

Listing 1: Example of a definition of UDE with PyTorch. The model refers to the bud-worm experiment done in chapter 2.

code. The JAX ecosystem for UDEs is composed of the following libraries:

- *Equinox* [41] to define the neural networks, we can consider it as a "JAX-equivalent" version of PyTorch, in particular the `torch.nn` module
- *Diffrax* [40] for the differentiable solvers, some sort of "JAX-equivalent" version of `torchdiffeq`
- *Optimistix* [48] and *Optax* [31] for optimization and minimization tools, we can think of this as a "JAX-equivalent" version of the `torch.optim` module of PyTorch

The syntax used to define the UDEs in the JAX ecosystem is almost the same as in PyTorch. Indeed, one of the "main" philosophies of JAX is to be as similar as possible to NumPy, enabling duck-typing. An example of a UDE model implemented with JAX is given in listing 2. The significant difference lies in the definition of the training loop, as the functions that return the loss quantities should be compiled natively (JIT), having the best computational efficiency. An example of a definition of training loop in PyTorch and JAX is given listings 3 and 4.

```python
class ChuaUdeX(eqx.Module):
    net: eqx.nn.MLP
    alpha: float
    beta: float
    gamma: float

    def __init__(self, alpha, beta, gamma, key):
        self.net = eqx.nn.MLP(
            in_size='scalar',
            out_size='scalar',
            width_size=128,
            depth=5,
            activation=jax.nn.relu,
            key=key,
        )

        self.alpha = alpha
        self.beta = beta
        self.gamma = gamma

    def physics(self, t, state):
        # known terms
        x, y, z = state
        return jnp.array(
            [self.alpha*y, x-y+z, -self.beta*y-self.gamma*z]
        )

    def residual(self, t, state):
        # universal approximator part
        x, _, _ = state
        return jnp.array(
            [-self.alpha*self.net(x), 0,0]
        )

    def __call__(self, t, state, args):
        return self.physics(t, state) + self.residual(t, state)
```

Listing 2: Example of a definition of UDE with JAX (and Equinox and Diffrax). The model refers to the Chua experiments done in chapter 5.

```python
# Train Normal UDE
key = jr.PRNGKey(0)
ude = ChuaUdeX(ALPHA, BETA, GAMMA, key)

lr = 0.01
n_epochs = 500
optimizer = opx.adam(lr)
opt_state = optimizer.init(eqx.filter(ude, eqx.is_inexact_array))

@eqx.filter_value_and_grad
def loss_fn(model, x0, y_true, ts):
    # calculate loss
    pred = solve_ude_light(model, x0, ts)
    return jnp.mean((pred - y_true) ** 2)

@eqx.filter_jit
def train_step(model, opt_state, x0, x_true, ts, optimizer):
    # one train step on the dataset
    loss, grads = loss_fn(model, x0, x_true, ts)
    updates, opt_state = optimizer.update(grads, opt_state, model)
    model = eqx.apply_updates(model, updates)
    return model, opt_state, loss

for i in range(0, n_epochs):
    ude, opt_state, loss = train_step(ude, opt_state,
        x_train[0],
        x_train, t_train,
        optimizer)
```

Listing 3: Example of a training loop of a UDE in JAX.

```
neuron_UDE = my_universal_neuron(
    threshold, torch.tensor([0.3]), torch.tensor([1.])
)
lr = 0.01
optimizer = optim.Adam(neuron_UDE.parameters(), lr=lr)
n_epochs = 2000

for EPOCH in (range(n_epochs)):
    optimizer.zero_grad()
    neuron_UDE.zero_grad()

    # forward solution
    x_pred = odeint(neuron_UDE,
                    x0_torch,
                    t_train_torch,
                    method="rk4",
                    )

    loss = torch.mean((x_pred-x_train_torch)**2)

    loss.backward()
    optimizer.step()

```

Listing 4: Example of a training loop of a UDE in PyTorch.

7.2 Symbolic Regression

For the SINDy symbolic regression, we used the `pysindy` library [35]. An example of a model recovery process is illustrated in listing 5. Also, we used either Optimistix or SciPy to optimize the recovered models. An example of this with Optimistix is given in listing 6.

7.3 Hyperparameter Optimization

Optuna [26] is a hyperparameter optimization framework in Python, which can be used with any other ML library such as Scikit-Learn, PyTorch or Equinox.

In Optuna, we define a trial as a realization of a model's training process from which we obtain a value of the objective function(s). The trials can be subsequently used to infer the best hyper parameters to solve the problem: the simplest way is to look at the chosen hyper parameters of the trial with the best score.

Additionally, Optuna also provides a dashboard functionality where advanced insights on the hyper parameters can be obtained. For example, one can look at the hyperparameter importance histogram, or the hyperparameter relationship plot, as well as rank plots.

Figure 7.3.1: Screenshot of the Optuna dashboard.

Figure 7.3.2: Another screenshot of the Optuna dashboard.

7.4 Miscellaneous

In this project, we used many other framework for various purposes. We used plotting frameworks Matplotlib [11] and Seaborn [42] to generate all of the plots present in this thesis. We also used SciPy [38] to solve ordinary differential equations, in order to obtain most of the datasets used in the experiments (in some other cases we used Diffrax with its own dedicated ODE solver).
Of course, we also used Numpy [32] to manipulate numerical vectors, as well as Pandas [37] to handle dataframes. The well-known Scikit-Learn library [14] was used to train linear regressions for chapter 5.

The code used to carry out the various experiments of this thesis is available in the following GitHub repository:

https://github.com/OdinMeng/Hybridizing-DeepLearning-and-ClassicalMathematicalModeling

```python
# Use the UDEs to get a new and bigger dataset
pred_new_np = odeint(my_lv,
    torch.tensor(x0, dtype=torch.float32),
    teval:=torch.arange(0, 3, 0.001)).detach().numpy()

T_train_ps = teval.detach().numpy()
X_train_ps = pred_new_np[:, 0]
Y_train_ps = pred_new_np[:, 1]

# Define the UDE model
model = ps.SINDy(
    feature_library=ps.PolynomialLibrary(10,
        include_interaction=True,
        include_bias=False),
    optimizer=ps.STLSQ(0.3)
)

# Calculate the derivatives of each point in the dataset
X_dot = np.zeros((T_train_ps.shape[0], 2))

for i,(t,x, y) in enumerate(zip(T_train_ps, X_train_ps,
    Y_train_ps)):

    X_dot[i, :] = my_lv(
        torch.tensor([t], dtype=torch.float32),
        torch.tensor([x, y], dtype=torch.float32)
    ).detach().numpy()

# Fit the SINDy model
model.fit(
    np.stack((X_train_ps, Y_train_ps), axis=-1),
    t = T_train_ps, x_dot=X_dot, feature_names=['x','y']
)

# Print the model
model.print()
```

Listing 5: Example of a SINDy model training process. Note that we are using UDEs to enhance the recovered model.

```python
def identified_chua(t, xyz, args):
    # Define the recovered equation
    a1, a2, a3, a4, b1, b2, b3, c1 = args
    x, y, z = xyz
    return jnp.array(
        [
            a1 * x + a2 * y + a3 * (x**3) + a4 * (x**2) * z,
            b1 * x + b2 * y + b3 * z,
            c1 * y,
        ]
    )

@eqx.filter_jit
def residual(args, _):
    # Define the residual of the problem
    term = dfx.ODETerm(identified_chua)
    solver = dfx.Tsit5()
    y0 = jnp.array((0.0001, 0.1, 0))

    # solve equation
    sol = dfx.diffeqsolve(
        term, solver, y0=y0,
        t0=t_train_reduced[0], t1=t_train_reduced[-1],
        saveat=dfx.SaveAt(ts=t_train_reduced), args=args,
        dt0=(t_train_reduced[1]-t_train_reduced[0]),
        adjoint=dfx.DirectAdjoint(),
    ).ys

    return sol - x_train_reduced

initial_guess = jnp.array([
    1.303, 9.965, -9.126, 0.158,
    1.003, -1.015, 1.003,
    -16.023]) # From a SINDy-recovered model

residual_solver = optx.LevenbergMarquardt(rtol=1e-8, atol=1e-8)
fixed_params = optx.least_squares(residual, residual_solver,
    initial_guess, max_steps=10000)
```

Listing 6: Example of an ODE optimization process, with Optimistix.

Chapter 8

Concluding Remarks

In this conclusive chapter, we will summarize all of the work done to outline the strengths and limitations of the *Universal Differential Equations*. Moreover, we will also discuss the scope of the UDEs, in particular looking at potential generalizations and other possible applications.

STRENGTHS First of all, this thesis has mainly shown that UDEs (and their variants) are able to work quite well on all datasets, in terms of "data fitting". In all of the case studies, we saw that the trained UDEs were able not only to fit the data points of the training set, but also interpolate the space between the points in an accurate manner. For example, figures 4.1.8 and 4.1.9 show that the UDEs are able to interpolate between the dynamics between the training points in a very accurate manner.

Moreover, as claimed by C. Rackauckas et al. [34], UDEs can be leveraged to get improved forms of symbolic regression. In particular, tables 5.2.3 and 5.2.4 show that the UDE-enhanched SINDy models are able to get a more accurate estimate of the coefficients. Coupled with ODE optimization techniques, we are able to get final models that are extremely close to the true model as seen in equation 5.7 (cf. tab. 5.2.3, first row). Also, as shown in chapter 3.2, we can see that UDE-enhanced SINDy models are quite robust to small amounts of noise.

If we were interested in only approximating the form of the unknown term (which is replaced by the neural network), we can also

see that UDEs work reasonably well in most cases. For example, figures 4.2.12 and 4.2.21 show that the UDEs managed to learn the form of the true term; however, be noted that the approximation is only accurate enough in the values range of the unknown term. Of course, this is a data availability issue rather than an intrinsic problem with UDEs.

Finally, we have also shown that the PEM-UDE variants are able to handle chaotic systems, even going as far as extrapolating their dynamics with SINDy in an accurate enough manner. Moreover, in some cases the PEM-UDEs are shown to exhibit transient behaviors, where the trained model passes from a transitory chaotic phase on the training time span towards a steady state or a different chaotic attractor (figs. 6.2.3, 6.3.6).

LIMITATIONS As good as the UDEs might be, unfortunately there have been some significant limitations or unexplained behaviors encountered during the development of this work.

First of all, a significant issue is that, in some cases, UDEs are not able to learn the functional form of the unknown term well enough but are still able to fit and interpolate the training set very well. In these cases, we suspected a non-identifiability issue as shown in experiment 1 of chapter 4.1 (figs. 4.1.2, 4.1.3). Similarly, as seen in experiment 3 of chapter 4.1 if we task a UDE with approximating multiple unknown terms instead of only one, UDEs fail at learning the functions but they succeed in fitting and interpolating the dataset. Also, a similar problem arises in learning the parameters of a network model, as shown in experiments 1, 2 and 3 of chapter 4.2.2. This is an issue that deserves further investigation, especially whether this is an problem with UDEs that can be somehow resolved, or an intrinsic problem with learning the dataset.

Moreover, another aspect that deserves further investigation is the strange dependency on the dataset of UDEs; figure 3.1.7 shows that the UDEs performances do not necessarily improve with a bigger dataset size, without even showing a clear pattern.

Another problematic aspect of UDEs are their sensitivity to noise;

figures 3.2.13, 3.2.14 show that the final training losses of UDEs degrade exponentially (if not stronger) with the presence of noise. To be noted that despite this weakness, they are still able to enhance the SINDy-recovered equations well enough (or in some cases, even slightly improving the results).

We have also found out that UDEs requires a correct modeling of the UDE as an assumption; as shown in experiment 2 of chapter 5, if we model the UDE incorrectly their relative performance can potentially degrade. Figure 5.2.10 shows that incorrect assumptions (blue line) has significantly degrades the performances (red line represents the correct assumptions).

Lastly, another important issue of UDEs is the instability in their training iterations. As we can see in figures 4.1.3, 4.1.9, 4.2.15, 4.2.17, 4.2.21, 5.2.13, 6.3.5 many training instances of UDEs show instability in training with sudden peaks in the training curve. This is a phenomenon that is verified also with a small amount of learning rate, such as 3×10^{-3} or 1×10^{-3}. This kind of behavior deserves further investigation, to understand the source of this instability.

POTENTIALITIES AND FURTHER GENERALIZATIONS Despite the potential issues that we reported, in this thesis we still think that UDEs are a great method for studying dataset with an underlying physical structure. Moreover, this method holds a lot of potentialities in terms of possible generalizations and research area.

The first possible direction of generalization is changing the universal approximators used in the UDEs: as mentioned in [34], one could choose to use other universal approximators, such as Chebyshev polynomials or random forests. Alternatively, we could also use other NN architectures, as in this thesis we have only used Multi-Layer Perceptrons (MLPs). For example, one could possibly opt for alternative architectures such as Convolutional Neural Networks, Recurrent Neural Networks or Attention Networks [33]. Another potential path is to employ specific methods from literature on Machine Learning applied to chaotic systems, such as the Feedforward-Recursive Neural Networks and Long Short-Term Memory Networks

(LSTMs) [46] and Reservoir Computing (RC) [21].

Another possible way to generalize UDEs is the type of model we are handling: that is, up until now we have only considered Ordinary Differential Equations, but potentially UDEs are able to handle other types of equations. For example, [34] mentions the possibility of applying a similar method to Stochastic Differential Equations (SDEs), Delay Differential Equations (DDEs) or Integro-Differential Equations (IDEs). In fact, for chapter 4.2.2 we could have potentially considered using the "continuous" version of the biological neurons network model, as described in equation 1.27.

Lastly, we can use UDEs for other kinds of applications. In our case, we used UDEs with the end of extrapolating the dynamics underlying a dataset. However, there can be potential other use cases of UDEs: for example, Wenjing Zhang et al. [52] applied UDEs to solve optimal control problems, in particular solving a problem on cancer therapy.

CONCLUSION With everything said, we conclude saying UDEs represent a good way of intersecting Machine Learning and scientific domain knowledge. Also, there are a lot of possible generalizations of this hybrid model, making it very flexible and adaptable to various kind of contexts. Thus, UDEs offers a vast area of possible research, both in its theoretical aspects as there are still a lot of unexplained behaviours, and in new applications as there are still other scientific fields where we can potentially apply UDEs to novel problems.

Bibliography

[1] F. Rosenblatt. "The perceptron: A probabilistic model for information storage and organization in the brain." In: *Psychological Review* 65.6 (1958), pp. 386–408. ISSN: 0033-295X. DOI: 10.1037/h0042519. URL: http://dx.doi.org/10.1037/h0042519.

[2] D. Ludwig, D. D. Jones, and C. S. Holling. "Qualitative Analysis of Insect Outbreak Systems: The Spruce Budworm and Forest". In: *The Journal of Animal Ecology* 47.1 (Feb. 1978), p. 315. ISSN: 0021-8790. DOI: 10.2307/3939. URL: http://dx.doi.org/10.2307/3939.

[3] G. Cybenko. "Approximation by superpositions of a sigmoidal function". In: *Mathematics of Control, Signals and Systems* 2.4 (Dec. 1989), pp. 303–314. ISSN: 1435-568X. DOI: 10.1007/BF02551274. URL: https://doi.org/10.1007/BF02551274.

[4] L.O. Chua. "A zoo of strange attractors from the canonical Chua's circuits". In: *[1992] Proceedings of the 35th Midwest Symposium on Circuits and Systems*. 1992, 916–926 vol.2. DOI: 10.1109/MWSCAS.1992.271147.

[5] Guo-Qun Zhong. "Implementation of Chua's circuit with a cubic nonlinearity". In: *IEEE Transactions on Circuits and Systems I: Fundamental Theory and Applications* 41.12 (Dec. 1994), pp. 934–941. ISSN: 10577122. DOI: 10.1109/81.340866. URL: http://ieeexplore.ieee.org/document/340866/ (visited on 03/30/2026).

[6] ANSHAN HUANG et al. "CHUA'S EQUATION WITH CUBIC NONLINEARITY". In: *International Journal of Bifurcation and Chaos* 06.12a (1996), pp. 2175–2222. DOI: 10.1142/S0218127496001454. eprint: https://doi.org/10.1142/S0218127496001454. URL: https://doi.org/10.1142/S0218127496001454.

[7] Robert Tibshirani. "Regression Shrinkage and Selection Via the Lasso". In: *Journal of the Royal Statistical Society Series B: Statistical Methodology* 58.1 (Jan. 1996), pp. 267–288. ISSN: 1467-9868. DOI: 10.1111/j.2517-6161.1996.tb02080.x. URL: http://dx.doi.org/10.1111/j.2517-6161.1996.tb02080.x.

[8] Kathleen T Alligood et al. *Chaos: an introduction to dynamical systems*. 1997.

[9] James D Murray. *Mathematical Biology*. en. Ed. by J D Murray. 3rd ed. Interdisciplinary Applied Mathematics. New York, NY: Springer, Jan. 2002.

[10] Keith O'Donoghue et al. "A FAST AND SIMPLE IMPLEMENTATION OF CHUA'S OSCILLATOR WITH CUBIC-LIKE NONLINEARITY". In: *International Journal of Bifurcation and Chaos* 15.9 (Sept. 2005), pp. 2959–2971. ISSN: 0218-1274, 1793-6551. DOI: 10.1142/S0218127405013800. URL: https://www.worldscientific.com/doi/abs/10.1142/S0218127405013800.

[11] J. D. Hunter. "Matplotlib: A 2D graphics environment". In: *Computing in Science & Engineering* 9.3 (2007), pp. 90–95. DOI: 10.1109/MCSE.2007.55.

[12] Holokx A. Albuquerque and Paulo C. Rech. "A PARAMETER-SPACE OF A CHUA SYSTEM WITH A SMOOTH NONLINEARITY". In: *International Journal of Bifurcation and Chaos* 19.4 (Apr. 2009), pp. 1351–1355. ISSN: 0218-1274, 1793-6551. DOI: 10.1142/S0218127409023676. URL: https://www.worldscientific.com/doi/abs/10.1142/S0218127409023676.

[13] Gonzalo M. Ramírez-Ávila and Jason A.C. Gallas. "How similar is the performance of the cubic and the piecewise-linear circuits of Chua?" In: *Physics Letters A* 375.2 (Dec. 2010), pp. 143–148. ISSN: 03759601. DOI: 10.1016/j.physleta.2010.10.046. URL: https://linkinghub.elsevier.com/retrieve/pii/S0375960110014064.

[14] F. Pedregosa et al. "Scikit-learn: Machine Learning in Python". In: *Journal of Machine Learning Research* 12 (2011), pp. 2825–2830.

[15] Ch. Tsitouras. "Runge–Kutta pairs of order 5(4) satisfying only the first column simplifying assumption". In: *Computers Mathematics with Applications* 62.2 (2011), pp. 770–775. ISSN: 0898-1221. DOI: https://doi.org/10.1016/j.camwa.2011.06.002. URL: https://www.sciencedirect.com/science/article/pii/S0898122111004706.

[16] *Neural Fields: Theory and Applications*. Springer Berlin Heidelberg, 2014. ISBN: 9783642545931. DOI: 10.1007/978-3-642-54593-1. URL: http://dx.doi.org/10.1007/978-3-642-54593-1.

[17] Nitish Srivastava et al. "Dropout: a simple way to prevent neural networks from overfitting". In: *J. Mach. Learn. Res.* 15.1 (Jan. 2014), pp. 1929–1958. ISSN: 1532-4435.

[18] Kaiming He et al. *Deep Residual Learning for Image Recognition*. 2015. arXiv: 1512.03385 [cs.CV]. URL: https://arxiv.org/abs/1512.03385.

[19] Steven L Brunton, Joshua L Proctor, and J Nathan Kutz. "Discovering governing equations from data by sparse identification of nonlinear dynamical systems". en. In: *Proc. Natl. Acad. Sci. U. S. A.* 113.15 (Apr. 2016), pp. 3932–3937.

[20] Joseph Redmon et al. *You Only Look Once: Unified, Real-Time Object Detection*. 2016. arXiv: 1506.02640 [cs.CV]. URL: https://arxiv.org/abs/1506.02640.

[21] Jaideep Pathak et al. "Using machine learning to replicate chaotic attractors and calculate Lyapunov exponents from data". In: *Chaos: An Interdisciplinary Journal of Nonlinear Science* 27.12 (Dec. 2017). ISSN: 1089-7682. DOI: 10.1063/1.5010300. URL: http://dx.doi.org/10.1063/1.5010300.

[22] James Bradbury et al. *JAX: composable transformations of Python+NumPy programs*. Version 0.3.13. 2018. URL: http://github.com/jax-ml/jax.

[23] Ricky T. Q. Chen. *torchdiffeq*. 2018. URL: https://github.com/rtqichen/torchdiffeq.

[24] Ricky T. Q. Chen et al. "Neural Ordinary Differential Equations". In: *Advances in Neural Information Processing Systems*. Ed. by S. Bengio et al. Vol. 31. Curran Associates, Inc., 2018. URL: https://proceedings.neurips.cc/paper_files/paper/2018/file/69386f6bb1dfed68692a24c8686939b9-Paper.pdf.

[25] S.H. Strogatz. *Nonlinear Dynamics and Chaos: With Applications to Physics, Biology, Chemistry, and Engineering*. CRC Press, 2018. ISBN: 978-0-429-96111-3. URL: https://books.google.it/books?id=A0paDwAAQBAJ.

[26] Takuya Akiba et al. "Optuna: A Next-generation Hyperparameter Optimization Framework". In: *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. 2019.

[27] Christopher M. Bishop. *Pattern recognition and machine learning*. Information Science and Statistics. New York, NY: Springer Science+Business Media, LLC, 2019. 758 pp. ISBN: 978-0-387-31073-2.

[28] Adam Paszke et al. "PyTorch: An Imperative Style, High-Performance Deep Learning Library". In: *Advances in Neural Information Processing Systems 32*. Curran Associates, Inc., 2019, pp. 8024–8035. URL: http://papers.neurips.cc/paper/9015-pytorch-an-imperative-style-high-performance-deep-learning-library.pdf.

[29] M. Raissi, P. Perdikaris, and G.E. Karniadakis. "Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations". en. In: *Journal of Computational Physics* 378 (Feb. 2019), pp. 686–707. ISSN: 00219991. DOI: 10.1016/j.jcp.2018.10.045. URL: https://linkinghub.elsevier.com/retrieve/pii/S0021999118307125.

[30] Kathleen Champion et al. "A Unified Sparse Optimization Framework to Learn Parsimonious Physics-Informed Models From Data". In: *IEEE Access* 8 (2020), pp. 169259–169271. DOI: 10.1109/ACCESS.2020.3023625.

[31] DeepMind et al. *The DeepMind JAX Ecosystem*. 2020. URL: http://github.com/google-deepmind.

[32] Charles R. Harris et al. "Array programming with NumPy". In: *Nature* 585.7825 (Sept. 2020), pp. 357–362. DOI: 10.1038/s41586-020-2649-2. URL: https://doi.org/10.1038/s41586-020-2649-2.

[33] Stefan Leijnen and Fjodor van Veen. "The Neural Network Zoo". In: *Proceedings* 47.1 (2020). ISSN: 2504-3900. DOI: 10.3390/proceedings2020047009. URL: https://www.mdpi.com/2504-3900/47/1/9.

[34] Christopher Rackauckas et al. *Universal Differential Equations for Scientific Machine Learning*. 2020. arXiv: 2001.04385 [cs.LG].

[35] Brian de Silva et al. “PySINDy: A Python package for the sparse identification of nonlinear dynamical systems from data”. In: *Journal of Open Source Software* 5.49 (2020), p. 2104. DOI: 10.21105/joss.02104. URL: https://doi.org/10.21105/joss.02104.

[36] Marcin Szczepański. “Is data the new oil?” In: 2020. URL: https://api.semanticscholar.org/CorpusID:211095343.

[37] The pandas development team. *pandas-dev/pandas: Pandas*. Version latest. Feb. 2020. DOI: 10.5281/zenodo.3509134. URL: https://doi.org/10.5281/zenodo.3509134.

[38] Pauli Virtanen et al. “SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python”. In: *Nature Methods* 17 (2020), pp. 261–272. DOI: 10.1038/s41592-019-0686-2.

[39] George Em Karniadakis et al. “Physics-informed machine learning”. In: *Nature Reviews Physics* 3.6 (May 2021), pp. 422–440. ISSN: 2522-5820. DOI: 10.1038/s42254-021-00314-5. URL: http://dx.doi.org/10.1038/s42254-021-00314-5.

[40] Patrick Kidger. “On Neural Differential Equations”. PhD thesis. University of Oxford, 2021.

[41] Patrick Kidger and Cristian Garcia. “Equinox: neural networks in JAX via callable PyTrees and filtered transformations”. In: *Differentiable Programming workshop at Neural Information Processing Systems 2021* (2021).

[42] Michael L. Waskom. “seaborn: statistical data visualization”. In: *Journal of Open Source Software* 6.60 (2021), p. 3021. DOI: 10.21105/joss.03021. URL: https://doi.org/10.21105/joss.03021.

[43] Nicolas Brunel and Vincent Hakim. “Neuronal Dynamics”. In: *Statistical and Nonlinear Physics*. Ed. by Bulbul Chakraborty. New York, NY: Springer US, 2022, pp. 495–516. ISBN: 978-1-0716-1454-9. DOI: 10.1007/978-1-0716-1454-9_359. URL: https://doi.org/10.1007/978-1-0716-1454-9_359.

[44] Alan A. Kaptanoglu et al. “PySINDy: A comprehensive Python package for robust sparse system identification”. In: *Journal of Open Source Software* 7.69 (2022), p. 3994. DOI: 10.21105/joss.03994. URL: https://doi.org/10.21105/joss.03994.

[45] Kevin P. Murphy. *Probabilistic Machine Learning: An introduction*. MIT Press, 2022. URL: http://probml.github.io/book1.

[46] Matteo Sangiorgio, Fabio Dercole, and Giorgio Guariso. *Deep learning in multi-step prediction of chaotic dynamics*. en. 1st ed. SpringerBriefs in Applied Sciences and Technology. Cham, Switzerland: Springer Nature, Feb. 2022.

[47] Ashish Vaswani et al. *Attention Is All You Need*. 2023. arXiv: 1706.03762 [cs.CL]. URL: https://arxiv.org/abs/1706.03762.

[48] Jason Rader, Terry Lyons, and Patrick Kidger. "Optimistix: modular optimisation in JAX and Equinox". In: *arXiv:2402.09983* (2024).

[49] Philip K. Shiu et al. "A Drosophila computational brain model reveals sensorimotor processing". en. In: *Nature* 634.8032 (Oct. 2024), pp. 210–219. ISSN: 1476-4687. DOI: 10.1038/s41586-024-07763-9. URL: https://www.nature.com/articles/s41586-024-07763-9.

[50] Rasha M. Yaseen et al. "Effect of the Fear Factor and Prey Refuge in an Asymmetric Predator–Prey Model". In: *Brazilian Journal of Physics* 54.6 (Sept. 2024). ISSN: 1678-4448. DOI: 10.1007/s13538-024-01594-9. URL: http://dx.doi.org/10.1007/s13538-024-01594-9.

[51] Anthony G. Chesebro et al. *Scientific Machine Learning of Chaotic Systems Discovers Governing Equations for Neural Populations*. 2025. arXiv: 2507.03631 [cs.LG]. URL: https://arxiv.org/abs/2507.03631.

[52] Wenjing Zhang, Wandi Ding, and Huaiping Zhu. *Universal differential equations for optimal control problems and its application on cancer therapy*. 2025. arXiv: 2504.16035 [math.OC]. URL: https://arxiv.org/abs/2504.16035.

[53] Isaac Robinson et al. *RF-DETR: Neural Architecture Search for Real-Time Detection Transformers*. 2026. arXiv: 2511.09554 [cs.CV]. URL: https://arxiv.org/abs/2511.09554.